

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE



论文题目 基于云架构协同的多域柔性建模技术

专业学位类别	电子信息
学 号	202122010218
作者姓名	曹宇茜
指导教师	王 坚 教 授
学 院	信息与通信工程学院

分类号 N945.12 密级 公开
UDC ^{注1} 621.39

学 位 论 文

基于云架构协同的多域柔性建模技术

(题名和副题名)

曹宇茜

(作者姓名)

指导教师 王 坚 教 授
电子科技大学 成 都
(姓名、职称、单位名称)

申请学位级别 硕士 专业学位类别 电子信息
专业学位领域 通信工程（含宽带网络、移动通信等）
提交论文日期 2024 年 4 月 12 日 论文答辩日期 2024 年 5 月 28 日
学位授予单位和日期 电子科技大学 2024 年 6 月
答辩委员会主席 林水生
评阅人 白智元 李桓

注 1：注明《国际十进分类法 UDC》的类号。

Multi-domain Flexible Modeling Technique Based on Cloud Architecture Collaboration

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline **Electronic Information**

Student ID **202122010218**

Author **Yuqian Cao**

Supervisor **Prof. Jian Wang**

School **School of Information and Communication**

Engineering

摘要

随着系统的复杂性越来越高，基于模型的系统工程建模要求满足各种建模语言及建模规范、建模具有精细颗粒度、支持多领域专家协同建模等功能。因此，本文研究了基于云架构协同的多域柔性建模技术，用于解决针对复杂系统进行基于模型的系统工程建模时领域描述能力不足、异构数据互操作能力低、多领域建模存在隔阂等问题。

首先，本文提出基于 GOPPRRE(Graph, Object, Point, Property, Relationship, Role, Extension) 的建模方法，结合系统所需建模语言或建模规范进行元模型库的构建，并且基于元模型库构建模型。前端采用 mxGraph 支持柔性建模渲染及敏捷交互，后端采用 SpringMVC 框架支持柔性建模数据处理。基于柔性建模技术结合建模语言或建模规范构建各类元模型库，验证柔性建模技术对各类建模语言及建模规范的兼容性。通过针对海上应急救援工程进行柔性建模验证本文柔性建模技术的可行性及有效性。通过 OntoQA 方法针对柔性建模项目导出本体进行定量分析，验证本文柔性建模技术具有精准颗粒度及较强领域描述能力。

其次，本文在柔性建模技术基础上，研究了基于 Convergence 协同引擎的多域柔性建模技术。通过基于柔性建模模型与 JSON 格式数据相互转换的方案实现基于协同域的多人实时在线柔性建模，通过 MySQL 数据库及 Convergence 协同引擎实现用户项目管理。提出了采用协同冲突处理算法及用户标记共享解决协同操作中冲突问题。针对柔性建模平台进行 Docker 镜像化处理，并在云服务器进行部署。

最后，通过本文的基于云架构协同的多域柔性建模平台，针对海上应急救援工程进行多域柔性建模，证明多域柔性建模的可行性及有效性。并且通过对不同复杂度模型进行操作同步的时延测试，证明多域柔性建模技术具备低时延性能。通过使用本文基于云架构协同的多域柔性建模平台进行海上应急救援案例构建，验证本文提出的基于云架构协同的多域柔性建模技术具备可行性及有效性。

关键词：系统工程建模，多域柔性建模，云架构，GOPPRRE 方法

ABSTRACT

Due to the growing complexity of systems, model-based systems engineering modeling requires various functions, such as meeting different modeling languages and specifications, modeling with fine granularity, and supporting collaborative modeling by multi-domain experts. This thesis investigates a flexible multi-domain modeling technique based on cloud architecture, which is used to solve the problems of insufficient domain description capability, low interoperability of heterogeneous data, and misunderstanding of multi-domain modeling in model-based systems engineering modeling for complex systems.

This thesis proposes constructing metamodel libraries using the GOPPRRE(Graph, Object, Point, Property, Relationship, Role, Extension) modeling method and the required modeling language or specification. Models will be built based on these metamodel libraries. The front-end utilizes mxGraph for flexible modeling rendering and agile interaction, while the back-end employs the SpringMVC framework for flexible modeling data processing. Construct metamodel libraries using flexible modeling technology combined with modeling languages or specifications. Verify compatibility of the technology with various modeling languages and specifications. The thesis verifies the feasibility and validity of the flexible modeling technology of this thesis through flexible modeling for maritime emergency rescue projects. The OntoQA method is used to quantitatively analyze the ontology derived from the flexible modeling project, confirming the technology's accurate granularity and strong domain description capability.

This thesis explores the multi-domain flexible modeling technology based on Convergence Collaboration Engine based on flexible modeling technology. The scheme is based on the inter-conversion of flexible modeling models and JSON format data to achieve multi-person online real-time flexible modeling and project management based on the Convergence domain. The OT algorithm and user tag sharing are proposed to solve the conflict problem in collaborative operation. The flexible modeling platform is made as a Docker image and deployed on a cloud server.

Finally, this thesis studies the multi-domain flexible modeling for maritime emergency rescue engineering through the cloud architecture collaboration multi-domain flexible modeling platform, which proves the feasibility and effectiveness of multi-domain

flexible modeling. The delay test of operation synchronization for different complexity models proves the low delay performance of the multi-domain flexible modeling technology. The thesis demonstrates the feasibility and effectiveness of the cloud architecture collaboration multi-domain flexible modeling technology proposed in this thesis through the construction of maritime emergency rescue case on the cloud architecture collaboration multi-domain flexible modeling platform.

Keywords: System Engineering Modeling, Multi-domain Flexibility Modeling, Cloud Architecture, GOPPRRE Methodology

目 录

第一章 绪 论	1
1.1 研究工作的背景与意义	1
1.2 国内外研究现状	2
1.2.1 基于云架构柔性建模技术研究现状	2
1.2.2 多域技术在基于云架构柔性建模技术的应用	4
1.3 本文结构安排	5
第二章 基于云架构多域柔性建模相关理论与技术	7
2.1 云架构技术	7
2.2 柔性建模技术	8
2.2.1 基于 IBM Harmony SE 的建模技术	9
2.2.2 基于 No Magic MagicGrid 方法论的建模技术	9
2.2.3 基于 GOPPRRE 建模方法的建模技术	11
2.3 多域建模技术	15
2.3.1 多域建模技术协同方案分析	15
2.3.2 多域建模技术协同引擎	16
2.3.3 多域建模技术渲染工具	17
2.4 本章小结	20
第三章 基于 GOPPRRE 建模方法的柔性建模技术研究	21
3.1 现有建模技术问题分析	21
3.2 柔性建模渲染方法	22
3.2.1 支持柔性建模的 mxGraph 核心类说明	22
3.2.2 KARMA 模型基础规则及关键信息处理	26
3.2.3 基于关键信息柔性建模渲染	30
3.3 基 GOPPRRE 建模方法的柔性建模技术	38
3.3.1 柔性建模技术总体架构	38
3.3.2 柔性建模技术基本原理	39
3.3.3 柔性建模方法技术实现	43
3.4 应急救援工程的柔性建模与分析	47
3.4.1 案例背景	47
3.4.2 模型构建	48

3.4.3 柔性建模性能分析	53
3.5 本章小结	55
第四章 基于云架构的多域柔性建模技术研究	56
4.1 多域建模技术研究	56
4.1.1 多域建模技术总体架构	56
4.1.2 多域建模技术基本原理	58
4.1.3 多域建模技术方案实现	63
4.2 协同冲突问题处理	65
4.2.1 基于分布式 OT 技术的协同冲突处理	65
4.2.2 基于标记共享的协同冲突处理	67
4.3 基于云架构的多域柔性建模实现	69
4.3.1 建模平台 Docker 镜像化处理	69
4.3.2 针对 Docker 镜像进行云服务器部署	72
4.4 基于云架构协同的应急救援工程多域建模及分析	72
4.4.1 多域建模流程	72
4.4.2 基于云架构多域柔性建模性能分析	77
4.5 本章小结	80
第五章 全文总结与展望	81
5.1 全文总结	81
5.2 后续工作展望	81
致 谢	83
参考文献	84
攻读专业硕士学位期间取得的成果	88

图目录

图 1-1 GME 架构 ^[1]	3
图 1-2 MS DSL Tools 架构 ^[1]	4
图 1-3 MS Visio 元元模型架构 ^[1]	4
图 2-1 云技术计算架构	7
图 2-2 Docker 架构	8
图 2-3 Harmony SE 流程图	10
图 2-4 MagicGrid 框架	11
图 2-5 M3-M0 层建模要素	12
图 2-6 M3 层元元模型	13
图 2-7 KARMA 建模语言	14
图 2-8 diff 算法	15
图 2-9 协同平台架构图	17
图 2-10 mxGraph 架构	18
图 2-11 图形可视化核心类	18
图 2-12 mxGraphModel、mxCell 及 mxGeometry 关联	19
图 2-13 图形编辑核心类	19
图 3-1 relative 说明	23
图 3-2 基于 mxGraph 渲染分解剖视图层级	24
图 3-3 KARMA 语言基础数据类型	27
图 3-4 KARMA 类架构	28
图 3-5 项目/语言/包 KARMA 建模语言	28
图 3-6 属性/对象/关系/角色/点/图元模型 KARMA 建模语言	29
图 3-7 KARMA 模型中的普通图信息	31
图 3-8 JSON 格式 KARMA 模型关键信息	32
图 3-9 分解剖视模型展示	33
图 3-10 分解图后端处理流程	34
图 3-11 剖视图后端数据处理	35
图 3-12 分解剖视图前端渲染流程	36
图 3-13 分解剖视图实现	37

图 3-14 智能电动汽车时序图模型	37
图 3-15 柔性建模技术总体架构	38
图 3-16 柔性建模技术性能架构	39
图 3-17 柔性建模技术原理框架	40
图 3-18 对象添加流程	41
图 3-19 关系添加验证流程	42
图 3-20 React 生命周期	43
图 3-21 基于 React 前端页面组件	44
图 3-22 SpringMVC 操作流程	45
图 3-23 对象关系添加实现	46
图 3-24 柔性建模自动布局实现	47
图 3-25 英国救援体系概述	48
图 3-26 属性元模型概览	50
图 3-27 点元模型概览	50
图 3-28 对象元模型概览	51
图 3-29 角色元模型概览	51
图 3-30 关系元模型	52
图 3-31 图元模型概览	53
图 4-1 用户组织项目管理架构	57
图 4-2 多人实时协同建模架构	57
图 4-3 协同中心项目管理	59
图 4-4 协同中心各模块	60
图 4-5 mxGraph 类实例化对象与 JSON 格式转换	61
图 4-6 工作组管理界面	63
图 4-7 项目管理界面	64
图 4-8 协同引擎服务	64
图 4-9 OT 算法	66
图 4-10 OT 客户端状态转移	66
图 4-11 冲突操作解决效果	66
图 4-12 选择框管理构建流程	68
图 4-13 标记共享展示	69
图 4-14 镜像架构	70
图 4-15 docker-compose 架构	71

图 4-16 全景视点模型	73
图 4-17 AV-1 概要模型	73
图 4-18 能力视点模型	74
图 4-19 能力视点模型实例化	74
图 4-20 作战视点模型	75
图 4-21 作战视点模型实例化	75
图 4-22 服务视点模型	76
图 4-23 服务视点模型实例化	76
图 4-24 系统视点模型	77
图 4-25 系统视点模型实例化	77
图 4-26 多域建模技术稳定性测试	78

表目录

表 1-1	主流元建模语言分析	4
表 2-1	OT 算法与 CRDT 算法对比	16
表 3-1	主流公司建模技术可支持建模语言	21
表 3-2	主流公司建模技术可支持文件格式	22
表 3-3	mxCell 类函数及参数说明	22
表 3-4	mxLayoutManager 类变量及函数	24
表 3-5	mxStackLayout 类变量及函数	25
表 3-6	mxMultiplicity 类函数及参数	26
表 3-7	mxGraphHandler 类函数及参数	26
表 3-8	元模型数量统计	48
表 3-9	属性元模型	49
表 3-10	点元模型	49
表 3-11	对象元模型	50
表 3-12	角色元模型	51
表 3-13	关系元模型	52
表 3-14	图元模型	52
表 3-15	柔性建模技术元模型库	54
表 3-16	柔性建模技术本体数据类型统计	54
表 3-17	柔性建模技术性能	55
表 4-1	普通图柔性多域建模技术延时分析（单位/ns）	79
表 4-2	分解剖视图子节点延时分析（单位/ns）	79
表 4-3	分解剖视图父节点延时分析（单位/ns）	79
表 4-4	时序图延时分析（单位/ns）	80
表 4-5	多域柔性建模平均时延	80

第一章 绪论

1.1 研究工作的背景与意义

体系建模作为理解、设计、优化和管理复杂系统的重要手段,帮助参与者理解和分析系统、进行系统设计和优化、辅助参与者进行决策支持和策略优化^[2]。通过上述内容,提高系统的性能、系统搭建效率及可靠性。美国从冷战时期开始对复杂军事体系建模,并且将体系建模技术运用于军事训练与战场模拟、战争规划与军事决策、资源管理与物流、城市规划等领域,取得不错的成果^[3-5]。因此,建模技术是提升系统性能、系统搭建效率及可靠性的重要途径。

随着信息技术的发展,目前复杂体系中数字化对象愈发复杂,复杂体系由多个子系统组成,各子系统涉及各个领域^[6]。例如,在智慧交通体系中,包括交通控制、车辆管理、信息通信网络 and 用户需求等多个子系统,各子系统之间的相互关联^[7]。在作战体系中,包含指挥与控制、战术领域、技术装备、情报与侦察、物流与保障等多个子系统,子系统之间相互关联^[8]。由于体系建模从传统单领域、单层次到多领域、多层次的演化,多域柔性建模成为未来发展的必然趋势^[9]。

2006 年亚马逊推出了 Amazon Web Service^[10],通过为用户提供基础设施即服务 (Infrastructure as a Service, IaaS) 为云平台建模奠定基础,2007 年谷歌发布了 App Engine^[11],为开发者提供构建和托管 Web 应用程序平台,推动云架构体系建模的发展。之后,美国国防部先后公布云计算战略、云计算安装指南等,并推动基于云的军事建模与仿真 (Cloud-Based Military Modeling and Simulation, CBMS) 项目^[12],提升建模的协同能力和实时性。因此将体系建模技术与云架构技术进行融合具有极高的研究价值。

各个国家基于云架构针对不同建模框架进行体系建模,开发多种基于云架构的建模平台,目前基于云架构的建模平台受限于特定建模语言及建模框架等问题,主要针对特定领域进行建模。对于基于云架构的协同建模研究由美国最先展开,进行云平台建设、跨领域协同及标准与规范制定,并且成功应用于医疗保健等领域。但目前协同建模云平台多采用国外云服务,因此,针对涉密体系进行建模会存在极高泄密风险。此外,现有基于云架构协同建模平台存在针对特定领域特征化功能描述能力不足的问题。因此,研发独立自主的基于云架构协同建模平台具有必要性。

本文在 GOPPRRE 建模方法^[13]及多架构建模语言 (Kombination of Architecture Model Specification, KARMA)^[14]基础上,采用 mxGraph 图形库作为前端渲染工

具,支持基于 KARMA 语言进行基于模型的系统工程建模,支持创建元模型库并且基于元模型库构建模型实现多层次高精度建模。mxGraph 提供强大的应用程序编程接口和工具,支持自动布局、规则验证及敏捷交互等功能,支持基于模型的系统工程柔性建模^[15]。并且 mxGraph 支持将模型信息转化为 JSON 格式数据,适合后续基于云架构协同的多域建模技术中协同建模功能开发。

在基于模型的系统工程柔性建模基础上,本文基于 Convergence 协同引擎进行扩展开发,构建一个支持多域柔性建模的平台,该平台具备针对基于模型的系统工程进行柔性建模、多人实时协同建模及平台管理等功能。将多域柔性建模平台进行镜像化处理,通过云平台进行快速部署和扩展,实现多域柔性建模平台资源隔离并且提高多域柔性建模平台安全性。

1.2 国内外研究现状

1.2.1 基于云架构柔性建模技术研究现状

柔性建模技术通过元对象机制增强对元对象和元数据的利用率,提高柔性建模技术的扩展性、互操作性和开放性^[1]。元建模语言是柔性建模技术的基石,元建模语言由元元模型描述,可以定义元模型。下面针对目前常用的元建模语言进行介绍。

集成信息系统结构 (Architecture of Integrated Information System, ARIS)^[16]是由德国萨尔大学 Scheer 教授于 1992 年提出,该系统结构面向过程进行建模,其架构包括组织视图、输出视图、功能视图、数据视图和控制视图,其中组织视图、输出视图、功能视图、数据视图支持从四个方面独立且全面进行模型构建,通过控制视图描述上述四个视图之间的关系,将其进行关联。综上 ARIS 可减少问题描述的冗余,并且为实现分布式协同建模提供基础。ARIS 主要针对企业架构进行模型构建。

GOPPRR^[17] (Graph, Object, Point, Property, Relationship, Role) 建模方法是基于 M0-M3 建模架构,围绕图、对象、属性、点、关系、角色六种元素进行构建, GOPPRR 建模方法需构建元模型库,元模型库主要针对点、对象、属性、关系、角色及图进行元模型构建。

Ecore^[18]是 eclipse 建模框架 (Eclipse Modeling Framework, EMF) 的元元模型,支持基于 Ecore 使用 Eclipse 进行领域特定语言建模, Ecore 属于通用建模语言 (Unified Modeling Language, UML) 的子集,其包括 EClass、EAttribute、EReference 及 EDataType,分别描述领域类、领域类的属性、领域类之间建立的关联及属性类型。Ecore 通过提供根对象表示模型,根对象的子对象为 package, package 的子节

点为 class，class 的子节点为 attribute。通过上述基于根节点的树形结构可以表示 EMF 的模型结构。

通用建模环境（Generic Modeling Environment，GME）^[19]可以使用 GME 元建模语言进行特定建模语言的构建。GME 支持元模型构建，基于元模型构建进行业务模型的构建，GME 提供了 Atom、Model、Reference、FCO、Connector 及 Constraint 元元模型，基于上述元元模型构建 Product、Role、Property 及 Connection，其具体架构如图1-1所示。

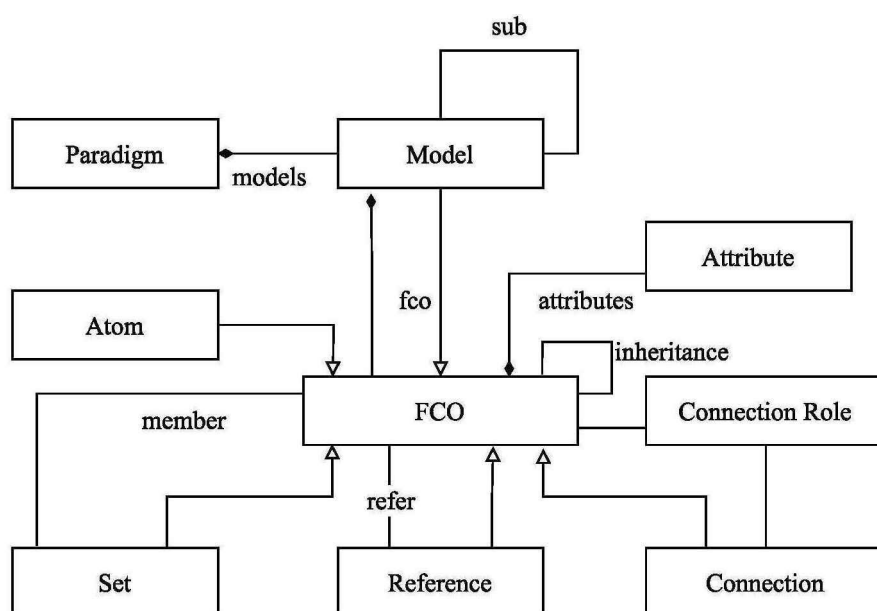


图 1-1 GME 架构^[1]

MS DSL Tools^[20]支持领域特定语言建模，其元元模型如图1-2所示。基于图中元元模型，支持领域特定语言设计、代码生成、集成到开发环境、支持模型驱动开发及自定义扩展。通过以上功能辅助开发人员灵活且快速建模。

Microsoft Visio^[21]元元模型如图1-3所示，主要针对流程图、组织结构图及平面图进行构建，支持数据连接、协作与共享及自定义功能，但缺乏特定领域的元模型库构建。

表1-1针对上述元建模语言支持的属性进行进一步分析。

据表1-1可得，目前主流的元建模语言中，GOPPRR 元建模语言拥有较强柔性，但仍存在描述不精准等问题，因此本文提出 GOPPRRE 元建模语言，并且基于该建模语言进行柔性建模。GOPPRRE 元建模语言在 GOPPRR 元建模语言基础上增加了元素之间的约束，增强了元建模语言描述的精确性，通过将约束应用于建模过程中，实现敏捷建模。

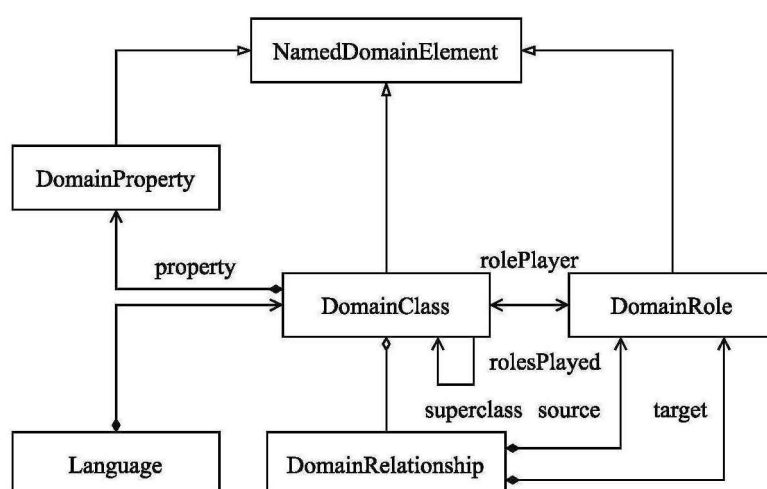


图 1-2 MS DSL Tools 架构^[1]

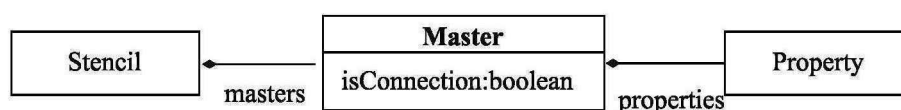


图 1-3 MS Visio 元元模型架构^[1]

表 1-1 主流元建模语言分析

	ARIS	Ecore	GOPRR	GME	MS DSL Tools	MS Visio
Multiplicity	single-value	multi-value	multi-value	single-value	single-value	multi-value
Unique	-	√	√	-	-	×
Ordered	-	√	×	-	-	√
Default value	×	√	√	√	√	√
Object	√	√	√	√	√	√
Relationship	√	×	√	√	√	√
Role	-	-	√	×	×	-
Port	-	-	√	×	-	-
Model	√	-	√	√	-	-
Simple	√	√	√	√	√	√
Metamodel element	×	×	√	×	×	×

1.2.2 多域技术在基于云架构柔性建模技术的应用

多域技术主要包含协同建模及用户管理，其中协同主要基于 B/S 端或 C/S 端针对柔性模型数据进行同步，并且实时同步各户用操作。

Leslie Lamport^[22]于 1978 年在其关于分布式系统基本事件关系中提出实施协同编辑系统是实施协同工作系统重要部分，并且强调人人交互和人机交互的重要性^[22-26]。针对跨领域复杂系统，需要多领域专家进行协同构建，协同系统可减少多领域专家协同建模时的理解偏差，Ellis 等人^[27]于 1989 年提出的协同编辑系统

具备较好的实践功能。在后续发展中,出现了 Google Wave、Codoxware 及 IBM OpenCoWeb 等协同应用。

解决协同过程中操作冲突问题是实现一个优秀协同技术的关键,目前常见的操作冲突解决技术包括锁机制^[28]、操作转换(Operational Transformation, OT)算法、地址空间转换(Address Space Transformation, AST)算法及无冲突可复制数据类型(Conflict-free Replicated Data Type, CRDT)算法。

通过锁机制解决协同冲突效率低下且无法完全实现协同操作的问题。Ellis 和 Gibbs^[27]于 1989 年提出 OT 算法及该算法基本模型,通过进行操作转换最后达成一致性结果。后续 Ressel、Li 及 Sun 等人^[25,29-40]继续针对 OT 算法进行研究,但 OT 算法无法很好适用于所有场景。Ning Gu 等人针对 AST 算法进行相关研究,AST 算法通过实现文档地址空间的转换,支持将文档回退到操作前初始状态的位置。与 OT 算法相比,AST 算法改变了获取操作执行位置的方式。法国机构 INRIA 提出了一系列 CRDT 算法,CRDT 算法通过给每个操作对象划分唯一标识符,支持交换执行每个并发操作。与 OT 算法及 AST 算法相比,在操作上 CRDT 无需进行操作的转换,并且不需要针对操作历史进行保存。在性能上,CRDT 算法具备更好的伸缩性,但是 CRDT 算法具有较大的开销。

目前国内外常见的协同建模工具有 Miro 和 BoardMix,其中国外协同建模工具 Miro 可以支持研究与设计、敏捷工作流、战略与规划、绘图和图解等功能,其主要针对业务流程进行建模,无法支持特定领域的复杂系统柔性建模。国内协同建模工具 BoardMix 支持项目管理、产品设计、策略与咨询等功能,但其产品设计仅停留于图形化层面,无法深入特定领域进行柔性建模。综上,目前缺乏支持柔性建模的多域建模平台,因此针对多域柔性建模技术研究具有重要价值。

1.3 本文结构安排

本文各章节主要内容安排如下:

第一章介绍了研究基于云架构多域柔性建模技术的背景与意义,分别从柔性建模技术、多域建模技术两方面介绍国内外研究现状,阐述现有多域柔性建模技术的不足,说明本文所研究技术的必要性及先进性。

第二章研究了基于云架构协同的多域柔性建模技术的基础理论研究及相关技术和工具,为本文后续的研究提供理论基础。

第三章研究了基于 GOPPRRE 建模方法的柔性建模技术。首先介绍现有柔性建模技术问题,阐述基于 GOPPRRE 建模方法进行柔性建模原理及流程,并且基于柔性建模技术结合建模语言或建模框架进行元模型库及模型构建,验证柔性建

模技术的兼容性。通过 OntoQA 方法针对海上救援及智能电动汽车案例本体进行定量分析，验证柔性建模技术的优良性能。

第四章研究了基于云架构的多域柔性建模技术。首先介绍了多域建模技术实现原理及方法，针对多域柔性建模技术操作冲突提出两种解决方案。将多域柔性建模技术部署至云平台，并且通过基于云架构协同的多域柔性建模技术进行海上应急救援工程的模型构建，针对基于云平台的多域柔性建模技术进行性能分析，验证了本文技术的性能。

第五章针对本文主要研究内容进行总结，同时对本文研究过程中遇到的问题及后续的研究方向进行进一步说明。

第二章 基于云架构多域柔性建模相关理论与技术

上一章针对多域柔性建模技术的国内外研究现状进行分析。本章介绍了主流的柔性建模技术，后续针对柔性建模技术的研究将通过基于 GOPPRRE 建模方法的柔性建模技术进行展开。

为实现多域柔性建模技术，本章针对多域建模技术的协同方案、协同引擎及协同工具进行详细介绍。

为实现基于云架构的多域柔性建模技术，本章针对现阶段云架构的组织结构及如何基于 Docker 进行云平台部署进行说明。

2.1 云架构技术

云服务^[41]是指通过互联网提供数据库、存储、网络 and 计算能力等各种计算服务。与物理服务器相比，云服务器的部署方式、管理方式、弹性扩展、成本及可靠性等具备优势。基于虚拟化技术实现的云服务器可支持基于云平台的快速创建及部署，部署方式更加方便。云服务器可基于云平台进行监控及自动化管理，无需手动管理和维护，管理方式更加智能。云服务器的弹性扩展无需购买新的硬件设备，仅通过业务需求可进行弹性扩展。成本方面，云服务器相较于可共享硬件资源成本相对较低。最重要的是云服务器通过多数据中心进行部署及资源备份，拥有更强的可靠性。

云计算作为云服务核心内容，其主要架构如下图所示。

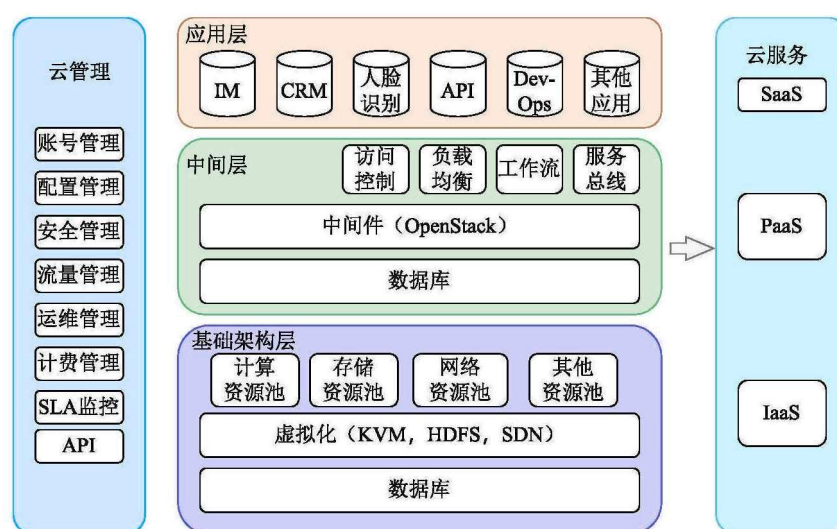


图 2-1 云技术计算架构

云计算总体架构主要分为应用层、中间层和基础架构层^[42]，其中应用层是软件即服务，通过以 Web 的方式给客户提供应用。中间层是平台即服务，该层提供给客户的服务是应用的开发和部署平台。基础架构层是基础设施即服务，该层提供给用户的服务是各种的底层计算和存储资源。Docker^[43]是支持将应用程序及依赖打包为独立容器的开源应用容器引擎，遵从 Apache2.0 的协议并且基于 Go 语言，基于 Docker 的镜像可发布至 Linux 机器中。支持可扩展、轻量级及可移植的软件交付。Docker 架构如图2-2所示。

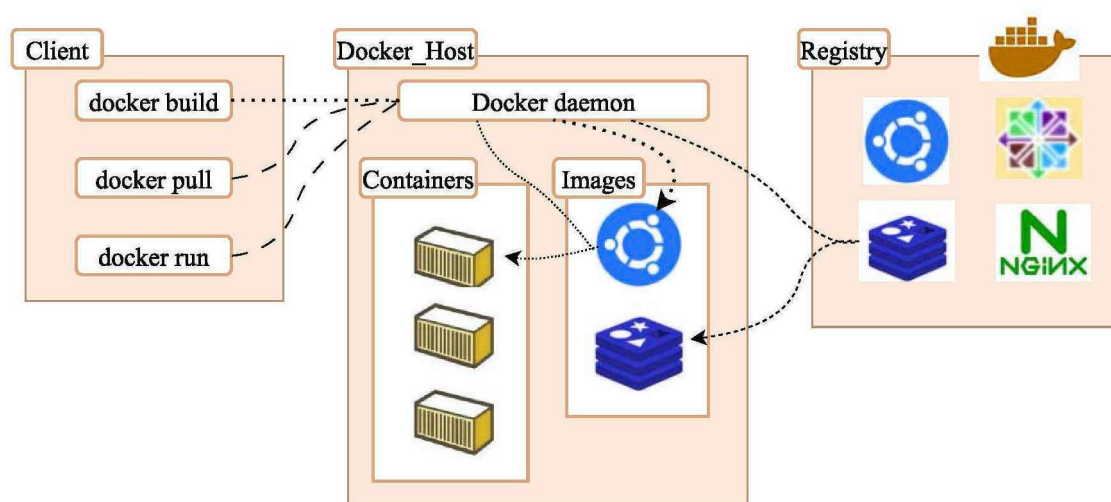


图 2-2 Docker 架构

综上所述，在云服务中部署 Docker 镜像可结合各自优点，具备弹性扩展、简化部署、环境的一致性及便捷开发和测试等优势，通过云平台基于应用负载情况调整资源用量，结合 Docker 快速启动，实现应用灵活扩展和缩减。将 Docker 部署在云平台从而实现基于云平台进行容器的监管、控制、创建、启动和停止。同时开发人员可在本地开发环境中进行应用程序的构建和测试，并且基于云平台进行生产部署。实现可靠性更强、安全性更高、效率更高的云架构建模平台。本文研究的基于云架构协同的多域柔性建模平台在柔性建模技术基础上，结合协同引擎实现多人协同建模及项目用户管理，最后通过镜像部署至云平台。

2.2 柔性建模技术

柔性建模技术主要针对多领域、多层次的复杂系统建模，通过采用 MOF 元对象机制，结合各类建模语言或建模框架构建元模型库，基于元模型库进行模型构建。柔性主要体现为包容开放性、灵活适应性、系统整体性及支持敏捷建模。本节主要针对目前常见的两种建模方法及本文采用的柔性建模方法分别进行介绍。

2.2.1 基于 IBM Harmony SE 的建模技术

IBM 公司提出了 IBM Harmony SE 方法论^[44], IBM Harmony SE 方法论采用系统建模语言 (System Modeling Language, SysML) 作为建模语言。该方法论包括了需求分析、功能分析和架构分析与设计三部分流程。在层次方面, 包括了需求、功能和架构三个层次。并且强调从 SysML 的活动图到顺序图到状态图三种行为图的依次建模, 一定程度减弱了参数图的重要性。

SysML^[45]是基于 UML 发展而来的用于描述系统工程和硬件系统的系统建模语言, 于 2003 年由对象管理组织 (Object Management Group, OMG) 的参与者提出。SysML 主要包括模块定义图、内部模块图、用例图、活动图、时序图、状态机图、参数图、包图及需求图等。SysML 是基于 UML 的扩展, 通过提供针对系统工程的图形表示和建模元素增强了对系统工程的语义表达。在支持系统行为、结构和交互建模基础上, 增加了功能、信号流、需求及物理架构等方面的建模。并且提供了可扩展机制, 支持用户根据需求自定义建模元素及图表。基于以上优点, SysML 成为了现在系统工程领域重要的建模语言。

该方法论在构建基于模型的系统工程时的关键目标是, 首先确认工程中系统需求, 其次明确系统的状态和模式, 最后针对上述系统状态和模式进行拆分, 分配到子系统中。

该建模技术采用自上向下的基于高层的抽象建模。子系统重点在于通过需求确认须实现的功能, 并且实现基于状态的行为。不对功能行为的细节进行约束, 图2-3阐述了基于 Harmony SE 方法论。

首先是需求分析阶段, 将涉众需求转化为功能性需求和非功能性需求等系统需求。基于需求的定义建立需求库, 并且通过理解需求将需求组合成用例。其次是功能分析, 功能分析是将系统的功能性需求转化为对应的系统功能, 并且通过在需求阶段基于需求用例产生可以执行的模型, 该模型用于理解需求、确认需求和验证需求。在设计综合阶段, 需要设计满足功能性需求和非功能性需求的架构, 从而指导系统开发。

IBM Harmony SE 方法论突出建模驱动、统一性、迭代和递增等关键部分, 有助于开发团队通过建立合理开发流程, 提高开发效率、质量和可维护性, 从而推动系统开发过程的规范化。

2.2.2 基于 No Magic MagicGrid 方法论的建模技术

美国 No Magic 公司提出的 No Magic MagicGrid 方法论^[46]是基于 SysML 建模语言发展而来的, 方法论主要以矩阵的形式进行展示, 该方法论矩阵纵向按照

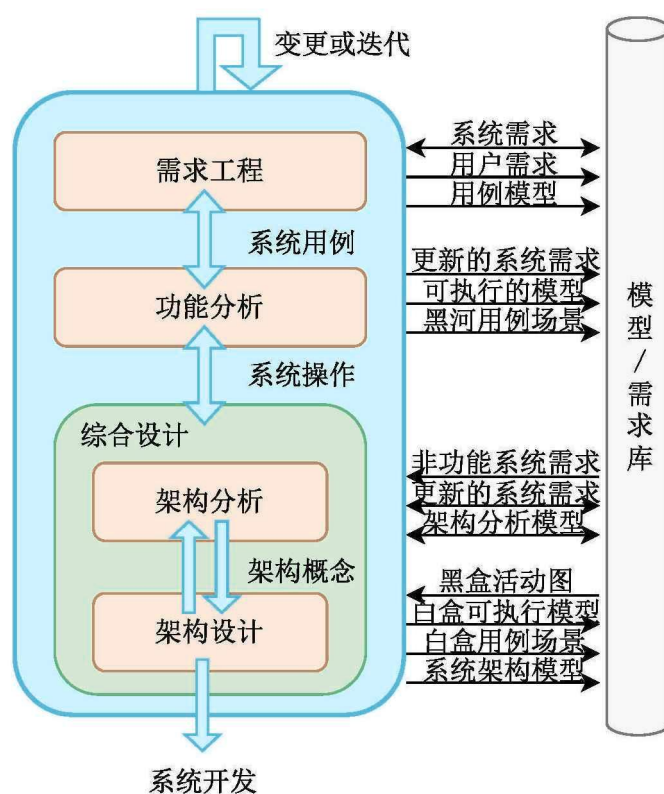


图 2-3 Harmony SE 流程图

SysML 四方面的关键信息分为四列，包括需求、行为、结构和参数。方法论矩阵横向按照问题域和解决域分为两部分，其中问题域包括黑盒问题和白盒问题。具体流程是首先进行需求分析，然后定义系统中的黑盒功能与结构，再进一步定义系统中白盒功能与结构，定义系统的参数约束，最终推导出系统的解决方案。与 IBM Harmony SE 方法论相比较，该方法论更重视参数图的模型构建，其具体框架如图2-4所示。

问题域定义是分析利益相关者需求和使用 SysML 模型，准确地描述系统需要解决问题的过程。分为黑盒和白盒分析两个阶段，关注点和视角各不相同，但都考虑了系统的需求、行为、架构和参数。该过程能确保系统问题得到准确和全面的描述和分析，为后续的设计和开发提供有效基础。

解决方案域模型是一个精准的系统设计逻辑模型，通过提供一个或多个解决问题的方案，并进行权衡分析以选择最佳的实现方案。解决方案模型涵盖了系统需求、行为、架构和参数，并经历了多次迭代，通常与问题域模型一起定义。在大型复杂系统中，问题域和解决方案域可能由不同部门负责，但解决方案域模型的复杂性通常比问题域模型更大。将解决方案模型划分到不同项目模型中，可以更好地管理复杂性，实现更细颗粒度的权限管理、变更控制和重用。

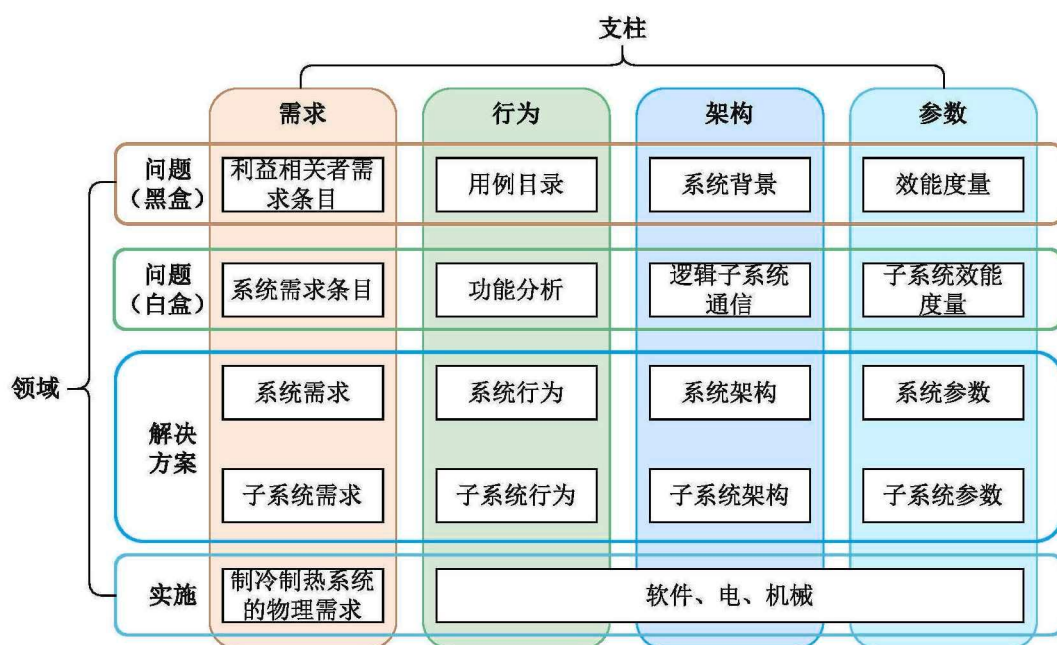


图 2-4 MagicGrid 框架

2.2.3 基于 GOPPRRE 建模方法的建模技术

GOPPRRE 建模方法^[47]基于元对象机制（Meta Object Facility, MOF）的元模型架构，MOF 元模型构建主要由元元模型层、元模型层、模型层、模型实例层四层组成。GOPPRRE 建模方法中的 MOF 架构主要 M3-M0 四层构成，如图2-5所示。

M3 是元元模型层，该层由语义信息描述和元元数据的结构组成，用于对元模型层的描述进行定义。在 GOPPRRE 建模方法中，该层如图2-6所示。

M3 层包含六种元素以及元素与元素之间的约束，六种元素分别是图（Graph）、对象（Object）、属性（Property）、点（Point）、关系（Relationship）及角色（Role）。六种元素之间的关系如上图所示，图元素中规定该类图可以添加的关系及对象，关系两端的连接设置等。对象与对象之间可由关系直接进行连接，也可通过对象上的点进行连接。关系两端有角色，角色控制关系连接终端及始端的方向及箭头样式。图、对象、关系、点、角色五种元素都可包含属性元素。

M2 是元模型层，元模型层是描述模型层的一种抽象语言，元模型层是元元模型实例化的集合。基于不同系统建模规范及系统建模框架，针对 M3 层进行实例化构建对应的元模型库，元模型库包括对象元模型、图元模型、点元模型、属性元模型、角色元模型和关系元模型等。

M1 是模型层，模型层是基于元模型层确定各个元素之间的规则和约束，进行具体模型的构建。

M0 是物理实体，基于模型对系统或体系工程的视角进行描述。基于不同建模

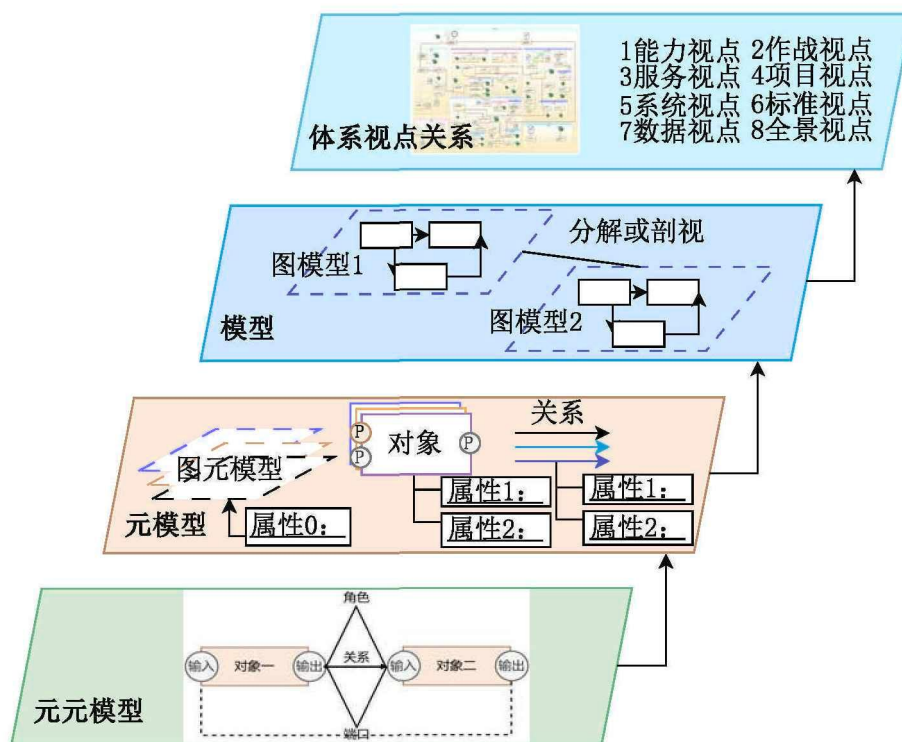


图 2-5 M3-M0 层建模要素

语言或建模规范能构建不同的 M0 层。

GOPPRRE 建模方法是基于 M0-M3 建模架构，围绕 GOPPRR（图、对象、属性、点、关系、角色）六种元素及上述元素之间连接进行构建，下面针对六种元素进行介绍。

图是点、对象、关系、角色及上述元素间关联的集合。M3 层的图元素可实例化为 M2 层的图元模型，M2 层的图元模型可以在 M1 层实例化为模型，并且在 M0 层可将不同的模型进行组合分类。

对象是图中可独立存在的一类元素，包含属性元素，能够承载点元素，对象与对象之间可以通过关系直接连接，也可通过附着于对象上的点元素通过关系进行连接。在模型构建时可将对象元模型实例化为具体状态或操作等。

角色作为关系和对象的连接，支持定义关系连接始端和终端的方向及连接处箭头样式。

关系是图中不可独立存在的一类元素，关系与对象之间通过角色连接，不同图中对关系两端的连接限制不同。关系元素可实例化为关系元模型，并且在 M1 模型层对对象进行连接。

点附着于对象，可作为对象与关系连接的端口。

属性可描述其他五类要素的特征，需附着于其他五类元素，不可单独存在。

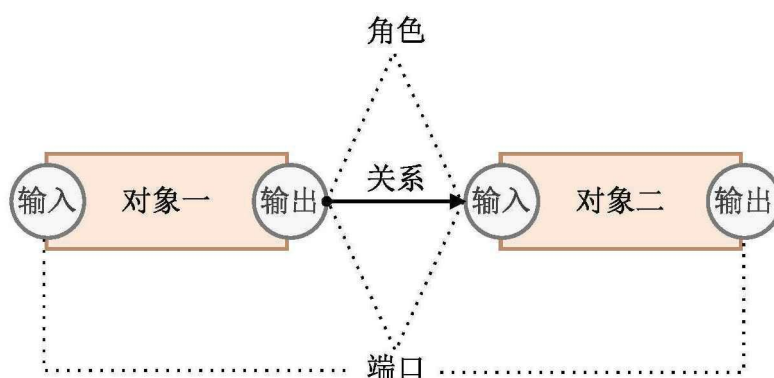


图 2-6 M3 层元元模型

根据上述关系，基于 GOPPRRE 的元建模语言可用表达式进行表示。图元模型与对象元模型、关系元模型、角色元模型、点元模型及属性元模型之间的关系如下所示。

$$\begin{aligned} Gra_{GT} ::= & \\ & \{\sum Obj^{obtp}, \sum Rel^{retp}, \\ & \sum Ro^{rotp}, \sum Po^{potp}, \sum Pro^{prot p}\} \end{aligned} \quad (2-1)$$

图元模型是由点元模型、角色元模型、关系元模型、属性元模型、对象元模型共同定义，公式左侧表示的是本体论概念中的图元模型，右侧分别表示本体论概念中的对象元模型、关系元模型、角色元模型、点元模型、属性元模型。并且右上角为对应元模型在本体论概念中的类型。

针对图元模型定义完毕后，可进一步定义基于图元模型实例化的模型，如下所示。

$$\begin{aligned} Gra_{GT}(graIns) ::= & \\ & \{\sum Obj^{obtp}(objIns), \sum Rel^{retp}(relIns), \\ & \sum Ro^{rotp}(roIns), \sum Po^{potp}(poIns), \sum Pro^{prot p}(proIns)\} \end{aligned} \quad (2-2)$$

等号左侧表示基于图元模型实例化的模型实例，等号右侧表示点元模型、属性元模型、关系元模型、对象元模型、角色元模型及图元模型的实例化元素。

在 GOPPRRE 建模方法中，点元模型、角色元模型、关系元模型及对象元模型之间存在连接关系，连接关系可用一个连接器表示，如下所示。

$$linker(conIns) = \{Rel^{retp}, Ro^{rotp}, Obj^{obtp}, (\vee Po^{potp})\} \quad (2-3)$$

上述式子通过连接器描述了角色元模型、关系元模型、对象元模型中的点元模型或对象元模型之间的约束关系。

通过连接器可以将实例化的对象元模型及实例化的点元模型之间实例化的关系连接定义如下。

$$link^{retp}(reIns) = \{linker(conIns') \Rightarrow linker(conIns')\} \quad (2-4)$$

上述公式支持 GOPPRRE 建模方法的形式化表达，并且支持基于上述规则构建元模型库及模型，实现针对模型的构建及基于规则的推理功能。

KARMA 语言^[48]是工业本体组织（Industrial Ontologies Foundry, IOF）、中国系统工程学会科普工作站、全国信标委员会软件与系统工程分会及特定域建模组织认可并支持的多架构建模语言。KARMA 语言是基于 GOPPRRE 建模方法进行构建，并在较高抽象层次进行模型构建的文本式建模语言，以面向对象方法为核心，支持集成多种基于模型的系统工程建模语言，支持可视化技术及求解技术等，最终实现复杂体系的统一设计。KARMA 语言语法如图2-7所示。

固定语法	描述
Class	
Project XXX annotation (local label = YYY) end XXX	这部分是描述MBSE形式化整体的脚本，其中XXX表示Project的ID，是项目的唯一标识符。“annotation (local label = YYY)”是固定语法，在Project类中，包含子类Language。
Language XXX annotation (local label = YYY) end XXX;	这部分是描述特定域语言及相关的模型的脚本。XXX是Language的ID，该部分包含7个子类，分别描述了6种元模型类 Graph, Object, Property, Point, Role, and Relationship, 和1中模型类Model。
Graph/Object/Point/Relationship /(Export/Import/Indirect) Role XXX end XXX;	这部分是描述在一个Language下的 Graph, Object, Point, Role, and Relationship 5个元模型的脚本，XXX是它们的ID，特别的，在Role前面有关键词“Export/Import/Indirect”，表示角色的方向有输入、输出和无向。
String/Int/Double/Cell/Boolean Property XXX annotation (local label = YYY) ;	在每个Language类下面，包含了超过5种类型的属性元模型，“String”表示字符串，“Int”表示整型，“Double”表示双精度浮点型，“Cell”表示矩阵，“Boolean”表示布尔类型。XXX表示Property的ID，YYY是Property的名称。
Model XXX instance of VVV end XXX;	这部分是描述基于图元模型VVV的模型的脚本，包含了模型的组成和属性，元模型VVV可以被多次继承为模型。

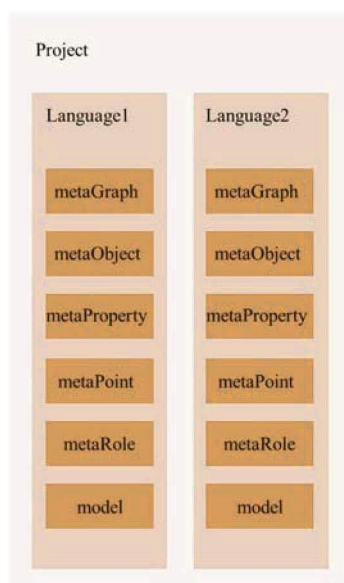


图 2-7 KARMA 建模语言

图2-7左半部分是 KARMA 语言的固定语法，具体内容在 3.2.2 小节中介绍。右侧是基于 KARMA 语言构建项目的架构，一个系统工程项目中可包含多种建模语言，每种建模语言包含不同元模型库及基于元模型库构建的模型。

KARMA 语言支持对系统建模进行元元模型描述、元模型描述及模型描述。针对元模型层按复杂度由低到高进行建模，由于各种元模型之间存在约束或者包含的关系如下，图元模型中包含对象及关系元模型，关系元模型包含角色元模型、属性元模型，对象元模型包含点元模型、属性元模型，点元模型和角色元模型中包含属性元模型。因此按照属性元模型、角色元模型、点元模型、对象元模型、关

系元模型、图元模型顺序进行元模型构建。

2.3 多域建模技术

2.3.1 多域建模技术协同方案分析

多域建模技术主要包括协同建模及用户管理。其中多域建模的难点在于协同建模，协同建模必然会遇到多用户同时编辑产生冲突的问题。目前常见的冲突处理方案有编辑锁、diff-patch、OT 算法和 CRDT 算法。下面分别进行介绍。

首先是编辑锁，编辑锁的实现方式为当有用户在进行编辑时，当前用户编辑的文档在用户编辑过程中会被系统锁定，只允许当前用户进行编辑，其余用户无法编辑此模型或文档。实现方法简单且不易丢失数据，但用户体验度低，无法真正实现实时在线协同。

diff-patch 算法实现主要经过以下三个步骤，首先是生成新的虚拟节点，其次是将新的虚拟节点与旧的虚拟节点进行对比，最后用新的节点替换旧的节点。在第二步中有 diff-patch 的核心算法 diff 算法，diff 算法用于支持两棵树节点的差异对比，diff 算法的主要内容如图2-8所示。

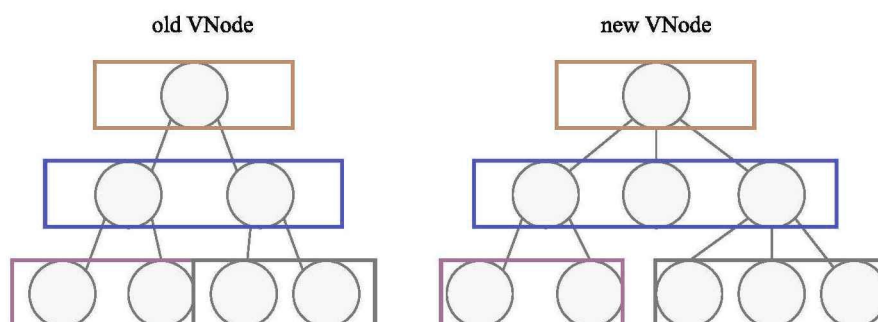


图 2-8 diff 算法

图2-8中两棵树分别是新老节点，使用 diff 算法针对新老节点进行差异对比时，并非通过逐层搜索遍历的方式进行对比，而是通过针对相同层的树节点进行对比，即将上述新旧节点中相同颜色框中的节点进行对比，通过对比同层中的树节点获取差异，基于此种对比方法的时间复杂度为 $O(n)$ ，提高了算法的效率。通过对比得到变更的节点，将变更状态更新至视图，从而完成协同的一致性处理。

OT 算法可以解决目前协同操作的冲突问题，由于协同编辑中涉及多个用户同时操作，为避免冲突，不采取直接应用每个操作的方法，基于最终操作一致性原则将用户操作通过协同算法进行操作转换后进行应用。

CRDT 算法^[49]支持被基于网络的多台计算机进行复制，并且复制文件副本可自主更新，独立且并行运行。CRDT 算法包括基于操作的 CRDT 和基于状态的

CRDT 两种类型。CRDT 算法和 OT 算法都可用于协同操作中，两者算法主要对比如表2-1所示。

表 2-1 OT 算法与 CRDT 算法对比

OT	CRDT
最早论文于 1989 年提出	最早论文于 2006 年提出
算法设计复杂度更高	算法设计更简单
依赖中心化服务器，难以应用于分布式环境	通过 P2P 方式完成数据同步
更容易保留用户意图	更不易保留用户意图
不影响文档体积	协同文档体积更大

2.3.2 多域建模技术协同引擎

Convergence 协同引擎是由 Convergence Labs 开发，Convergence 引擎用于构建实时协同功能。提供各类易于使用的 API 支持协同应用程序的扩展开发。该程序的核心是协同分布式数据模型，协同中心支持存储该数据模型，并且支持用户修改及实时查看彼此的修改。Convergence 提供有关实时分布式数据、用户和身份、在线用户、聊天和信息等 API 支持协同技术的实现。

下面针对协同平台的系统架构进行介绍，Convergence 协同引擎是一个具有基于 socket 端口和方便的 JavaScript API 的服务器。JavaScript API 主要利用 Convergence 平台的实时功能，包括改变数据、监听数据变化、广播用户界面提示及监听参与者是否在线等。通过将客户端代码和服务器代码紧密耦合，为程序应用过程中的冲突提供解决方案。如图2-9所示。

将数据库与服务器捆绑，便于自动、持久、及时进行数据模型的更改，由此支持一些强大的功能，如数据模型历史记录、重做及撤销等。

每个 Convergence 部署都捆绑一个基于 web 工具的 Convergence 命令控制台，可支持创建和管理域、创建和管理集合、创建和管理模型、创建和管理域用户、为每个域配置身份验证等信息、在模型、集合或域级别管理权限。

通过数据库的共享数据便于各个用户更好协同，并且将数据保存在自己的应用程序中。其主要工作流程如图2-9所示，首先用户进入房间进行协同合作，将数据转换为 JSON 格式的实时模型传输到 Convergence 域中。用户可通过应用程序的交互界面对实时模型进行编辑。同时，另一个用户进入协作室，该用户从 Convergence 域中请求数据。协作室中两个用户都针对更改事件进行监听，当 Convergence 域中发出远程事件更改时，应用程序能够基于 Convergence 域中的共享数据进行 UI 交互界面的更改及视觉渲染。协作室中所有用户都以这种方式处理统一数据模型。

实时模型是在协同工作过程中所采用的分布式结构，Convergence 支持任何可

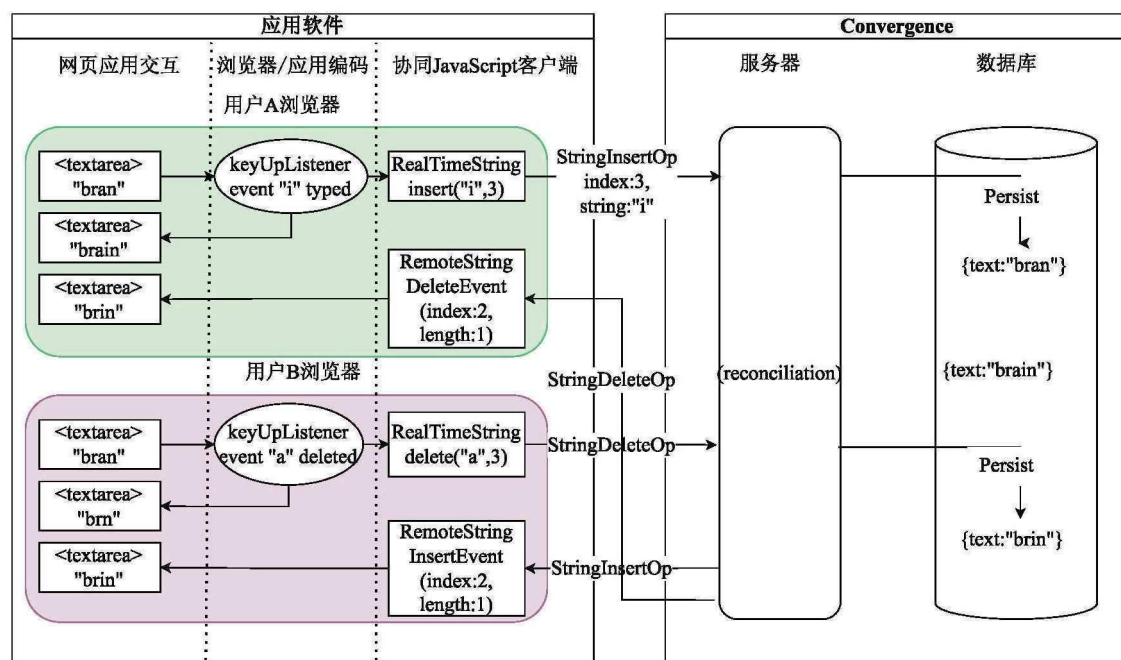


图 2-9 协同平台架构图

以存储在 JSON 格式中的数据的共享，并且能够解决各种可能组合，包括嵌套对象、数组等。

2.3.3 多域建模技术渲染工具

mxGraph 是一个支持 Java、JavaScript、Php、.Net 的画图框架，可用于图形可视化、图形交互、图形布局和图形分析。其中，图形可视化中 mxGraph 绘图主要由点和边组成，支持常见的实体关系图、思维导图及流程图等。在图形交互方面，mxGraph 支持拖拽、选择、复制及调整大小等操作，并且通过一个 API 类可支持在线编辑。在图形布局方面，mxGraph 提供栈式布局、树形布局、圆形布局等多种布局方式。在图形分析方面，mxGraph 具备图相关的算法分析功能。

在 mxGraph 中，为了使前端界面能够展示图形，首先需要创建一个用于显示图形的 DOM 节点，将此 DOM 节点作为创建 mxGraph 的参数可创造出用于编辑的 mxGraph，将 mxGraph 进行实例化可创造出图形。mxGraph 是中心类，可以将 mxGraph 类作为入口，对其他类进行编辑。主要涉及的类如图2-11所示。

如图2-11所示，与 mxGraph 相关的类主要有 mxGraphView、mxGraphSelection、mxGraphModel、mxCellRenderer、mxStylesheet 几个类，其中 mxGraphView 主要负责图形的视图，该类主要负责计算几何图形的绝对坐标、周长等，并且将相关信息缓存在 mxCellStates 中便于检索，每当模型元素进行平移、缩放发生状态改变时，都会更新 mxCellStates 中的状态。

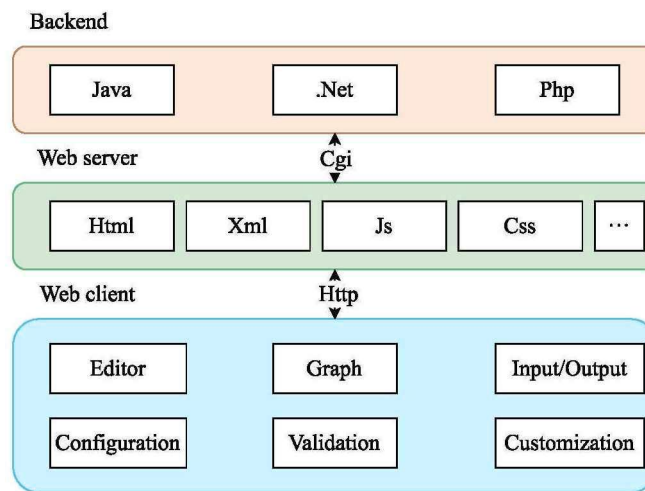


图 2-10 mxGraph 架构

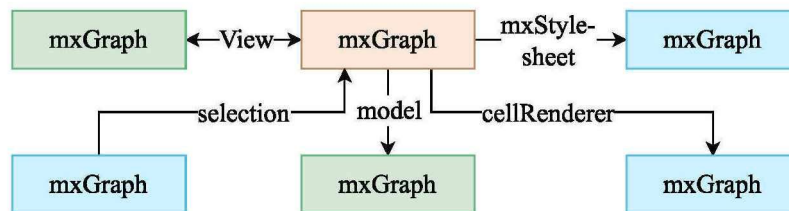


图 2-11 图形可视化核心类

`mxGraphModel` 是通过扩展 `mxEventSource` 实现的。基于 `mxGraphModel` 的事件更改，能够为模型中所有更改提供事件通知，包括监听 `mxRoot`、`mxChild`、`mxTerminal`、`mxValue`、`mxStyle`、`mxGeometry`、`mxCollapse`、`mxVisible`、`mxCellAttribute` 等更改，该功能为之后在线实时协同建模奠定基础。`mxGraph` 提供应用程序接口支持针对模型的 `mxGraphModel` 进行编码及解码。编码是通过实例化一个 `mxCodec` 类，通过类的 `encode()` 函数针对实例化的 `mxGraphModel` 进行编码。`mxGraphSelection` 类支持对选中的模型添加监听函数，例如删除元素、添加元素、改变元素等监听函数，支持针对选中元素进行一系列操作。`mxStylesheet` 支持定义图形中元素的样式，默认样式表中包含对象默认样式、关系默认样式两种内置样式。支持通过获取默认样式、设置默认样式的方法对默认样式进行修改。支持通过以 `mxStylesheet` 的实例化类为入口，通过 `putCellStyle()` 函数将创建出来的格式放入格式表中。`mxCellRenderer` 支持创建、重绘、销毁与单元格状态相关的形状和标签。

图中的模型元素由 `mxCell` 进行实例化而来，并且 `mxGraphModel` 提供针对 `mxCell` 的各项操作。如图2-12所示。

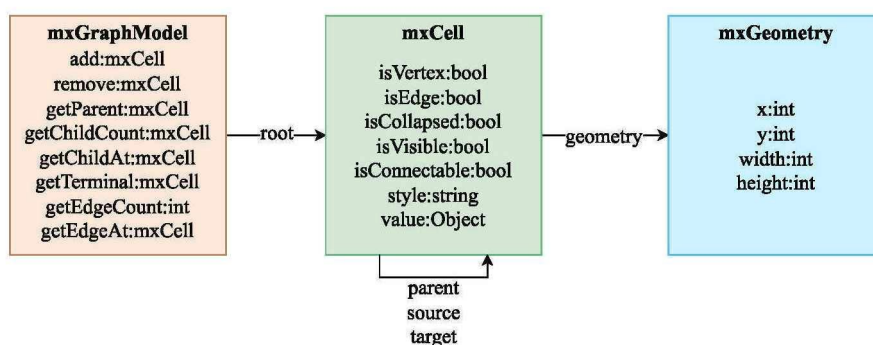


图 2-12 mxGraphModel、mxCell 及 mxGeometry 关联

通过实例化 `mxGraphModel` 类可以支持添加 `mxCell`、删除 `mxCell`、获取 `mxCell` 的父节点、获取 `mxCell` 的子节点数、获取 `mxCell` 某一个子节点、获取 `mxCell` 的端对象、获取连接该 `mxCell` 的边个数等。可对每一个实例化的 `mxCell` 类型进行判断、设置并判断是否可见、是否支持扩展、能否进行连接，并且提供 `style` 及 `value` 设置其外观和值。通过 `mxCell` 类为入口可获取 `mxGeometry` 属性，`mxGeometry` 主要包含 `x`、`y`、`width` 和 `height` 四个变量。从图2-12可知，`mxGraphModel` 类具有以下属性：首先是 `mxGraph` 的根元素包含图层，并且每个层的父节点是根元素。其次，一个图层可以包含 `mxGraph` 中的顶点、边和组等元素，并且组可以包含图模型的元素，进行递归。

`mxGraph` 除了支持图形可视化，同时支持图形交互，图形交互主要基于 `mxEditor` 类进行操作，并且由 `mxDefaultPopupMenu`、`mxDefaultToolbar`、`mxGraph`、`mxWindow` 及 `mxUndoManager` 等类辅助图形交互。如图2-13所示。

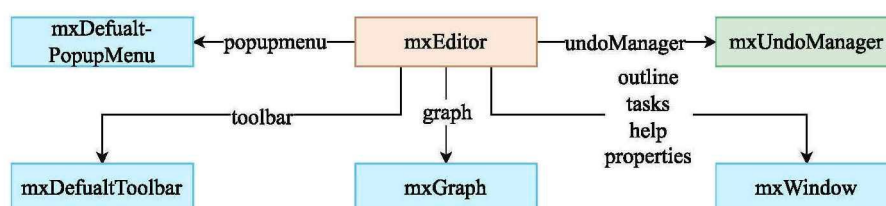


图 2-13 图形编辑核心类

在图2-13中，`mxDefaultPopupMenu` 类支持在 `mxGraph` 图编辑页面为鼠标事件创建右键菜单，以 `mxDefaultPopupMenu` 实例化为基础，通过向 `createMenu()` 函数中传入 `mxEditor` 实例化对象等参数创建右键菜单，并且通过 `addItem()` 函数给右键菜单添加选择项。`mxDefaultToolbar` 类可用于构建基于 `mxEditor` 类的工具栏，支持以拖拽方式添加对象。`mxUndoManager` 类支持将模型中的原子更改存储至 `mxUndoManager` 的历史记录中，模型中的原子更改可用 `mxRootChange`、`mxChildChange`、`mxTerminalChange` 等类的实例化结果表示。将基于 `mxUndoManager`

类实例化的对象为入口，通过 *undo()* 和 *redo()* 函数进行撤销和重做的操作。通过 *clear()* 操作将储存在 *mxUndoManager* 中的操作命令历史清空。*mxWindow* 类支持在图形界面创建一个简单窗口，支持显示模型编码、控制台信息等内容。

针对图形布局，*mxGraph* 支持基本的布局算法，主要通过基于 *mxGraphLayout* 的函数 *moveCell()* 进行布局操作。*mxGraphLayout* 类包括 *mxCircleLayout*、*mxCompactTreeLayout*、*mxCompositeLayout*、*mxFastOrganicLayout*、*mxParallelEdgeLayout*、*mxPartitionLayout*、*mxStackLayout* 等子类。

2.4 本章小结

本章介绍了云架构及云部署方式，选择基于 *Docker* 镜像进行云架构部署。

将主流柔性建模技术进行对比，选择基于 *GOPPRRE* 建模方法的柔性建模技术作为多域柔性建模技术的基础。

同时对多域建模技术的协同方案进行分析，选择基于 *OT* 的冲突解决算法作为多域建模技术的冲突解决方案，后续章节将基于此算法实现多域建模技术并解决协同操作中的冲突问题。

选择基于 *Convergence* 协同引擎进行多域建模，后续章节将基于该引擎实现多人在线实时协同建模及协同中心项目用户管理。

选择 *mxGraph* 作为建模的渲染工具，后续章节将基于 *mxGraph* 进行柔性建模并实现建模界面的敏捷交互。

第三章 基于 GOPPRRE 建模方法的柔性建模技术研究

上一章介绍基于云架构协同的多域柔性建模技术的相关理论与研究，本章将以上述理论及研究为基础，基于 GOPPRRE 建模方法及 mxGraph 进行柔性建模技术的研究，采用 React 框架构建柔性建模平台。基于柔性建模技术进行元模型库及模型构建，采用 OntoQA 方法针对柔性建模项目导出本体进行定量分析。为下一章基于云架构协同的多域柔性建模技术提供可操作的柔性建模平台。

3.1 现有建模技术问题分析

柔性建模技术需要具备针对变化的系统工程的适应能力，针对复杂系统工程的建模语言及建模框架的覆盖能力，针对长周期系统工程建模的迭代能力，支持基于建模为仿真提供统一真相来源的能力。

现有建模技术无法针对目前常用的建模语言及建模框架进行全覆盖，如表3-1所示。

表 3-1 主流公司建模技术可支持建模语言

	MagicDraw	Rhapsody	Capella	M-Design	Modelook
建模语言					
UML	√	√	√	√	√
SYSML	√	√	√	√	×
UPDM	√	√	×	×	×
UAF	√	√	×	×	×
BPMN	√	√	×	×	×
OPM	×	×	×	×	×
Capella 建模语言	×	×	√	×	×
Archimate	√	√	×	×	×
扩展	√	×	×	×	×

通过表3-1统计可得，目前主流建模工具采用的建模技术无法覆盖目前常用的建模语言及建模规范，针对复杂的基于模型的系统工程进行建模时，会涉及不同领域不同建模语言及框架，因此需要一种能够覆盖目前主流建模语言的建模技术，支持跨领域的基于模型的系统工程建模。

基于柔性建模技术构建出的模型具备包容开放性、灵活适应性、敏捷建模及系统整体性。包容开放性支持各类建模语言及建模框架。灵活适应性支持针对各种建模语言构建自定义元模型库，可根据系统复杂程度调整元模型库。敏捷建模支持建模约束、自动布局、规则验证、属性展示及修改等功能。系统整体性支持基于不同视角进行模型构建，支持分解剖视图构建厘清系统层级关系，支持时序图构建说明系统时序关系。

除上述特性之外，基于柔性建模技术构建出的模型需要具备与常见模型分析工具或仿真工具进行兼容的能力，便于基于模型的系统工程在模型构建后的测试、仿真、验证及迭代等。目前主流建模公司开发的建模工具兼容性如下表所示。因此需要一种柔性建模技术支持将模型信息导出为基于模型的系统工程中的常见格式，用于进一步仿真验证。

表 3-2 主流公司建模技术可支持文件格式

	MagicDraw	Rhapsody	Capella	M-Design	Modelook
本体	√	×	×	×	×
Modelica	√	×	×	√	√
传输格式 Simulink	√	√	×	√	×
报告	√	√	√	√	√
自定义	×	×	×	×	×

表3-2统计数据表明，目前主流建模公司开发的建模工具无法将模型转换为基于模型的系统工程中常见存储格式，用于支持主流模型分析工具的进一步分析和验证。

3.2 柔性建模渲染方法

3.2.1 支持柔性建模的 mxGraph 核心类说明

在本文柔性建模技术中，mxGraph 类、mxCell 类及 mxGeometry 类支持对象和关系的插入及展示，mxLayoutManager 类、mxGraphLayout 类及 mxStackLayout 类支持模型布局，mxMultiplicity 类及 mxConnectionHandler 类支持柔性建模中的关系连接验证，本小节分别针对上述类进行介绍。

mxGraph 中的 mxCell 类支持对模型中的对象关系进行构建展示，支持在画布中插入对象和关系的主要函数和参数如表3-3所示。

表 3-3 mxCell 类函数及参数说明

类	mxCell					
函数	insertVertex			insertEdge		
	parent	id	y	parent	id	relative
参数	value	style	height	value	style	source
	x	width	relative	target		

通过 *insertVertex()* 函数可以插入对象，需要提供插入的父节点、对象唯一的标识、值、横坐标、纵坐标、宽度、高度、外观、表明是否采用相对坐标的变量。*relative* 这一参数决定是否采用相对位置坐标，该参数在基于 GOPPRRE 建模方法进行柔性建模中十分重要，因为本文研究的柔性建模技术中存在对象元模型

和点元模型，在模型构建中，实例化的点元模型需要附着于实例化的对象元模型上，并且附着位处于对象边缘。

首先介绍一下 *relative* 的使用方法，如图3-1所示。

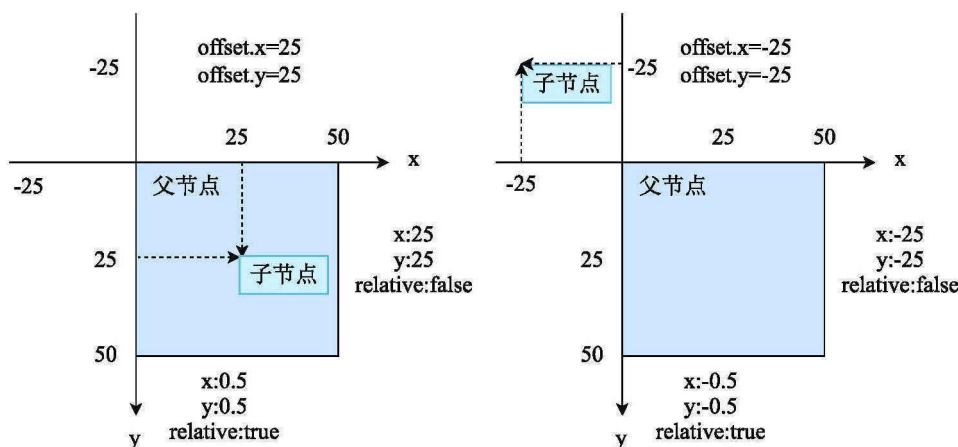


图 3-1 *relative* 说明

柔性建模技术中对象或画布的位置均以左上角点的位置为准，如果 *relative* 为 *false*，则横向偏移 *x* 和纵向偏移 *y* 都是正常坐标，如图3-1所示，子节点在插入时，*parent* 是父节点，*x* 和 *y* 都是 25。如果 *relative* 为 *true*，则横向偏移为相对父节点的起点横向偏移量除以父节点的宽度，纵向偏移量为相对父节点的起点的纵向偏移量除以父节点的高度，如上图所示，图3-1中通过 *insertVertex()* 插入时 *relative* 为 *false*，此处的 *x* 和 *y* 应该是 0.5。

除 *relative*、*x*、*y* 以外，通过 *mxCell* 可以设置 *mxGeometry* 类的实例化对象，*mxGeometry* 类中有 *offset* 变量，此变量可用于设置在通过 *relative*、*x*、*y* 确定位置后的偏移量，如图3-1所示。

通过上述类可实现柔性建模模型中的可附着点的对象。此外，柔性建模中的分解剖视图需要通过在父节点中插入子节点的方式进行构建。基于 *mxGraph* 构造的分解剖视图结构如图3-2所示。

基于 GOPPRE 建模方法的柔性建模技术中存在分解剖视图，分解剖视图可基于对象进行构建，但 *mxGraph* 中 *mxGraph* 类实例化后无法加入对象中，因此在 *mxGraph* 中通过增加对象层级来支持分解剖视图。如图3-2中的分解图一、分解图二、剖视图均通过对象来渲染表达。图3-2中的对象三、对象四、对象五均为外形为泳道的容器模型。渲染时涉及对象中分解剖视图的位置分布，需要使用 *mxGraph* 类、*mxLayoutManager* 类、*mxStackLayout* 类。

首先是 *mxLayoutManager* 类，该类支持构建布局处理器，支持画布在发生改变后按照布局处理器设置的布局样式进行处理。如表3-4所示。

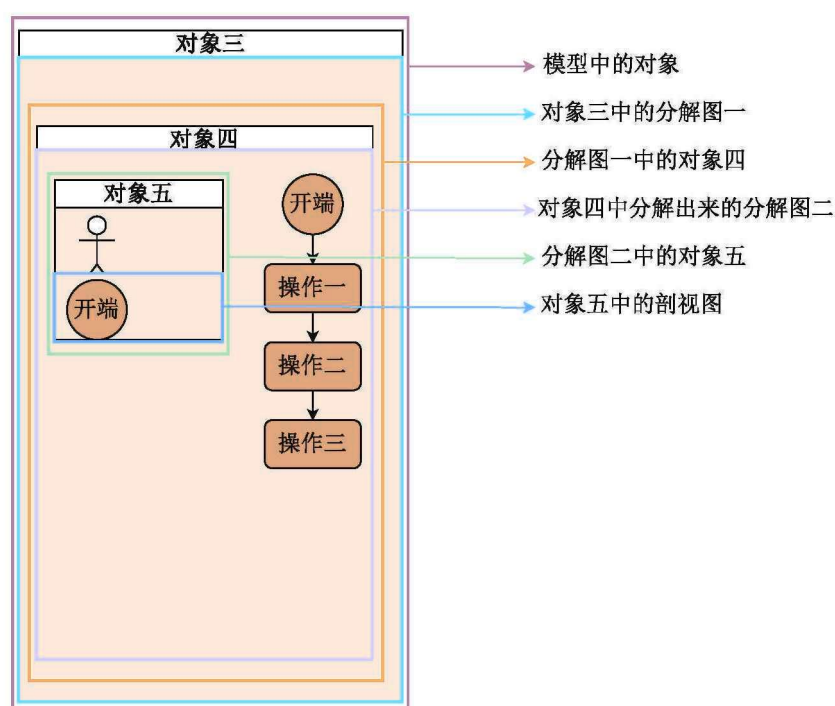


图 3-2 基于 mxGraph 渲染分解剖视图层级

表 3-4 mxLayoutManager 类变量及函数

类	mxLayoutManager				
函数	getGraph	setBubbling	isEnabled	getLayout	executeLayoutForCells
参数	graph	enabled	bubbling	resizeHandler	moveHandler

该类的变量主要包括支持该布局处理器处理的图，*bubbling* 用于设置该布局处理器布局时是否遵循冒泡原则。*Enabled* 用于控制该布局处理器是否启动。*moveHandler* 用于决定该触发器是否可以基于移动事件进行触发，*resizeHandler* 用于决定该触发器是否可以基于尺寸改变事件进行触发。函数中重要的是 *setBubbling()* 和 *getLayout()* 两个函数，*setBubbling()* 函数用于设置该触发器是否遵循冒泡原则，*getLayout()* 用于返回一个该触发器支持的布局样式。目前常见的布局样式有 *mxCircleLayout*、*mxCompactTreeLayout*、*mxCompositeLayout*、*mxEdgeLabelLayout*、*mxFastOrganicLayout*、*mxGraphLayout*、*mxParallelEdgeLayout*、*mxPartitionLayout*、*mxRadialTreeLayout* 及 *mxStackLayout* 等。*mxGraphLayout* 是所有布局算法类的基础类，*mxCircleLayout*、*mxCompactTreeLayout*、*mxCompositeLayout*、*mxEdgeLabelLayout*、*mxFastOrganicLayout*、*mxParallelEdgeLayout*、*mxPartitionLayout*、*mxRadialTreeLayout* 都是 *mxGraphLayout* 的子类，*mxGraphLayout* 主要支持父节点的布局并且支持在父节点下基于布局算法移动其子节点。*mxCir-*

cleLayout 布局支持基于给定的半径进行圆形布局。mxCompactTreeLayout 布局支持实现一个紧凑布局, 该布局算法适合没有环的结构, 并且会忽略没有关系连接的对象。mxCompositeLayout 布局是复合布局, 该布局允许将多种布局组合到一个布局中。mxEdgeLabelLayout 实现关系的标签布局。mxFastOrganicLayout 实现快速的有机布局, 该算法针对有关系连接的对象, 并且忽略没有关系的对象。mxParallelEdgeLayout 算法用于连接两条及两条边以上的关系进行平行边布局排列。mxPartitionLayout 布局采用分区布局算法, 支持选择布局方向、设置布局间隔、设置布局边界等。mxRadialTreeLayout 布局可以实现径向树算法, 该布局算法针对没有环的图形, 通过设置根节点位置、节点距离、半径等实现根节点位于中心, 其他节点沿着半径方向排列的布局。mxStackLayout 可实现父节点中的子节点水平堆栈或垂直堆栈布局。该布局具体如表3-5所示。

表 3-5 mxStackLayout 类变量及函数

类	mxStackLayout				
函数	isHorizontal	getParentSize	getLayoutCells	snap	execute
参数	horizontal	spacing	fill	allowGaps	border
	resizeParent	resizeParentMax	resizeLast	borderCollapse	wrap

基于 GOPPRRE 建模方法的柔性建模技术中分解剖视图在对象中的布局由 mxStackLayout 类实例化实现。在基于 mxGraph 分解剖视图的渲染中, 通过在容器模式对象中插入 mxCell 实例化的 Vertex 节点实现分解剖视图, 因此需要保证以下几个要点: 首先每个对象需要紧密排布在容器模式的对象中, 其次每个分解剖视图由于内部模型复杂度不同, 因此分解剖视图的大小也不同, 需要通过算法计算出同一容器模式对象中各分解剖视图的最大宽或高, 以此确定父容器节点的大小。由于分解剖视图在同一容器模式的对象中没有绝对顺序, 前端展示界面应该支持分解剖视图在同一容器模式对象中的位置移动。基于以上分析, mxStackLayout 类实例化时, 各变量的设置如下: *resizeParent* 为 true、*resizeParentMax* 为 true、*fill* 为 true、*allowGaps* 为 true、*horizontal* 为 true、*spacing* 为 0、*resizeLast* 为 true。并且通过 *execute()* 函数对每一个容器模式对象进行布局处理。

mxGraph 中的 mxMultiplicity 类支持针对关系的约束, 该类的主要变量和函数如表3-6所示。

mxMultiplicity 类支持定义无效连接及无效连接时弹出错误消息, 并且通过将实例化的 mxMultiplicity 类添加进入 mxMultiplicity 数组中定义图中关系连接规则, 通过删除 mxMultiplicity 数组中的 mxMultiplicity 类实例化对象删除图中关系连接规则。mxMultiplicity 类中存在 *type*、*attr*、*value*、*source*、*min*、*max*、

表 3-6 mxMultiplicity 类函数及参数

类	mxMultiplicity				
函数	check	checkNeighbors	checkTerminal	checkType	
参数	type	attr	value	source	min
	typeError	countError	validNeighborsAllowed	max	validNeighbors

validNeighbors、*validNeighborsAllowed*、*countError* 及 *typeError* 十个变量。*source* 定义该规则约束是针对 *source* 或者 *target*, *type* 表明该约束规则针对的对象类型, *attr* 是通过类型中的属性名称添加约束, *value* 是通过类型中的属性值添加约束, *min* 是定义该对象允许输入关系的最小数量, *max* 是定义该对象允许输入关系的最大数量, *validNeighbors* 是定义需要约束的邻居节点, *validNeighborsAllowed* 定义上文中的邻居节点是否符合条件, *countError* 定义该类型对象发生连接数量错误时的报错信息, *typeError* 定义该类型对象发生连接类型错误时的报错信息。

柔性建模技术中进行关系的添加需要使用 *mxConnectionHandler* 类, 该类支持创建连接图形的事件处理程序, 在进行关系创建时支持覆盖该类的 *mxConnectionHandler.prototype.insertEdge()* 函数进行关系插入时的规则制定。

柔性建模技术基于 GOPPRRE 建模方法进行构建, 在 GOPPRRE 建模方法中点元模型实例化后需要附着在对象元模型实例化的对象模型上, 并且需要保证点元模型只能在对象元模型边缘进行移动, 实现点模型路径约束需要使用到 *mxGraphHandler* 类。*mxGraphHandler* 类函数和参数具体如表3-7所示。

表 3-7 mxGraphHandler 类函数及参数

类	mxGraphHandler				
函数	getInitialCellForEvent	start	snap	getDelta	reset
参数	graph	enabled	updateCursor	guidesEnabled	shape

3.2.2 KARMA 模型基础规则及关键信息处理

KARMA 语言是可实现多种架构表达及描述的面向对象的可读式文本语言, 能够实现各种建模语言和建模框架的元模型库开发及建模, 例如 UML、SysML、业务流程建模标注 (Business Process Modeling Notation, BPMN)、Capella、针对 DoDAF 和 MoDAF 的统一规范 (Unified Profile for DoDAF/MoDAF, UPDM)、统一架构框架 (Unified Architecture Framework, UAF) 等, KARMA 语言可对基于模型的系统工程模型进行可读文本的形式化表达, 增强语义描述的准确性。

KARMA 语言的语法结构主要包含三个部分：基础数据模块、类和方法模块及表达式模块。基础数据模块定义了语言中使用的数据类型，类和方法模块提供了模型相关操作的封装，而表达式用于描述数据类型操作的方式。下面分别进行介绍。

KARMA 语言中的数据类型如图3-3所示：



图 3-3 KARMA 语言基础数据类型

KARMA 语言中的基础数据类型可以用于 KARMA 语言下包含的各个模块，保证 KARMA 语言下各个模块的数据类型的一致性。基础数据类型主要包括七种类型，分别是数字数据、字符串数据、布尔型数据、数组数据、矩阵数据、元胞数组数据和集合数据。首先数字数据包括整型和实数型，整型用于表达整数概念，实数型用于表达实数概念。字符串数据中可以放入除双引号和反斜杠的任意字符，如果想放入双引号或者反斜杠字符，需要在双引号或者反斜杠字符前面加上一个反斜杠转义符进行转义。布尔型数据包含 true 和 false 两个值。一维数组结构包括索引和值，索引的类型默认为整型，值的类型包括算术型、字符串型、布尔型。二维数组矩阵中只能存放算术类型的数据，并且在矩阵类型声明时需要声明存储的是整数类型还是实数类型。元胞数组是基于元胞的索引数据容器的数据类型。元胞可以包含布尔值、数值和字符串等类型的数据。集合类型的特点是在初始化后集合类型的大小可以进行改变。集合类型包括 Set、OrderSet、List、Map 四种。上述四种集合类型支持整型数据、实数型数据、字符串型数据及布尔型数据等基础数据类型。

KARMA 语言中的将数据和操作这些数据的方法封装在一起组成类，问题域的抽象概念可以通过类表示。

KARMA 的元素类如图3-4所示:

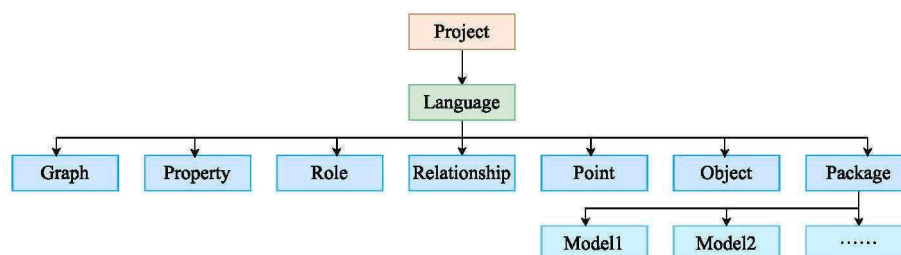


图 3-4 KARMA 类架构

首先是 Project 类，在 KARMA 语言中，Project 类是作为整个项目的根节点，标志着一个 KARMA 项目的开始。通过定位 Project 类，可以确定一个 KARMA 项目的起始点。Project 类通过唯一标识符进行实例化，没有对自身内容的描述。Project 类中只包含了 Language 类，用于描述项目所使用的建模语言。其次是 Language 类，针对某种建模语言的元模型库及基于元模型库构建模型的集合可以用 Language 类进行定义。一个 Project 类可以包含多个 Language 类，不同的 Language 类包含基于不同建模语言的元模型库及基于元模型库构建的模型。每一个 Language 类需要通过一个唯一的 Language 类标识进行实例化。

Package 类是由相关性较大的模型或由同一元模型实例化的模型的集合。模型的导航或归类可以通过 Package 类的添加来实现。该类通过唯一的 Package 类 id 进行实例化。在 Language 类中可以定义 Package 类，但是 Package 类中无法定义 Package 类。

项目	package 项目id annotation (localLabel = "项目标签") end projectSystemName
语言	Language 建模语言id annotation (localLabel = "建模语言标签") end languageSystemName
包	package 包id annotation (localLabel = "包标签") end bao1

图 3-5 项目/语言/包 KARMA 建模语言

在元模型层面中，KARMA 语言语法结构如下图所示。

首先是 Object/Point/Property/Relationship 关键字表明该元模型种类，关键词后面跟上元模型 id，在对象元模型中，通过 *initial Property* 关键字说明该对象元模型中包含的属性元模型，通过 *Undirected Point* 关键字说明该对象中包含的点元模型。在关系元模型中，除了属性元模型关键字以外，通过 *Role* 关键字表示角色元模型信息。

在角色元模型中,通过 *Import/Export Role* 关键词对角色元模型进行定义,通过 *initial Property* 关键字说明该角色元模型中包含的属性元模型。

在图元模型中,通过 *partial Graph* 关键字表明该元模型是图元模型,通过 *initial Property* 关键字说明该图元模型中包含的属性元模型。通过 *Object* 加上对象元模型 *id* 表明该图元模型中包含的对象元模型,通过 *Relationship* 关键字加关系元模型 *id* 表明该图元模型包含的关系元模型。如图3-6所示。

属性	String Property 属性元模型id annotation (localLabel = "名称", unit = "null", description = "");
对象	Object 对象元模型id initial Property 属性元模型引用id="null" extend 属性元模型id annotation (localLabel = "属性元模型标签") annotation (localLabel = "对象元模型标签", icon = "对象元模型图标", description = "" shape = "对象元模型形状" visualization = "对象元模型展示形式" imageAddress = ""); end 对象元模型id;
关系	Relationship 关系元模型id initial Property 属性元模型引用id="null" extend 属性元模型id annotation (localLabel = "属性元模型标签") Role 角色id annotation (localLabel = "关系元模型标签", icon = "关系元模型图标", description = "" shape = "关系元模型形状"); end 关系元模型id;
角色	Import/Export Role 角色元模型id initial Property 属性元模型引用id="null" extend 属性元模型id annotation (localLabel = "属性元模型标签") annotation (localLabel = "角色元模型标签", description = "" shape = "角色元模型形状"); end 角色元模型id;
点	Point 点元模型id initial Property 属性元模型引用id="null" extend 属性元模型id annotation (localLabel = "属性元模型标签") annotation (localLabel = "点元模型标签", description = "" shape = "对象元模型形状" imageAddress = ""); end 点元模型id;
图	partial Graph 图元模型id initial Property 属性元模型引用id="null" extend 属性元模型id annotation (localLabel = "属性元模型标签") Object 对象元模型id Relationship 关系元模型id Constraint (bind connector (关系连接对象/点id, 关系id, 连接角色id),) annotation (localLabel = "图元模型标签", type = "图元模型图标", description = ""); end 图元模型id;

图 3-6 属性/对象/关系/角色/点/图元模型 KARMA 建模语言

3.2.3 基于关键信息柔性建模渲染

柔性建模中的前端模型渲染主要包括普通图、分解剖视图及时序图的渲染, 主要包括后端数据处理模块和前端渲染模块两部分。本小节首先针对后端数据处理进行介绍, 其次基于处理后的数据针对前端渲染流程进行介绍。

一个支持柔性建模的 KARMA 模型中包含的信息有模型信息和元模型信息, 元模型主要包括对象元模型、关系元模型、图元模型、点元模型、角色元模型和属性元模型, 关键信息渲染部分只使用模型信息。模型的类别包括普通图模型、分解剖视图模型和时序图模型。

首先介绍普通图的柔性建模渲染, 图3-7是普通图进行柔性渲染需要用到的 KARMA 模型关键信息。KARMA 模型支持柔性建模的关键信息主要包括对象和关系。其中对象的用于渲染的关键信息包括唯一标识符、边框样式、边框粗细、横坐标、纵坐标、宽度、高度、对象背景颜色、对象形状、对象模式、字体大小/粗细/是否斜体/标签。以上是对象的一些基本属性, 除以上基本属性外, 在 KARMA 模型的基础规则中介绍过, 点元模型实例化是依附于对象元模型的实例化存在的, 因此对象中包含点的关键信息, 包括点的标识、点形状、点大小、点元模型标签、点类型、点位置、点元模型标识、点元模型形状等。针对于 KARMA 模型中的关系的关键信息提取, 包括关系的标识、标签、布线信息、关系形状、源对象标识、目标对象标识、关系颜色、关系起点样式、关系终点样式、关系文字大小/边缘颜色/粗细/是否斜体、关系光滑度、关系元模型标识/模式/形状/标签、关系跳转样式、关系跳转类型等。

关键信息提取后, 转换为 JSON 格式数据, 从后端传送至前端, 采用 `mxGraph` 进行渲染。JSON 格式的数据主要如图3-8所示:

图3-8是针对一个模型的 KARMA 关键信息提取, 每个普通图中包含一个模型, 一个模型关键信息分为三部分, 第一部分为模型总体信息, 包括模型的图元模型标识、模型名称、模型类型。图元模型标识用于之后获取对应的图元模型中配置的对象元模型和关系元模型, 及其对应的对象添加约束和关系连接规则。模型类型用于前端判断该模型的种类, 通过模型类型在前端渲染流程中进入对应图类型的渲染流程。第二部分是对象关键信息, 包含对象渲染所需关键信息。第三部分是关系关键信息, 包含关系渲染所需的关键信息。

获取到关键信息后, 采用 `mxGraph` 进行渲染, 普通图的渲染流程如下。首先获取 KARMA 模型关键信息, 由于 KARMA 模型中关系不能单独存在并且关系两端均需连接对象, 因此基于 `mxGraph` 中 `insertEdge()` 插入函数需要传入 `target` 和 `source` 参数, 因此前端渲染流程中应先插入对象, 再插入关系。具体流程如下: 首

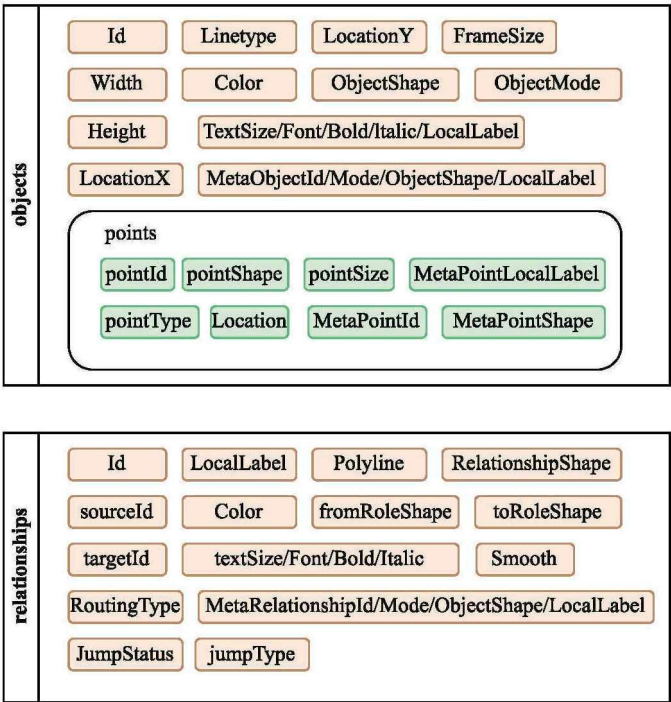


图 3-7 KARMA 模型中的普通图信息

先，遍历模型中对象关键信息，获取每一个对象关键信息，通过 *insertVertex()* 函数插入对象。其次，判断该对象是否存在点，如果不存在点，则判断该对象是否为最后一个对象。如果存在点，则遍历对象关键信息中的点关键信息，获取当前点关键信息通过 *insertVertex()* 方法以对象为父节点插入子节点，判断当前点是否为最后一个点，如果不是，继续遍历下一个点通过 *insertVertex()* 进行插入，如果是最后一个点，判断该点的父节点是否是最后一个对象，如果不是最后一个对象，则继续遍历下一个对象，重复上述流程，如果是最后一个对象，则该模型中对象的渲染完成，进入模型关系的渲染流程。

首先遍历模型关键信息中关系关键信息，获取每条关系的 *sourceId* 和 *targetId*，遍历当前模型中所有插入的对象，判断对象的 *Id* 是否等于 *sourceId*，如果与 *sourceId* 相同，则该对象为当前关系的 *source*，如果对象的 *Id* 不等于 *sourceId*，判断该对象的 *Id* 是否等于 *targetId*，如果两个 *Id* 相等，则该对象为当前关系的 *target*，如果不等，判断该对象是否有子节点，如果没有子节点，则判断当前子节点是否为最后一个子节点，若不是最后一个子节点，则遍历下一个子节点，若是最后一个子节点，判断当前节点对应的父节点是否为最后一个对象，若不是最后一个对象，遍历下一个对象，若是最后一个对象，则通过 *insertEdge()* 方式插入关系，若该关系不是最后一条关系，则遍历下一条关系关键信息，如果是最后一条关系，则结束整个流程。



图 3-8 JSON 格式 KARMA 模型关键信息

以上是普通图的前端渲染流程，分解剖视图基于普通图进行搭建，分解剖视图支持模块组织和结构分解、层次化结构展示、模块间关系展示、支持系统演化和变更管理、支持团队协作等。下面介绍分解剖视图的数据处理和渲染流程。分解剖视图如图3-9所示。

图3-9展示的是分解剖视图，对象一中展示的是分解图。对象二中包含一个分解图和两个剖视图，除了对象一和对象二的两种形式，对象三中首先包含一个分解图，分解图中包含对象四，对象四中包含分解图，该分解图中包含对象五和其他对象，对象五中包含两个剖视图。由上述分析可知，分解剖视图存在的形式多样，分解图、剖视图可分别单独存在于一个对象中，一个对象中也可同时存在分解图和剖视图，分解图中可以添加对象，对对象进行分解剖视，剖视图也可存在对象，对对象进行分解剖视以此迭代。

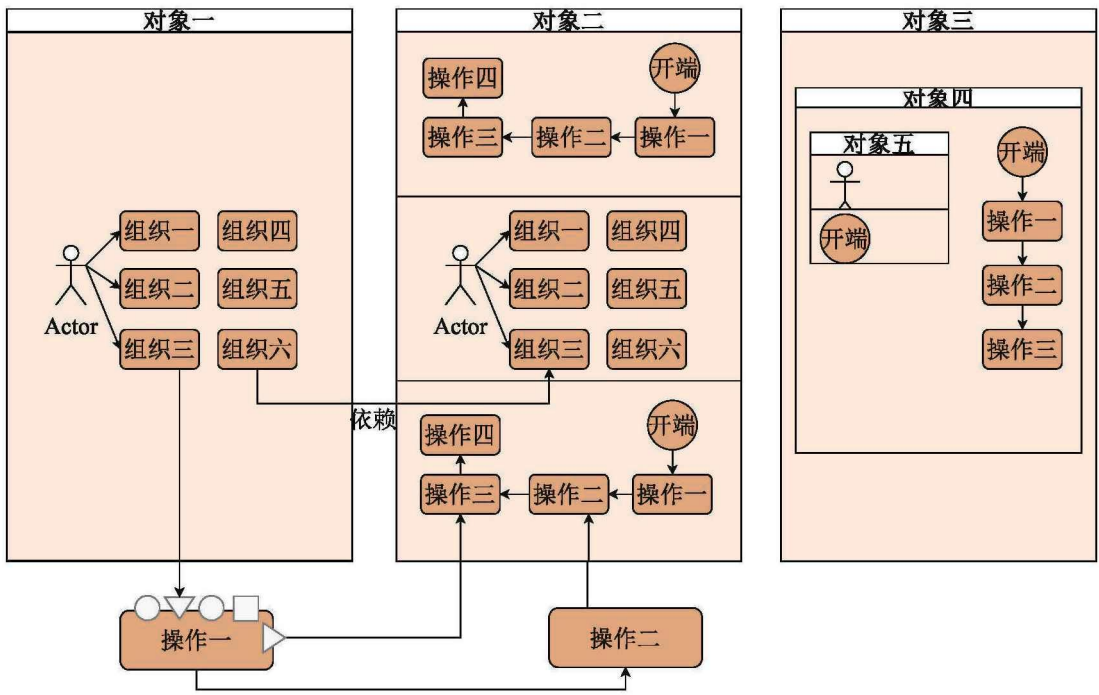


图 3-9 分解剖视模型展示

分解剖视图是基于模型的系统工程中用于描述系统的一种视图类型。通过对视图中的对象进一步进行分解剖视将系统中的各部分分解为更小的组件，通过分解剖视的形式可以将系统的颗粒度由粗到细进行描述，通过分解剖视保证了系统划分的一致性，提高了系统建模的准确性和可理解性。

分解剖视图的优势主要体现于以下几个方面，首先是将复杂系统进行简化，通过将系统各部分进行逐步分解和关联，用于清晰展示系统的架构、脉络和功能。其次，可以将系统通过分解剖视分解为模块化组件，通过模块化组件进行模块化设计，从而提高组件的可维护性，并且方便扩展。分解剖视图的分层和结构化模式，能够提高问题和潜在风险定位的效率，便于问题追踪和性能针对性调试。分解剖视图更利于厘清各组件之间的层级和关系，清晰的层级和关系利于各组件系统成员之间的协同。分解剖视图支持对系统的分解和重组，便于系统演化时进行组件优化。

以下是分解剖视图基于 mxGraph 进行渲染的前后端数据处理流程。

分解图中可以进行剖视，剖视图中可以进行分解，并且分解剖视图层数与实际系统架构相关，因此分解剖视图后端数据应支持无限迭代功能，采用模块化流程进行后端数据处理。如图3-10所示，首先将模型中的对象作为输入，遍历当前模型中所有对象，判断当前对象是否包含分解剖视图，若不包含，则判断当前对象是否是最后一个对象。若包含分解图，则将分解模型存储在分解模型数组中，下一

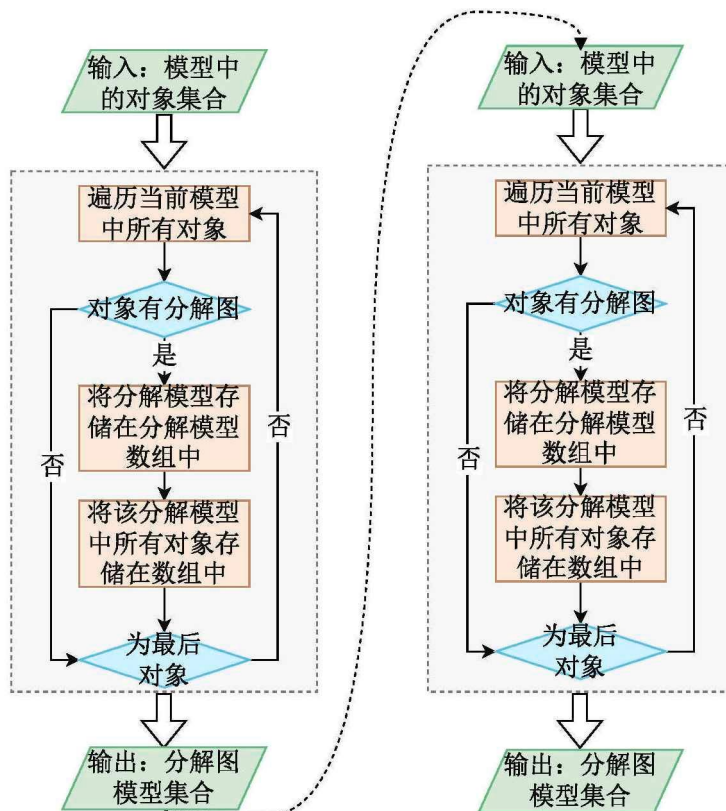


图 3-10 分解图后端处理流程

步将分解模型中所有对象存储在对象数组中。然后判断包含分解图的对象是否为最后一个对象，如果不是，则遍历下一个对象，如果是，则将当前对象数组和模型数组进行输出。并且将输出对象数组作为下一模块输入，直至输出不包含对象。

同理，剖视图后端数据处理也支持无限迭代，如图3-11所示，与分解图后端处理不同之处在于，一个容器模式对象中一层最多包含一个分解图，但可能包含一个或一个以上剖视图，因此剖视图后端数据处理流程中增加了对该图层所有剖视图进行遍历的处理。

通过上述后端处理，可以获取一个模型中所包含的所有分解剖视图对应的模型数组。对数组进行遍历，并且按普通图获取关键信息流程获取数组中每个模型的关键信息，传递至前端，用于前端渲染展示。

前端分解剖视图渲染流程如图3-12所示。

由于分解剖视图存在无穷迭代的可能性，因此传递至前端的数据会存在无穷层迭代，因此前端处理流程需考虑无穷层迭代的情况，通过模块化数据流解决无穷层迭代问题。如图3-12所示，模块化流程主要如下，首先遍历模型关键信息中的对象关键信息，通过 *insertVertex()* 函数插入对象，判断对象是否存在点，若对象不存在点，则遍历当前模型的对象关键信息，若对象存在点，通过 *insertVertex()*

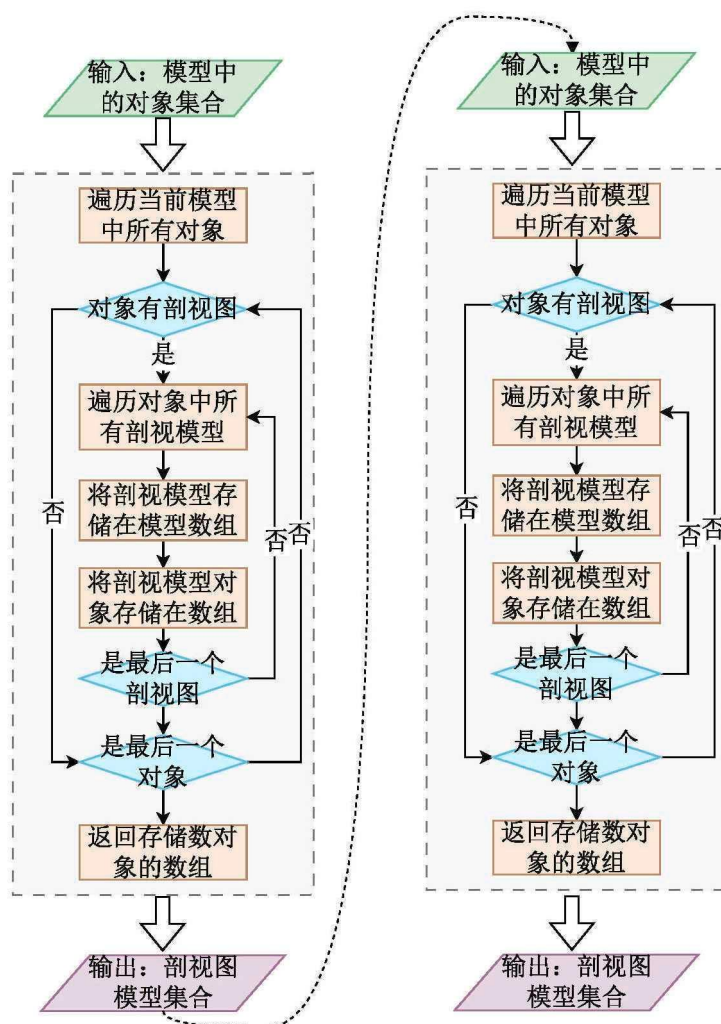


图 3-11 剖视图后端数据处理

函数添加点,再遍历当前模型对象关键信息,判断对象是否包含分解图,如果不包含,则判断对象是否包含剖视图。如果包含分解图,则创建字典用于保存分解图类型、*Id*、名称、元模型 *Id*。并将字典放入图数组中。判断对象是否包含剖视图,若不包含,判断图数组中是否有对象。若包含剖视图,则遍历所有剖视图,针对当前剖视图创建字典保存剖视图类型、*Id*、名称、元模型 *Id* 等,并将字典放入图数组中。判断当前是否为最后一个剖视图,若不是,继续遍历下一个剖视图,若是最后一个剖视图,则判断图数组中是否有对象,如果没有则退出该流程,如果有对象,则遍历图数组中所有对象,处理图数组中分解剖视图层、对象、关系的添加。

添加完当前层的图、对象和关系后,遍历当前图层所包含的所有对象,通过自定义的 *Id* 命名规则将图层对象和模型对象分别放入两个数组,并且返回由这两个数组组成的字典。判断字典是否为空。若不为空,说明当前模型还有更深一层分解剖视,则继续遍历更深一层图层。若为空,则表明当前图层为最深图层,该

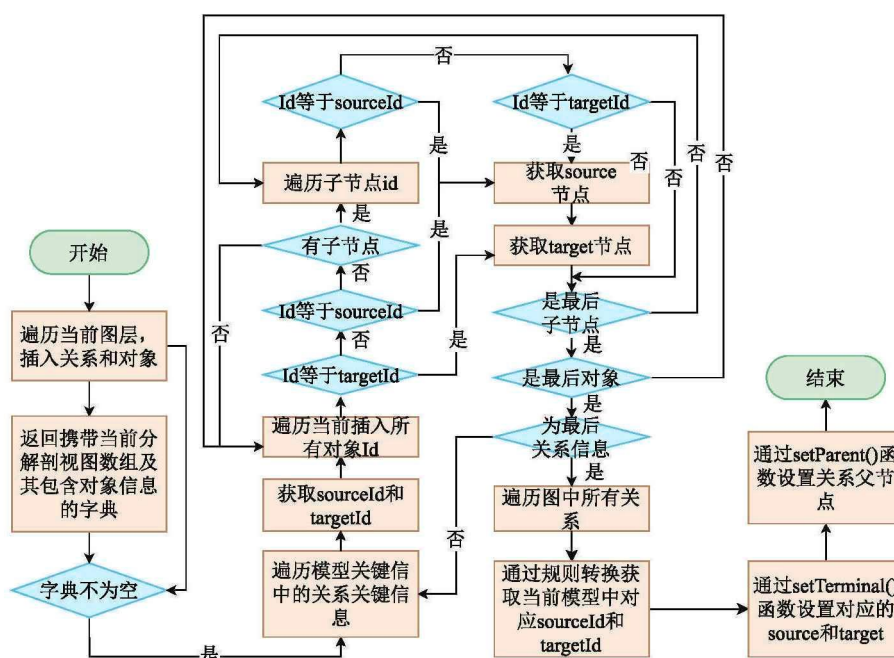


图 3-12 分解剖视图前端渲染流程

图层包含的模型不再进行分解剖视，并且进入跨图层的渲染流程。

通过上文对分解剖视图的介绍，除了分解剖视图内部的关系连接，分解剖视图中还存在同一层级分解图对象与剖视图对象之间的关系连接，以及不同层级分解剖视图对象之间的连接。因此在进行分解剖视图前端对象和关系渲染时，采取的策略是先渲染分解剖视图内部的对象，再渲染分解剖视图内部的关系，最后渲染跨图的关系。主要流程与普通图关系渲染流程类似，主要区别在于通过 *sourceId* 和 *targetId* 结合分解剖视图命名规则确定当前关系的 *source* 和 *target*，通过 *setTerminal()* 函数确定关系两端连接，通过 *setParent()* 确定关系所在图层。最终柔性建模技术中分解剖视图实现结果如图 3-13 所示。

如图 3-13 所示，图中展示了两层分解剖视图，每个分解剖视图都是一个独立的模型，并且分解剖视图可以拥有不同图元模型，分解剖视图均需要遵循对应图元模型中的约束规则。

时序图在 UML 建模语言中经常使用，支持对对象之间交互关系的描述。时序图优点体现在支持交互顺序可视化、支持行为预览和理解、支持异常流程的可视化、支持系统性能分析等。柔性建模技术通过构建相关元模型库支持针对模型的时序图构建，下面介绍时序图的后端数据处理及前端界面渲染。

时序图的后端数据处理与普通图的后端数据处理相同，在此不再赘述。时序图的前端渲染基于 *mxGraph*，*mxGraph* 本身不具备时序图样式，因此需要在对象样式表中进行时序图中生命线对象及执行器对象的注册。与普通图类似，前端能

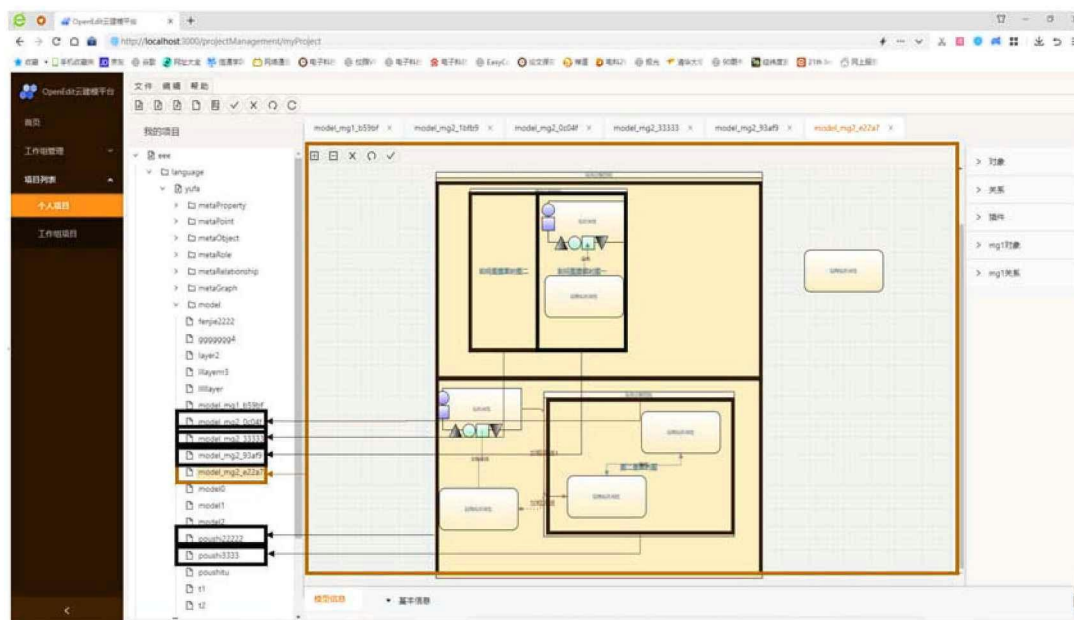


图 3-13 分解剖视图实现

够从后端获取时序图模型信息、对象信息及关系信息。首先遍历对象信息，将对象分为生命线和执行器两类，并且处理执行器对象时，获取其对应的生命线对象 *Id*，分类完成后，将生命线对象通过 *insertVertex()* 函数进行插入，所有生命线对象插入完毕后，再通过 *insertVertex()* 函数插入执行器，并且执行器的父节点是其对应的生命线对象。生命线对象及执行器对象均插入完毕后，遍历图中所有关系关键信息，获取每条关系对应起止点，通过 *insertEdge()* 函数插入关系。图3-14展示了智能电动汽车的时序图模型。

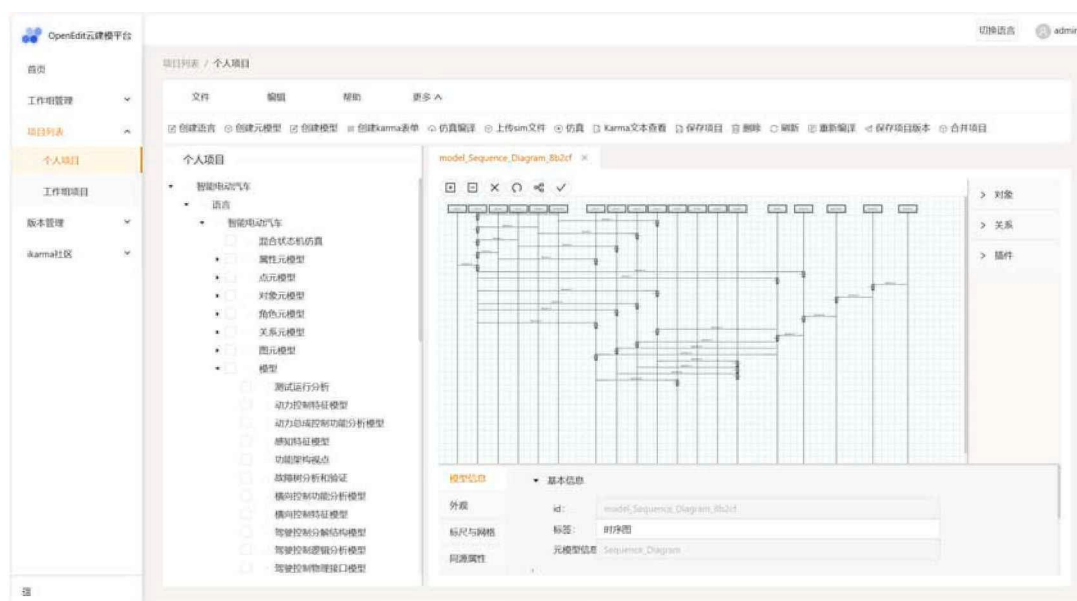


图 3-14 智能电动汽车时序图模型

3.3 基 GOPPRRE 建模方法的柔性建模技术

3.3.1 柔性建模技术总体架构

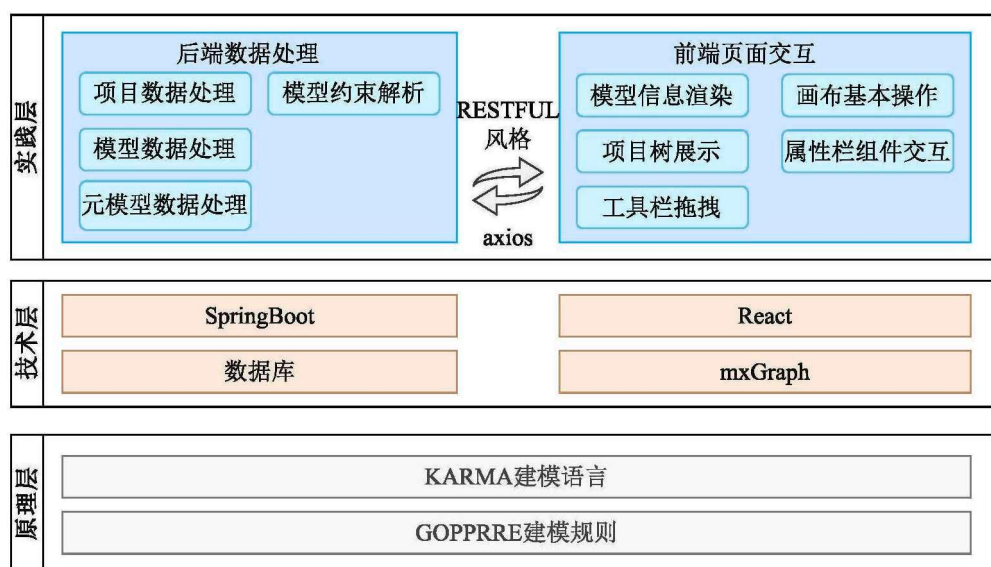


图 3-15 柔性建模技术总体架构

柔性建模技术总体架构如图3-15所示，主要包括原理层、技术层和实现层三个层次，整个柔性建模技术是基于 KARMA 语言和 GOPPRRE 建模方法实现，基于上述原理可以选择合适建模语言及建模框架进行元模型库构建，并且基于元模型库进行模型构建。通过 MOF 元模型架构，基于 GOPPRRE 建模方法设置基础元素包括图、属性、点、对象、关系及角色六类元元模型并且定义六种元素间的约束规则。在上述原理基础上进行平台搭建，主要包括后端数据处理及前端建模交互两部分，后端数据处理是基于 SpringBoot 框架及 MySQL 数据库，支持项目数据处理、模型数据处理、元模型数据处理及模型约束解析。前端页面交互主要包括模型信息渲染、项目树展示、工具栏拖拽、画布基本操作及组件栏交互等功能，上述功能主要通过交互界面及交互操作进行，交互界面由 React 框架进行构建，交互功能由 mxGraph 支持。以上是柔性建模技术的结构框架，图 3-16从柔性建模技术性能实现出发针对柔性建模平台优势进行介绍。

柔性建模技术柔性主要体现在包容开放性、灵活适应性、敏捷建模、系统整体性四个方面，其中，包容开放性是基于 KARMA 语言及 GOPPRRE 建模方法进行模型构建，支持各类建模语言及建模框架。灵活适应性基于柔性建模技术采用 MOF 元模型架构，支持针对各类建模语言构建自定义元模型库，支持针对复杂度不同的系统，通过本体技术自定义元模型库的复杂程度。敏捷建模通过用户交互

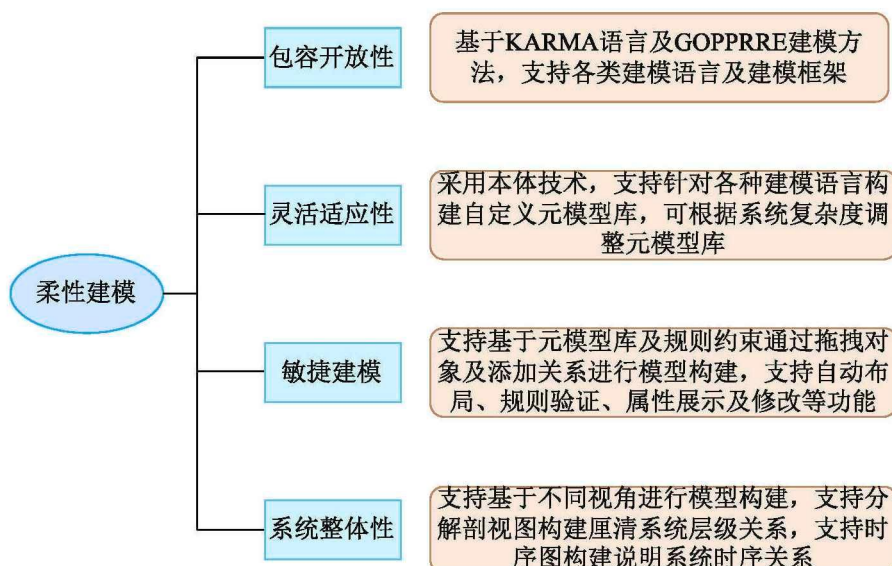


图 3-16 柔性建模技术性能架构

界面的交互操作及后端数据处理进行规则约束协同实现，具体包括通过模型规则约束进行模型及关系构建、模型自动布局、规则验证、属性展示及修改功能。系统整体性首先体现在柔性建模技术支持通过不同视角进行模型构建，其次通过构建分解剖视图厘清系统层级关系，最后通过支持时序图构建说明系统各活动时序关系。

3.3.2 柔性建模技术基本原理

柔性建模技术基于 GOPPRRE 建模方法进行模型构建，采用 `mxGraph` 进行前端渲染，支持敏捷交互，原理架构如图3-17所示。基于 GOPPRRE 建模方法的柔性建模技术包括元模型库构建及模型构建，在元模型库基础上进行模型构建。元模型库构建方式有两种，一种是基于包含元模型库的本体文件导入，另一种是通过柔性建模平台按照属性、点、对象、角色、关系、图元模型依次构建。元模型库构建过程中，后端主要支持元模型库的数据处理，前端通过 `React` 框架基于 `mxGraph` 支持用户交互。模型构建主要分为基于元模型库进行构建及基于本体导入模型，其中基于本体文件导入模型过程中，后端支持普通图、分解剖视图及时序图数据处理，前端通过 `mxGraph` 支持普通图、分解剖视图及时序图渲染。在基于元模型库进行模型构建流程中，后端支持元模型库数据处理，前端支持基于 `mxGraph` 的柔性建模基础操作、拖拽添加对象、添加关系。并且基于 `mxGraph` 支持模型元素自动布局、针对对象及关系添加进行约束等操作，上述操作中需要后端配合进行对象验证数据处理、关系连接数据处理及模型变更数据处理等。并且通过在前端进行数据缓存缩短柔性建模操作响应时间。

柔性建模技术的后端数据处理及前端界面渲染已在 3.2 小节详细介绍，本小节主要针对柔性建模技术中模型构建基础操作进行介绍，阐述了柔性建模技术中的对象添加约束及关系添加约束的原理及流程。

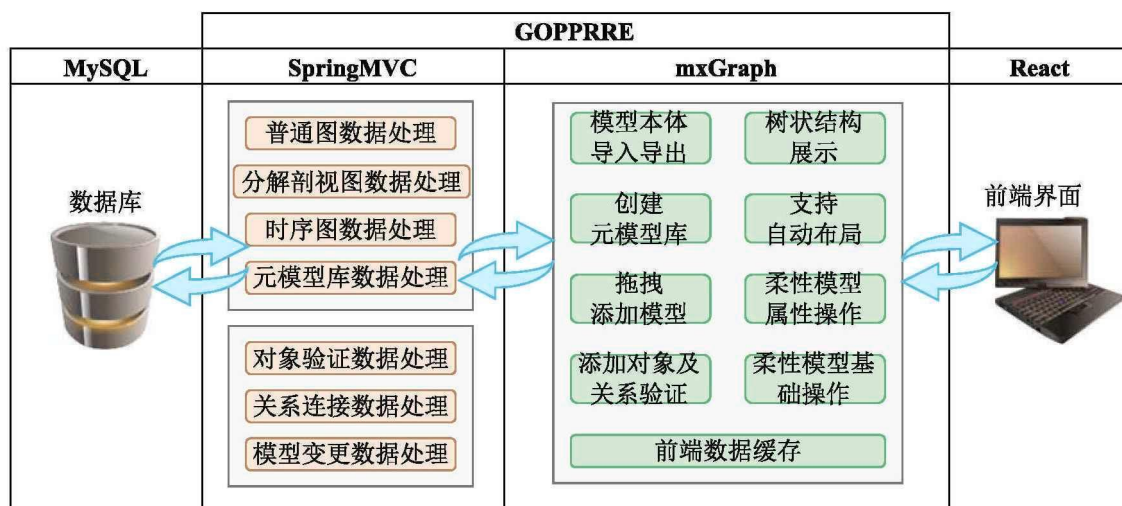


图 3-17 柔性建模技术原理框架

柔性建模技术支持基于图元模型的关系及对象约束规则进行关系和对象的添加，由 KARMA 语言基本规则可知，柔性建模技术中模型是基于图元模型构建的，图元模型中会进行对象及关系设置，在对象及关系设置中会将该图元模型允许添加的对象和关系加入对象元模型设置及关系元模型设置。并且在该图元模型中，支持对每类关系两端能够连接的对象或者点进行设置。基于以上设置，在基于图元模型进行模型构建时，需要基于图元模型中的规则进行约束。首先是模型中对象的拖拽建模，使用了 *mxUtils.makeDraggable()* 函数，该函数需要传入四个参数，分别是支持拖拽的图片、需要插入对象的 *mxGraph* 实例化图、拖拽函数及拖拽过程中显示的图标。上述函数支持基于 KARMA 模型规则实现拖拽生成对象，接收五个参数，分别是实例化的 *mxGraph* 图、触发事件、拖拽图标所在的目标对象、拖拽目标所在的横坐标及纵坐标，该函数具体实现流程如图 3-18 所示。

首先传入对象关键信息，判断该图是否能添加对象，若不能添加对象，则退出流程。若能添加对象，基于当前时刻随机生成字符串作为对象 *Id*，判断对象是否添加到现有对象中，若不是，则判断图类型是否为时序图，若是时序图，则基于对象关键信息添加生命线，结束流程。若图类型不是时序图，判断对象模式是否为容器模式，若是容器模式，则基于对象关键信息添加容器模式对象。若不是容器模式，解析对象关键信息，通过 *insertVertex()* 函数进行添加。若需要添加到现有对象中，判断对象是否为图层对象，若不是图层对象，则结束进程。若是图层对象，则获取图层对象元 *Id*，判断图层对象元 *Id* 是否等于对应图元 *Id*，若

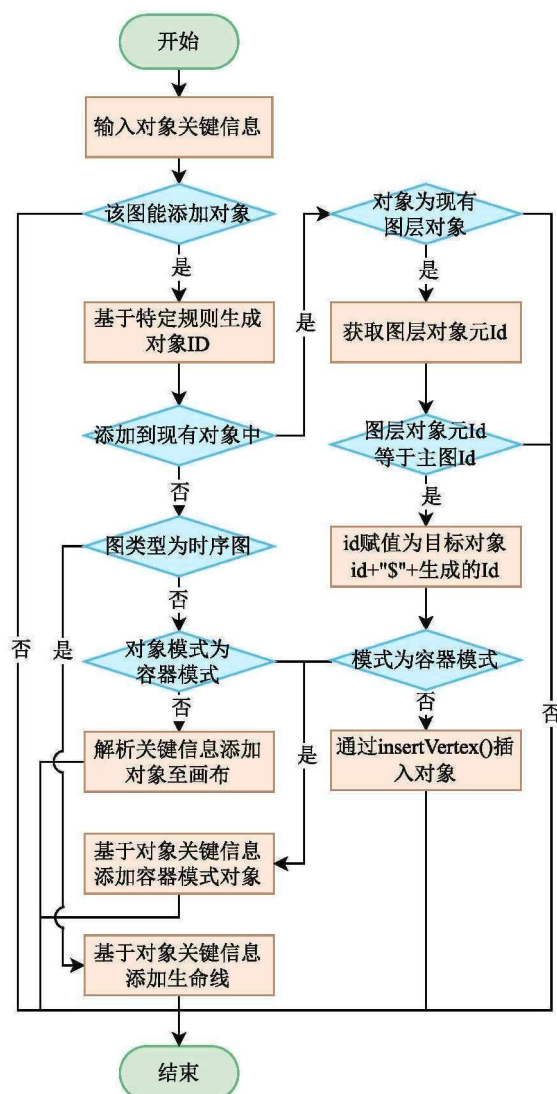


图 3-18 对象添加流程

不相等则直接结束进程，若相等，则将 Id 赋值为目标对象 Id 加上特定符号加上上文生成的 Id 。判断对象模式是否为容器模式，若不是，则解析对象关键信息通过 $insertVertex()$ 插入对象。若是容器模式，则基于对象关键信息添加容器模式对象。上述流程是支持基于 $mxGraph$ 渲染的柔性建模技术中对象拖拽的关键逻辑原理。

在基于 KARMA 语言的柔性建模中，支持点击关系后，选择两个对象进行关系连接。基于 KARMA 语言的柔性建模的关系插入中需要注意以下事项，首先是柔性建模技术中不允许关系任意一端没有对象，因此在进行关系插入中需要通过判断关系是否存在 $source$ 和 $target$ 来决定关系能否被插入。其次在普通图中，通过 $axios$ 向后端发送被选关系的相关信息，后端接收相关信息并且向前端发送两端可连接的对象元模型 Id ，前端接收 Id 并且进行处理，遵循柔性建模技术中的约

束规则进行 `mxMultiplicity` 类的实例化构建，以此约束关系的添加。但是在分解剖视图中，除了基于图元模型的关系连接设置的约束，还存在图元模型中是否允许该关系元模型添加的约束，因此，需要通过额外的逻辑进行判断。关系添加的主要流程如图3-19所示。

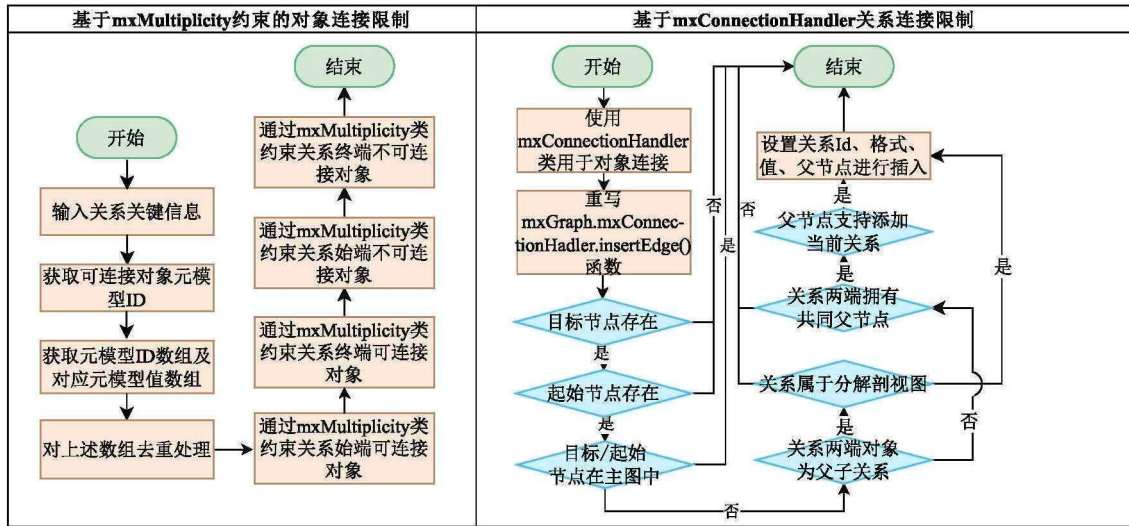


图 3-19 关系添加验证流程

关系连接约束主要由两部分组成，一方面是从对象的连接限制出发，使用 `mxMultiplicity` 类针对每一类对象进行实例化，通过限制关系输入输出、确定对象类型、限制关系连接数目、限制连接对象类型等约束当前画布所有对象的连接，主要流程如图3-19所示，首先传入关系信息，通过 `axios` 发送获取关系两端连接限制请求，获取关系两端允许连接的对象元模型 `Id` 的数组。遍历图中所有对象，将上一步元模型 `Id` 对应的对象数组及对象值的数组进行去重处理。为每一个允许关系输入的对象创建一个 `mxMultiplicity` 实例化类，并且将实例化出的 `mxMultiplicity` 类放入画布的 `multiplicities` 的数组中，以此将约束添加至模型画布中。为每一个允许关系输出的对象创建一个 `mxMultiplicity` 实例化类，并且将实例化的 `mxMultiplicity` 类放入画布的 `multiplicities` 的数组中，以此将约束添加至模型画布中。为每一个不允许关系输入的对象创建一个 `mxMultiplicity` 实例化类，并且将实例化出的 `mxMultiplicity` 类放入画布的 `multiplicities` 的数组中，以此将约束添加至模型画布中。为每一个不允许关系输出的对象创建一个 `mxMultiplicity` 实例化类，并且将实例化的 `mxMultiplicity` 类放入画布的 `multiplicities` 的数组中，以此将约束添加至模型画布中。

3.3.3 柔性建模方法技术实现

本文建模平台前端交互界面采用 React 框架，React 是由 Facebook 及相关公司开发及维护，主要用于 UI 层构建的 JavaScript 库。React 基于虚拟 DOM 进行展示，内容改变时进行虚拟 DOM 的差异渲染。并且 React 用户界面基于组件构建，组件之间代码相对独立。创建交互界面采用的是 JavaScript 的语法扩展 JSX，并且声明 UI 采用的是 XML 的标记方式。将生命周期引入 React 组件中，主要包括创建阶段、装载阶段、更新阶段和卸载阶段四个阶段。如图3-20所示，React 相较于传统的前端界面构建方式，具有更高渲染性能、易于复用和维护、易于理解和调试等优点。

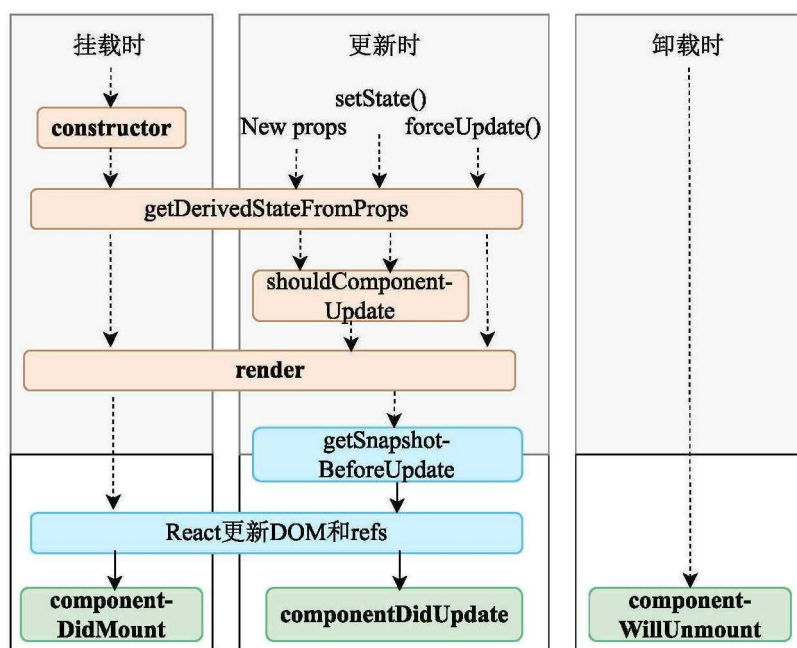


图 3-20 React 生命周期

如图3-20所示，核心有三个钩子函数，分别是 `componentDidMount`、`componentDidUpdate` 和 `componentWillUnmount`。`componentDidMount()` 函数中进行 DOM 操作、异步请求及获取数据等操作，主要在组件挂载于 DOM 后执行。在组件更新后触发 `componentDidUpdate()` 函数，支持处理更新后需要的操作。在组件卸载之前执行 `componentWillUnmount()` 函数，该函数中可以移走事件监听器、取消异步请求、移除定时器和取消订阅等。

由于 React 用户页面基于组件构建，本建模平台界面如图 3-21 所示，包括五个建模组组件，分别是项目树组件、头标签组件、画布组件、工具栏组件及属性栏组件。项目树组件主要用于项目结构树状展示，主要层级如下，项目名包含项目建模语言，一个项目可包含多种建模语言，每个建模语言下包含混合状态机仿真、

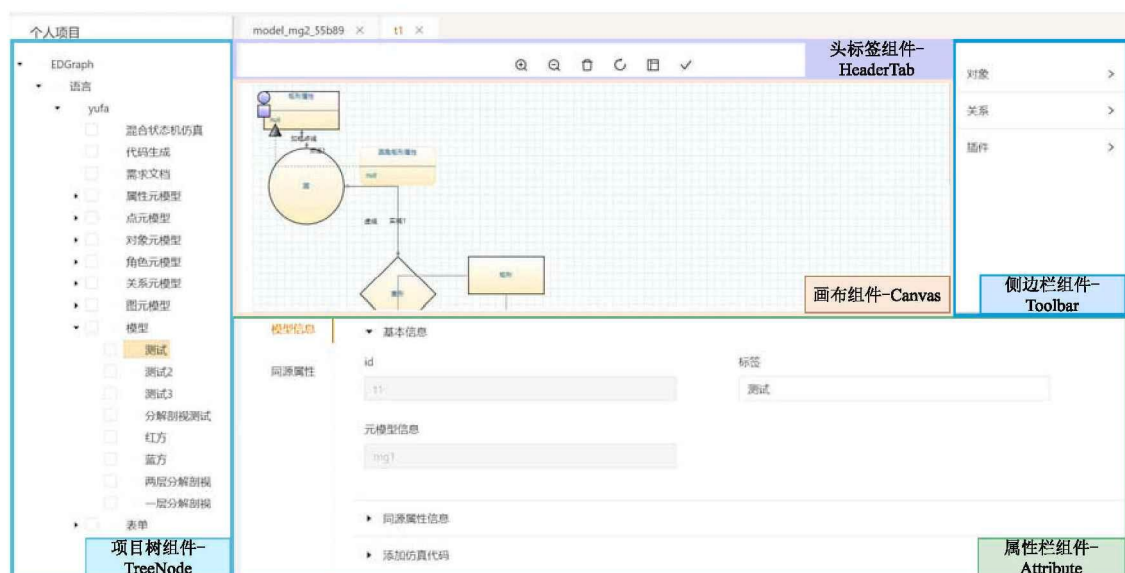


图 3-21 基于 React 前端页面组件

属性元模型、点元模型、对象元模型、角色元模型、关系元模型、图元模型、模型及表单。画布组件提供模型构建交互页面并且进行展示。头标签组件提供放大、缩小、删除、自动布局、保存五个按钮。分别支持针对画布进行放大缩小，针对画布中元素进行删除，针对画布中元素进行自动布局，针对画布进行保存。工具栏组件由折叠面板组成，支持展示该模型允许添加的对象、关系及插件。属性栏组件由标签页及折叠面板等组成，鼠标选中对象、关系或模型时，属性栏展示对应元素的相关属性。鼠标选中模型时，属性栏包括模型信息、外观、标尺与网格及同源属性。其中模型信息包括 *id*、标签、元模型等基本信息，包括同源属性信息及仿真代码信息。外观包括文字颜色、背景颜色和线条颜色。标尺与网格包含显示、测量和网格线等信息。鼠标选中对象时，属性栏包含对象信息、对象点信息、外观、同源属性、样式、分解与剖视。其中对象信息包括 *id*、标签和元模型信息等基本信息，并且包括同源属性信息及对象中的仿真代码。对象点信息包括输入点、输出点及无向点。外观包括对象中文字的样式信息。样式包含边框样式、文字样式、背景颜色等。分解与剖视属性主要针对分解剖视图，包括分解图 *id* 和分解图名称，剖视包括剖视图 *id*、剖视图名称及对应操作按钮。针对关系元素，其属性包含关系信息、角色信息、外观、同源属性及样式等。其中关系信息包含基本信息、同源属性信息及仿真代码。角色信息包含始端角色信息和终端角色信息。外观包括关系描述文字的字体和颜色、关系布线样式、关系平滑度及跳转连接。

图3-21中每个模块都对应支持柔性建模技术的具体功能，下面分别进行介绍，首先是工具栏组件，工具栏组件对应柔性建模技术中对象及关系展示及基于

KARMA 模型约束规则进行模型及关系添加的功能，首先是对象的展示。

柔性建模技术后端采用 SpringMVC 框架进行构建，通过 DispatcherServlet 可以将请求分发到不同处理器，基于 DispatcherServlet 设计 Spring 的网页框架。DispatcherServlet 实际是一个继承自 HttpServlet 基类的 Servlet。Spring MVC 框架以请求为驱动，通过中心 Servlet 分派各种请求并且提供各种功能，SpringMVC 原理如图3-22所示。

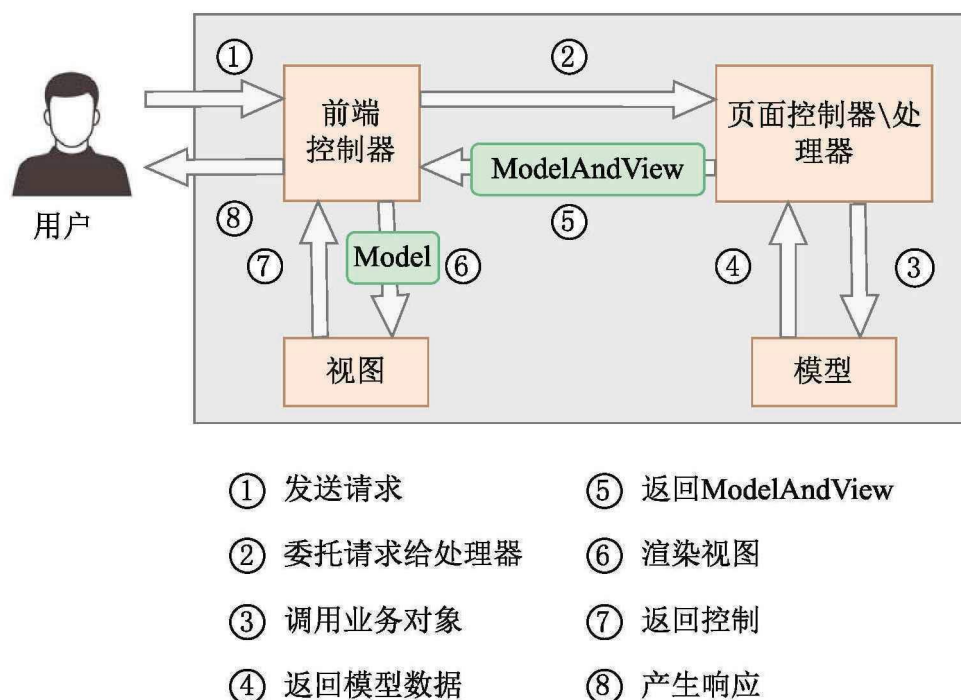


图 3-22 SpringMVC 操作流程

SpringMVC 操作流程如图3-22所示，首先用户向前端控制器发送请求，前端控制器接收到请求后，将请求委托给页面控制器或页面处理器，由页面控制器或处理器对请求进行处理，调用业务对象进行数据模型的创建，并且给中心控制器返回模型数据，中心控制器使用模型与视图对视图结果进行渲染，并且将渲染结果传递给前端控制器，最终将结果返回给请求者。

基于 React 和 mxGraph 可实现常见基础功能，支持柔性建模技术中基于画布的用户交互，通过可视化及用户友好界面提高用户建模交互体验。支持建模过程中的粘贴复制及撤销操作，支持选中所有关系、选中所有对象及选中所有元素等功能。

在柔性建模中，关系两端须连接对象，因此不支持针对单独关系的复制粘贴。并且在柔性建模中，模型是图元模型的实例化，因此模型中的复制粘贴需要遵循对应图元模型的约束，在通过右键菜单针对对象进行复制后，通过右键菜单进行

粘贴时通过粘贴位置，结合图元模型约束针对能否粘贴进行判断。在拖拽对象添加进入画布及选择关系起止端进行关系插入过程中，同样涉及基于图元模型的对象添加约束及关系连接验证，具体原理详见 3.2.2 小节，具体实现效果如图3-23所示。

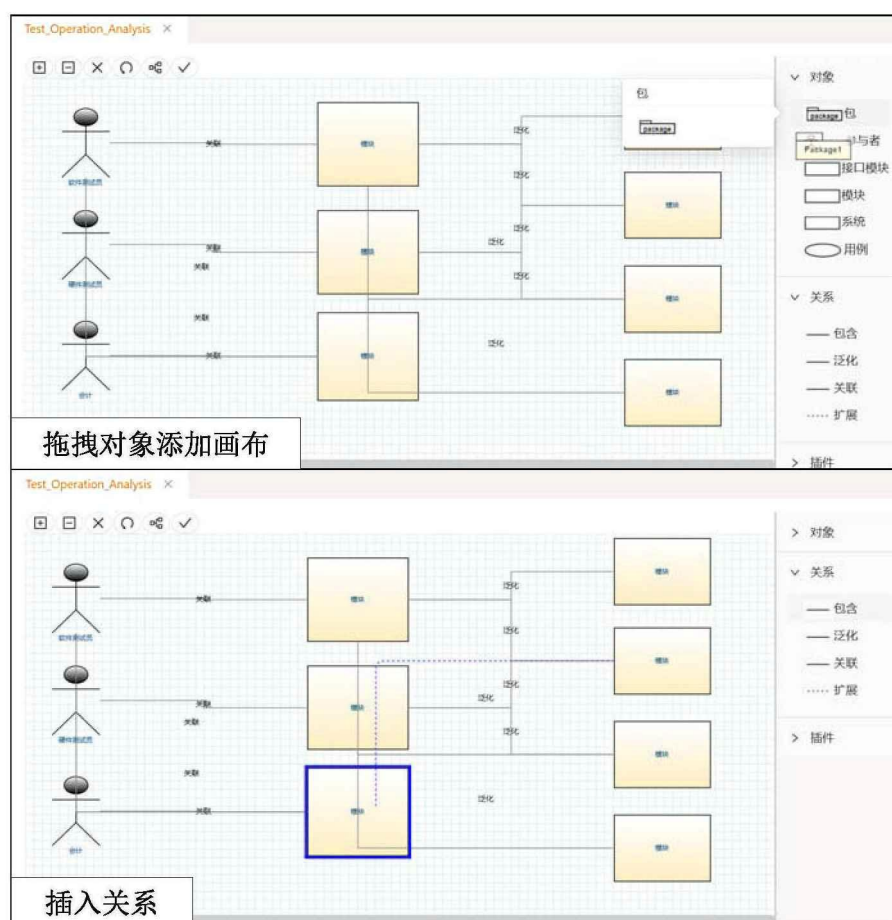


图 3-23 对象关系添加实现

图3-23展示了通过拖拽添加对象，通过选择起止端添加关系。在添加对象及关系的过程中，画布会基于图元模型规则针对对象及关系添加增加进行约束。满足约束能够进行添加，不满足约束前端会弹出警告。

柔性建模技术中的自动布局技术基于 mxGraph 中提供的类，包括 mxLayoutManager 基类及继承 mxLayoutManager 的子类 mxFastOrganicLayout 类、mxCircleLayout 类、mxHierarchicalLayout 类等。mxFastOrganicLayout 类是有机布局，支持快速进行组织状的布局，并且每次布局都是随机的，只对连接的对象有作用。mxCircleLayout 类是圆形布局，支持针对对象进行圆形布局。mxHierarchicalLayout 类是分层布局。上述类中存在相关的基本排布算法，可直接作用于画布元素。通过针对上述类进行实例化，并且适应于画布上的元素，最终实现效果如图3-24所示。

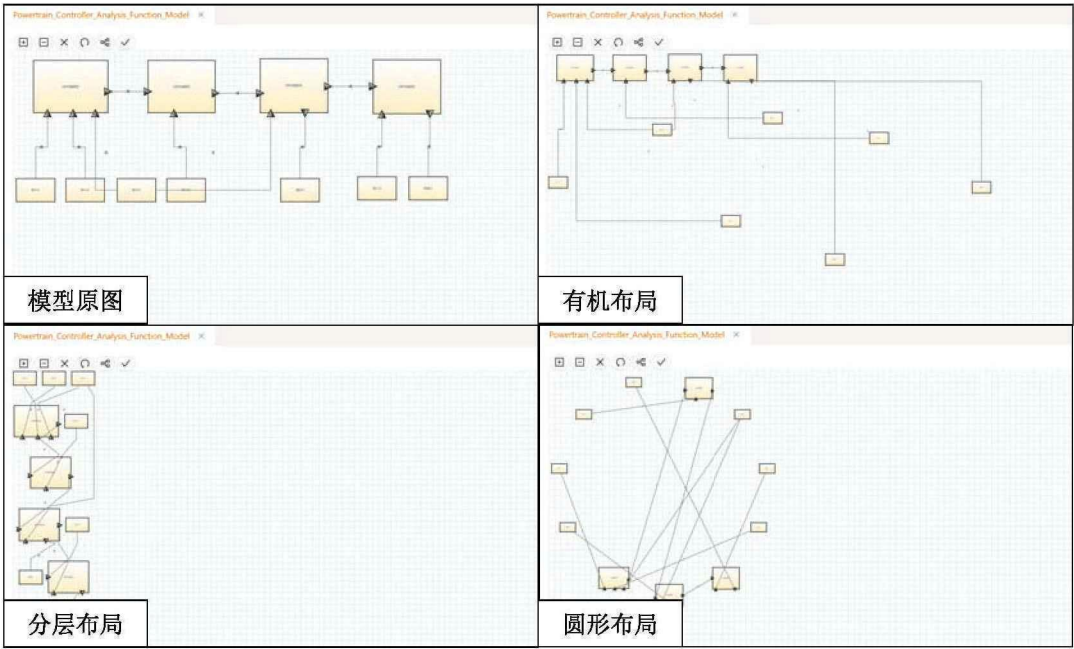


图 3-24 柔性建模自动布局实现

上述模型是由用户创建的全自动驾驶控制器分析功能模型，图3-24分别展示了全自动驾驶控制器分析功能模型原图，进行有机布局的全自动驾驶控制器分析功能模型，进行分层布局的全自动驾驶控制器分析功能模型，进行圆形布局的全自动驾驶控制器分析功能模型。

3.4 应急救援工程的柔性建模与分析

3.4.1 案例背景

英国救援组织体系负责针对海洋、土地和飞机及进行大规模搜救。搜救责任区主要包括东部搜救区、西苏格兰和北爱尔兰搜救区和南部搜救区。并且与搜救责任区相关的国家进行搜救协议的签订，例如葡萄牙、美国、西班牙、加拿大、法国、爱尔兰、德国、冰岛、丹麦和挪威等。

在组织结构上，英国救援组织包括航空器搜救协调机构、海上搜救协调机构及陆地搜救协调机构三类机构。其中搜救民用及军用的航空器相关工作主要由航空器救助协调中心负责，该中心由英国国防部和运输部共同成立。海上搜救协调机构包括运输部，运输部属于海事和海岸警备署，该机构设立了海上搜寻和救助协调中心，该中心主要负责海上搜救组织相关的协调工作。

本文通过柔性建模技术参考英国救援体系中海上应急救援工程进行基于模型的系统工程构建，结合 UPDM 建模标准构建定制化元模型库，并且基于元模型库进行模型构建，描述针对海上遇险船只的搜索和救援作战。主要流程如下，首先

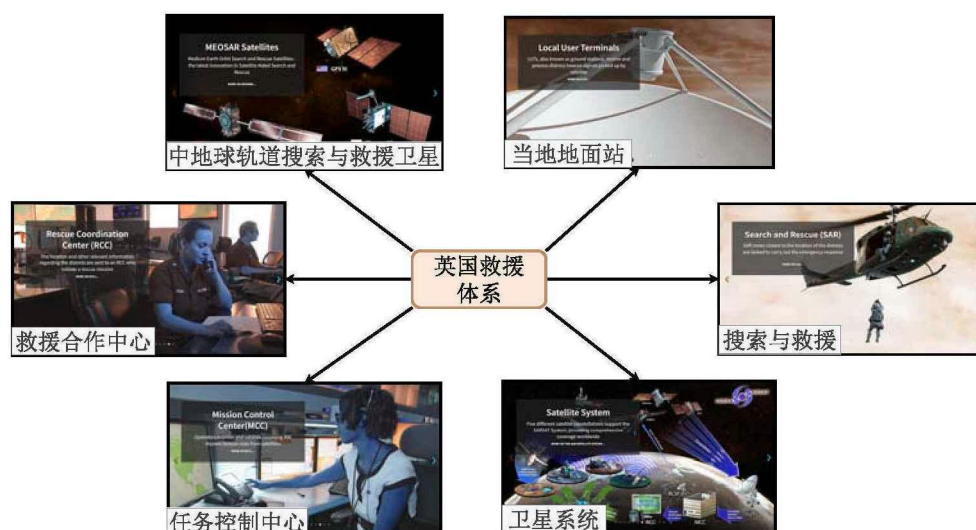


图 3-25 英国救援体系概述

遇险船只发出求救信号被监控单元捕捉到，其次监控单元将收到的求救信号发送给控制指挥中心。控制指挥中心对求救信号进行分析，根据分析结果协调皇家国家救生艇学会、直升机及海军舰艇进行搜救行动。通过柔性建模技术支持灵活敏捷构建细颗粒度的模型。

3.4.2 模型构建

使用柔性建模技术针对海上应急救援工程进行模型构建主要流程如下，首先确定选择 GOPPRRE 建模方法，结合 UPDM 建模框架进行元模型库构建，元模型库包括图元模型、对象元模型、属性元模型、点元模型、角色元模型及关系元模型。各元模型数量如表3-8所示。

表 3-8 元模型数量统计

元模型种类	图元模型	对象元模型	点元模型	属性元模型	角色元模型	关系元模型
数量	24	96	6	111	85	33

具体元模型库的构建流程是，首先构建属性元模型，属性元模型是元模型库中最基础元模型，可附着于对象、关系、角色、点、图元模型上。表3-9是基于 UPDM 建模框架针对海上应急救援案例构建的属性元模型概览。

图3-26是柔性建模平台属性元模型构建页面及海上应急救援案例中构建的属性元模型。属性元模型构建页面主要包括属性的本地名称、数据类型、单位及描述。

属性元模型构建完成后，进行点元模型构建，本案例包含的点元模型如表3-10所示。

表 3-9 属性元模型

属性元模型				
可说明性	实现效果	执行活动环境	实际测量设置	实际资源
带注释的元素	批准机关	架构师	架构框架	假设和约束
属性	作者	行为元素	行为类型	行为类型
利益	主体	能力任务	连接	上下文
传达	创建组织	自定义类	完成日期	可延期的触发
被描述于	学说	文档	都柏林核心元素	企业阶段
入口	事件元素	事件类型	能力展示	出口
功能	泛化组	保护	标识符	所需功能
实现者	处于状态	传入	是否抽象	是否活动
能力分解	功能组成	作战活动	资源执行者组成	是否共轭
实际企业阶段	作战执行者	由作战活动映射	是否只读	是否单一执行
授权日期	方法使用	里程碑	多重度	名称
节点	观察	传出	拥有的作战	拥有的参数
分区	作战执行者	执行功能	周期性	物理位置
端口	概率	目的	量种类	速率
批准组织	实现接口	建议	重定义节点	重定义转换
环境需求	资源	退役日期	安全可靠影响	安全等级
安全保密性影响	大小	规范	相关方关注点	开始日期
状态不变量	状态	子机	总结发现	投入产出比
时间性	是否存在	待办事项	工具使用	事务类型
类型	单位	URI	里程碑发布的版本	发布的版本
里程碑撤销的版本	撤销的版本	版本	视点	

表 3-10 点元模型

点元模型					
连接点引用	输入引脚	作战端口	输出引脚	伪状态	资源端口

图3-27是柔性建模平台点元模型构建页面及海上应急救援案例中构建的点元模型。柔性建模技术为从较细颗粒度描述模型，从总览及属性两部分构建点元模型，支持点元模型的本地名称、描述及固定语法。并且点元模型的属性包括属性引用标识、属性元模型标识、属性类型，属性唯一性及属性操作。

点元模型构建完成后，进行对象元模型构建，本案例包含的对象元模型如表3-11所示。

图3-28是柔性建模平台对象元模型构建页面。对象元模型的基本信息主要包括本地名称、描述、固定语法及显示属性。在基于 GOPPRRE 建模方法的柔性建模技术中，对象元模型上可附着点元模型，因此对象元模型中需要针对点进行设置，需要设置点元模型、点引用 Id、点方向及操作等。对象元模型同样支持属性设置，属性设置包括属性引用 Id、数据类型、方向及固定语法等。通过以上三部分构建对象元模型实现柔性建模。



图 3-26 属性元模型概览



图 3-27 点元模型概览

表 3-11 对象元模型

对象元模型				
激活	活动参数节点	实际组织	实际项目里程碑	实际项目
架构描述	架构元数据	能力配置概念角色	能力配置	能力属性
能力	选择为状态	复合状态	概念角色	条件
数据元素	决策节点	深层历史伪装态	企业目标	企业阶段
实体项目	交换元素	退出点	最终节点	最终状态
分支节点	分支伪装态	功能动作	功能	功能标准
高层作战概念	增加里程碑	信息元素	初始节点	初始伪装态
接口	汇合伪装态	连接伪装态	泳道	生命线
位置概念角色	位置类型角色	位置类型	位置	逻辑数据模型
汇合节点	元数据	使命	作战状态	作战角色
作战活动	作战活动动作	作战动作	模型	组织类型概念角色
组织类型	组织	正交状态	停止使用里程碑	执行者概念角色
职位	物理数据模型	人员	人员类型	人员类型概念角色
执行者	项目活动	项目里程碑	项目类型	项目
协议	资源构建	服务功能动作	服务访问	责任
资源状态	资源角色	资源参数	服务接口	服务参数
服务规范角色	服务规范	浅层历史伪装态	技能	系统概念角色
子机状态	状态指示器	标准配置	软件	软件概念角色
系统	技术标准	终止伪装态	视图	构想说明
构想				



图 3-28 对象元模型概览

对象元模型构建完成后，进行角色元模型的构建，本案例包含的角色元模型如表3-12所示。

表 3-12 角色元模型

角色元模型				
聚合角色	客户	组合角色	契约	定向角色 A
流来自组织资源	流向组织资源	来自实际里程碑	来自实际责任资源	来自架构描述
来自能力元素	来自概念角色	来自预期	来自实现	来自执行者
泛化部分	实现部分	部分	接收事件	角色 A
角色 B	发送事件	来源	具体部分	供应者
目标	到活动	实际里程碑	到实际项目	到实际状态
到架构描述	到能力	到概念角色	到规范	整体

图3-29是柔性建模平台角色元模型构建界面。角色元模型主要从基本信息及属性两方面进行描述，首先基本信息包括本地名称、描述、固定语法及方向，方向用于确定关系的两端的连接情况。属性包括角色所需属性引用 Id、属性元模型 Id、属性类型、唯一性及相关操作。



图 3-29 角色元模型概览

角色元模型构建完成后，进行关系元模型的构建，本案例包含的关系元模型如表3-13所示。

图3-30是柔性建模平台关系元模型构建界面。在基于 GOPPRRE 建模方法的柔性建模技术中，关系元模型上可附着角色元模型，角色元模型可对关系两端连

表 3-13 关系元模型

关系元模型				
能力的活动部分	项目的活动部分	执行者执行的活动	聚合	任意关系
架构引用	信息关联	关联	执行者能力	命令
组合	包含	定向组合	定向关联	定向聚合
细节	预期效果	依赖	展示	泛化
实现	接口实现	能够胜任	消息	负责
资源连接器	回复消息	作战对象流	作战控制流	里程碑
服务连接	转换	使用		

接进行约束。因此关系元模型进行构建时，需要对关系元模型本地名称、描述、固定语法及图表等基本信息进行设置，还需要附着在关系上的角色信息进行设置，包括角色标识、数据类型、方向及固定语法等信息。最后需要对关系元模型进行属性元模型的设置。附着在关系上的角色在图元模型构建时，由于不同图元模型中的关系两端的约束不同，因此在图元模型构建时，需要针对关系元模型两端可选择的对象进一步进行约束设置。



图 3-30 关系元模型

关系元模型构建完成后，进行图元模型的构建，本案例包含的图元模型如表3-14所示。

表 3-14 图元模型

图元模型		
AV-1 概要信息	CV-1 构想	CV-2 能力分类
CV-4 能力依赖	DIV-1 概念数据模型	DIV-2 逻辑数据模型
DIV-3	模型视图	OV-1 高层作战模型
OV-2 作战资源流描述	OV-4 组织关系图	OV-5a 作战活动分解图
OV-5b 作战活动模型	OV-6b 状态转移描述	PV-1 项目组合关系
时序图	SV-1 系统接口描述	SV-10b 系统状态转移描述
SV-2 系统资源流描述	SV-4 系统功能描述	SvcV-1 服务环境描述
SvcV-10b 服务状态转移描述	SvcV-2 服务资源流描述	SvcV-4 服务功能描述

图3-31是柔性建模平台图元模型构建界面。图元模型可实例化为模型，因此

图元模型需要进行本地名称、描述及类型等基本信息设置，并且针对图中的关系及对象等进行约束限制。柔性建模技术支持普通图、分解剖视图及时序图的构建，因此需要在图元模型中进行图元模型类型设置，用于图元模型实例化为模型时的类型标识。由于在不同建模语言及建模框架中，通过在不同的图元模型中设置该图元模型允许加入的对象及关系元模型，实现图元模型中元素的约束。通过在图元模型中设置关系两端可以连接的角色，实现关系两端的连接限制。



图 3-31 图元模型概览

3.4.3 柔性建模性能分析

柔性建模技术基于 GOPPRRE 建模方法进行模型构建，GOPPRRE 建模方法是以元对象机制为基础，将元对象机制作为元模型结构来定义柔性建模语言。柔性建模技术的元模型结构包括对象、点、属性、角色、关系及上述元素之间的连接，并且结合建模语言或建模框架，进行定制化的元模型库构建，因此本文的柔性建模技术支持各类建模语言及建模框架。本文研究中基于柔性建模技术构建各类建模语言及建模框架的元模型库。如表3-15所示。由于支持从元模型库进行构建，因此，可基于建模对象的复杂程度进行元模型库的补充或删减，并且支持自定义建模框架，表3-15中的自定义电车框架中结合了 UPDM、UML、SYSML、BPMN 及电子体系结构和软件技术体系描述语言（Electronics Architecture and Software Technology - Architecture DescriptionLanguage, EAST-ADL）等建模语言和建模框架，支持按需选取需要的结构针对电车模型进行定制元模型库的构建，增强柔性建模技术的灵活性、适应性及完备性。

为说明本文的柔性建模技术相较于目前主流建模语言进行模型构建具有较细的颗粒度，下面将基于柔性建模技术结合常见建模语言或建模规范构建的项目导出为本体形式，通过采用 OntoQA 方法基于不同维度对柔性建模技术性能进行定量分析。基于元对象机制的柔性建模技术可将项目导出为本体格式，并且可以基于本体文件数据进行柔性建模技术的性能分析，表3-16统计了 UPDM 建模框架及

表 3-15 柔性建模技术元模型库

	图元模型个数	对象元模型个数	点元模型个数	属性元模型个数	关系元模型个数	角色元模型个数
SYSML	10	49	2	38	25	53
Archimate	25	60	6	56	11	130
BPMN	3	58	10	55	3	4
OPM	1	5	0	14	15	18
UAF	153	170	8	146	53	104
UPDM	24	96	6	111	33	85
自定义电车框架	13	56	17	32	26	52

MOFLP 建模框架采用 OntoQA 方法进行量化分析所需数据。

表 3-16 柔性建模技术本体数据类型统计

建模语言	类的数量	类具有属性的数量	有实例的类的数量	实例的数量
UPDM	1278	1631	1277	2837
MOFLP	478	635	106	4398

下面基于以上数据，采用 OntoQA 方法针对柔性建模技术性能进行定量分析，从实例指标及模式指标两方面进行分析，其中实例指标包括类丰富性、平均分布两个指标，模式指标主要指属性丰富性。下面分别进行介绍，首先属性丰富性计算公式如式3-1所示。

$$AR = |att|/|C| \quad (3-1)$$

其中， $|att|$ 代表类的数量， $|C|$ 代表类具有属性的数量， AR 表示基于柔性建模技术生成本体文件包含的领域知识。通常柔性建模技术可传递的信息和知识与属性的定义量成正相关。

类丰富性的计算公式如式3-2所示。

$$CR = |C'|/|C| \quad (3-2)$$

其中， $|C'|$ 表示有实例的类的数量， $|C|$ 表示有类的数量， CR 是针对实例如何跨类分布进行描述的。在柔性建模技术中， CR 值越高，表示元模型利用越充分。

平均分布的计算公式如式所示。

$$P = |I|/|C| \quad (3-3)$$

其中， $|I|$ 表示实例的数量， $|C|$ 表示类的数量， P 用于描述柔性建模中的类是否有充足的实例。 P 越高，表明柔性建模技术通过基于元模型库建模能够进行充分表达。

表3-17分别针对基于 UPDM 建模框架的海上救援项目及基于自定义电车框架的智能电动汽车项目进行上述指标的计算。

由表3-17可得，基于 UPDM 建模框架的海上搜救案例本体属性丰富性为

表 3-17 柔性建模技术性能

建模框架	属性丰富性	类丰富性	平均分布
UPDM	1.2762	0.999	2.2199
MOFLP	1.3285	0.22175	9.2008

1.2762，说明基于柔性建模的海上搜救项目本体具有丰富的属性，能够进行丰富的属性表达。其类丰富性为 0.999，因为该案例中只有一个点元模型未被使用，因此该案例具有极高的元模型利用率从而导致接近 1 的类丰富性。从类丰富性可以看出，柔性建模是首先构建 GOPPRR 元模型库，并且基于元模型库进行模型构建，因此可针对建模对象定制化构建元模型库，从而提供类丰富性。该案例平均分布为 2.2199，说明柔性建模技术支持构建足够实例，充分描述建模对象。基于 MOFLP 建模框架的智能电动汽车案例本体属性丰富度为 1.3285，说明基于柔性建模的智能电动汽车本体能够充分支持其属性表达。该案例本体的类丰富性较低，是因为基于 MOFLP 建模框架的元模型库元模型较多，在实际建模中部分元模型未使用，但其一体化本体结构整体分布良好。该案例的平均分布为 9.2008，表明基于柔性建模技术的该案例能够充分描述智能电动汽车的各种元素。

3.5 本章小结

本章首先介绍了现有柔性建模技术的缺陷，通过对比现有柔性建模技术选择 GOPPRRE 建模方法作为本文柔性建模技术的理论基础。

由于柔性建模技术需要进行云端部署，因此采用 mxGraph 作为柔性建模技术的前端渲染工具，采用 React 框架进行柔性建模前端页面构建。并且介绍如何解析基于 GOPPRRE 建模方法建模的模型关键信息，及介绍基于关键信息通过 mxGraph 进行渲染的流程。

最后本章针对海上应急救援工程进行案例构建，基于柔性建模技术结合 UPDM 建模规范构建元模型库，基于元模型库进行模型构建。证明本文柔性建模技术的有效性及其可行性。通过基于 SYSML、Archimate、BPMN、OPM、UAF、UPDM 及自定义电车框架等建模语言及建模框架进行元模型库构建。证明柔性建模技术覆盖所有建模语言，并且支持较细颗粒度建模。此外，本章构建基于 UPDM 及 MOFLP 框架的元模型库及模型，通过 OntoQA 方法针对上述案例导出的本体文件进行定量分析，通过属性丰富度、类丰富度及平均分布进行柔性建模技术性能分析。证明柔性建模技术支持较细颗粒度建模并且具备较好语义表达能力，为下一章基于云架构协同的多域柔性建模技术提供支持。

第四章 基于云架构的多域柔性建模技术研究

上一章对柔性建模技术的 GOPPRRE 建模方法、模型关键信息提取、mxGraph 前端渲染流程及基于 React 框架建模平台的构建进行说明,通过基于柔性建模技术构建海上应急救援案例及结合各种建模语言及建模规范构建元模型库,通过 OntoQA 方法针对柔性建模技术进行性能分析。

本章将在柔性建模技术基础上,基于 Convergence 协同引擎开发多域建模技术,实现用户项目管理及多人实时在线协同建模,通过使用分布式 OT 算法及基于协同画布的标记共享解决协同操作冲突问题。最后通过针对海上应急救援工程进行基于云架构协同的多域柔性建模,进行多域建模性能分析。

4.1 多域建模技术研究

多域建模技术通过支持复杂系统的跨领域建模,减少跨专业领域专家之间理解隔阂,促进其交流与协作。多域建模技术具备综合建模能力,支持跨领域集成、多视角支持、模型同步及一致性管理、扩展性和定制性操作。综合建模能力指平衡不同领域建模的影响和关系,支持针对多个领域要素建模。跨领域集成指通过将不同领域模型集成实现信息跨领域传递与共享。多域建模技术通过实现组织项目管理及跨领域实时协同建模来满足以上需求,提高系统分析和设计的全面性和准确性。

4.1.1 多域建模技术总体架构

多域建模技术支持不同领域组织进行协同构建,基于多域建模技术进行模型构建主要包括用户组织项目管理及实时在线协同建模两部分内容。其中用户组织项目管理总体框架如图4-1所示。

多域建模技术中,用户组织项目管理流程如图4-1所示,首先进行用户管理,由超级管理员添加或删除用户,用户管理包括用户名设置、用户密码设置、针对用户进行操作管理。每一个用户对应一个登录账号,支持通过登陆页面进行登录。第二步是创建工作组,支持用户进行工作组创建,工作组创建包括工作组名称、工作组相关描述、工作组项目管理及工作组成员管理,其中工作组项目管理支持针对工作组中的项目进行管理,工作组成员管理支持将当前用户可支配用户加入或移除当前工作组。通过工作组项目管理可以进入项目组管理页面,该页面支持项目添加或删除,支持针对该项目组中每个项目进行用户管理。支持创建一个新项

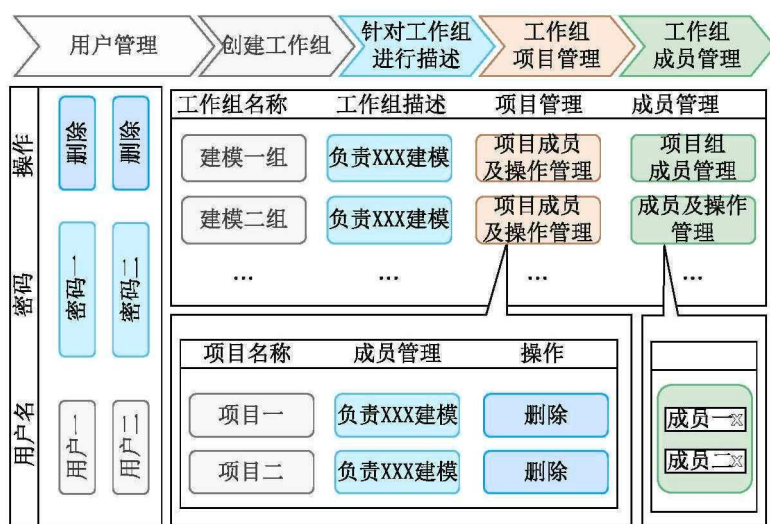


图 4-1 用户组织项目管理架构

目或基于本体文件导入的方式进行项目的创建。在每个项目的用户管理中，所管理的用户范围是在工作组用户管理中添加到该工作组的用户。

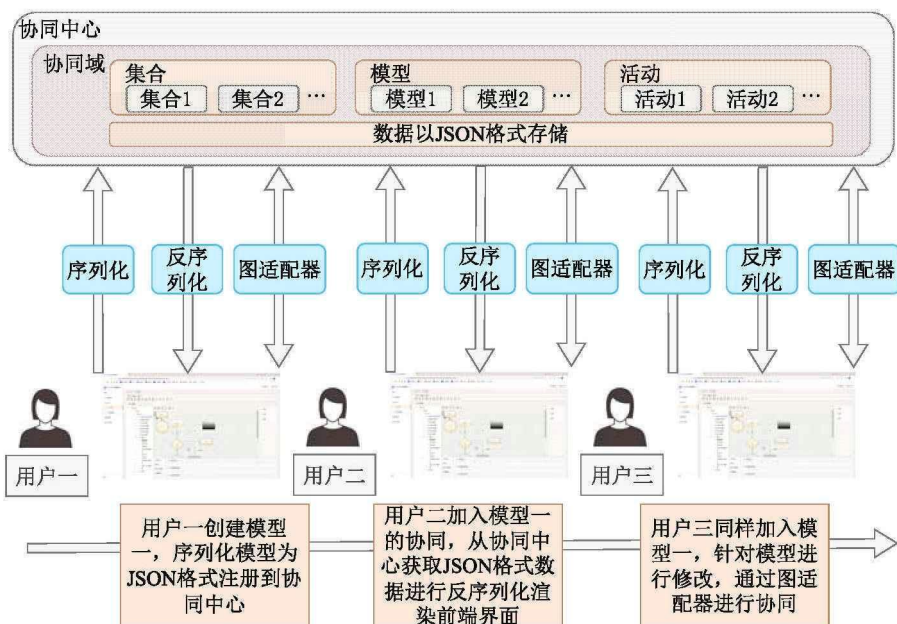


图 4-2 多人实时协同建模架构

多域建模技术另一部分是支持对同一模型具备操作权限的多领域用户实时协同建模。多用户实时协同建模整体流程如图4-2所示，首先是用户一创建模型一，由第三章可知，模型一通过 `mxGraph` 进行渲染，因此模型一是基于 `mxGraph` 类的实例化对象，由于协同中心数据以 JSON 格式储存，因此将 `mxGraph` 实例化类中描述模型的关键信息进行提取，转化为 JSON 格式数据，上传至协同中心的协同域中。并且将 JSON 格式数据同步到每位访问该模型的用户，当前所有访问该模

型的用户从协同中心的协同域中获取 JSON 格式数据，为实现访问模型一用户的前端界面图形化显示，通过针对协同域中 JSON 格式数据进行反序列化，通过关键信息构建 `mxGraph` 类的实例化对象，展示在前端画布中，上述流程是前端基于 `mxGraph` 的画布初始化流程。初始化完成后，画布的实时协同操作实现流程是在画布进行初始化时，针对画布及画布中每一个元素添加监听函数，并且针对协同中心元素添加监听器，当用户三对画布进行修改时，会触发画布上的监听器，对协同中心的数据进行修改，同时协同中心发布远程触发，登录当前模型的其余用户可以接收协同中心发布的触发信息，根据触发信息针对当前模型做出同步修改，由此完成了一次画布修改的同步操作。

4.1.2 多域建模技术基本原理

通过多域建模技术总体框架可以将多域建模技术主要分为项目用户管理技术和实时协同建模技术两部分。项目用户管理技术包括协同中心项目管理和客户端项目用户管理。实时协同建模技术基本原理可以划分为客户端模型序列化、协同域 JSON 格式数据反序列化、模型协同适配器三部分，下面分别进行介绍。

首先是多域建模的项目用户管理模块，协同中心包括协同域，每个域中包含 `Collections`、`Models` 及 `Activities`，其中一个 `Collection` 中可以包含多个 `Model`，同一个域中的 `Model` 可以通过 `CollectionId` 和 `ModelId` 来确定其唯一性。在客户端，可以创建不同工作组，每个工作组可以允许加入多个成员，每个工作组允许创建多个项目，支持从工作组成员中挑选用户参与不同项目，从而确定每个用户对于项目的操作权限。每个项目可以包含多个建模语言，每个建模语言可以包含多个模型。因此，在多域建模技术的用户管理模块，将项目名 `Id`、建模语言 `Id` 结合作为协同中心存放该模型的 `Collection` 的 `Id`，将模型 `Id` 作为协同中心的模型 `Id`，从而实现同一项目同一建模语言中的模型可以储存在协同中心的同一个 `Collection` 中，如图4-3所示。

图中包含工作组一和工作组二，工作组一只有项目一，项目一包含建模语言一和建模语言二，建模语言一包含模型一、模型二和模型三共三个模型。支持从客户端创建工作组，加入可操作该工作组的用户，并且将用户划分给具体项目，工作组二结构与工作组一结构相同。通过将项目 `Id` 和建模语言 `Id` 结合作为 `Collection` 的 `Id`，图中的两个项目四个建模语言十二个模型可以在协同中心注册为四个 `collection`，每个 `collection` 对应一个建模语言，每个建模语言中的模型对应协同中心协同域中的模型。协同中心的 `Collections`、`Models` 及 `Activities` 管理界面如图 4-4所示。

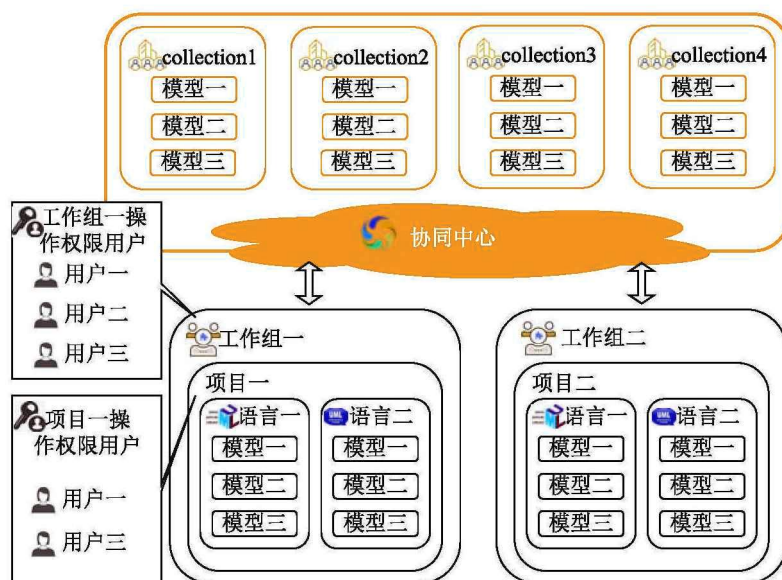


图 4-3 协同中心项目管理

首先是模型的序列化技术，序列化技术支持将基于 `mxGraph` 的 `mxGraph` 类的实例化对象序列化为协同中心可以支持的 JSON 格式数据。上述两种结构如图4-5所示。

图4-5展示了 `mxGraph` 类实例化对象与 JSON 格式对象的具体格式，左侧是基于 `mxGraph` 类实例化的对象，采用树状图展示其具体架构，首先是 `mxGraph` 类实例化后的对象包含 `mxGraphModel` 实例化的对象，模型的关键信息储存在 `mxGraphModel` 对象中的 `cells` 数组和 `root` 中，其中 `root` 是 `mxGraphModel` 类实例化对象的根节点，该节点下包含 `mxCell` 类的实例化对象组成的 `children` 子节点，包含基于 `mxGeometry` 类实例化的对象，该对象包含根节点的几何信息，具体包含由 `mxPoint` 类的实例化对象构成的起止节点，控制点偏移的 `offset` 变量，控制是否采用相对位置计算的 `relative` 变量，包含根节点的 `Id`、根节点的样式 `style`、根节点的值。 `cells` 数组是由 `mxCell` 类实例化对象组成的集合。主要包含一个 `Id` 为 0 的节点和一个 `Id` 为 1 的节点，除此之外，包括基于 `mxCell` 类实例化的对象节点和关系节点。其中对象节点包括 `collapse`、`connectable`、`edges`、`geometry`、`id`、`mxObjectId`、`parent`、`source`、`style`、`mxObjectId`、`target`、`value`、`vertex` 及 `visible`。其中关系节点包括 `collapse`、`connectable`、`geometry`、`id`、`mxObjectId`、`parent`、`source`、`style`、`mxObjectId`、`target`、`value`、`edge` 及 `visible`。

图4-5右侧是存储在协同中心的 JSON 格式数据，该 JSON 格式数据对应一个基于 `mxGraph` 类实例化的对象，主要结构包括 `cells` 和 `root` 两部分，`root` 代表根节点，`root` 值是根节点的 `Id`。 `cells` 是由模型中的对象及节点的关键信息构成的

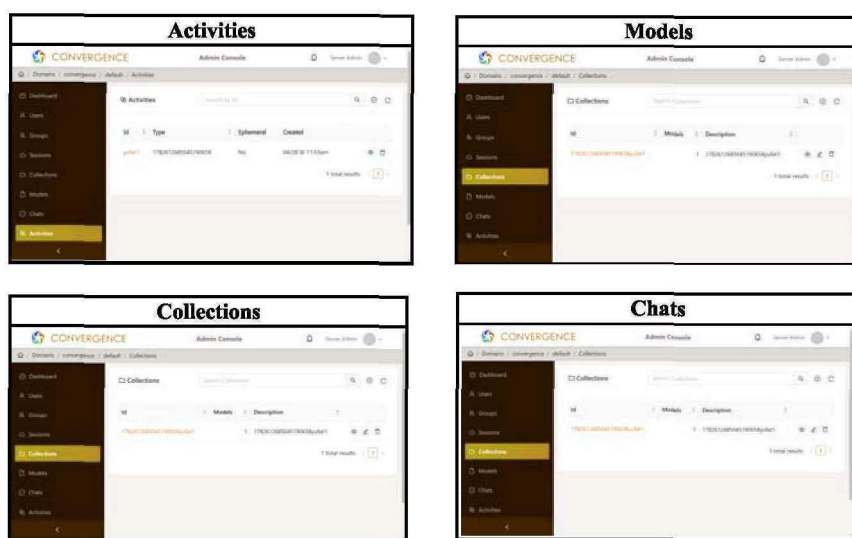


图 4-4 协同中心各模块

数组，首先是一个 *Id* 为 0 的根节点和一个 *Id* 为 1 的节点，上述两个节点不代表模型中真实的对象或者关系。其余是对象和关系的关键信息，其中对象关键信息主要包括 *collapsed*、*connectable*、*geometry*、*id*、*parent* 及 *style*，其中 *style* 由一个数组表示，主要包括 *aspect*、*dashed*、*fillColor*、*fontFamily*、*fontSize*、*gradientColor*、*gradientDirection*、*html*、*metaObjectId*、*metaObjectLocallabel*、*shape*、*strokeColor*、*strokeWidth* 及 *whiteSpace* 等样式信息。关系关键信息主要包括 *collapsed*、*connectable*、*edge*、*geometry*、*id*、*parent*、*source*、*style*、*target*、*value* 及 *visible* 等信息。

在画布初始化时需要将基于 *mxGraph* 类实例化的模型信息进行序列化处理转化为能够存储在协同中心的数据，主要流程如下：首先是创建一个对象，两个键分别是 *root* 和 *cells*，*root* 键的值为根节点的 *Id*，*cells* 的值是一个空数组，遍历模型中所有 *mxCell* 类实例化的对象，针对每一个 *mxCell* 类的实例化对象创建一个对象，首先放入当前 *mxCell* 类实例化对象的 *Id* 的键值对，接下来判断基于 *mxCell* 类实例化对象的 *style* 变量是否定义，如果有定义，则针对 *style* 进行序列化处理，并且将序列化后的 *style* 关键信息以键值对的方式存储在对象中，并且其值是由 *style* 各个变量的键值对组成的数组。判断对象的 *geometry* 变量是否定义，如果有定义，则针对 *geometry* 进行序列化处理，将处理后的数据以键值对的形式放入对象中，判断对象的 *connectable*、*visible*、*collapsed* 是否有定义，如果有，则将键值对放入对象中。然后判断对象是否为边节点，判断后将 *vertex* 的值放入对象中。判断对象父节点、源节点和目标节点是否存在，若存在则将父节点、源节点

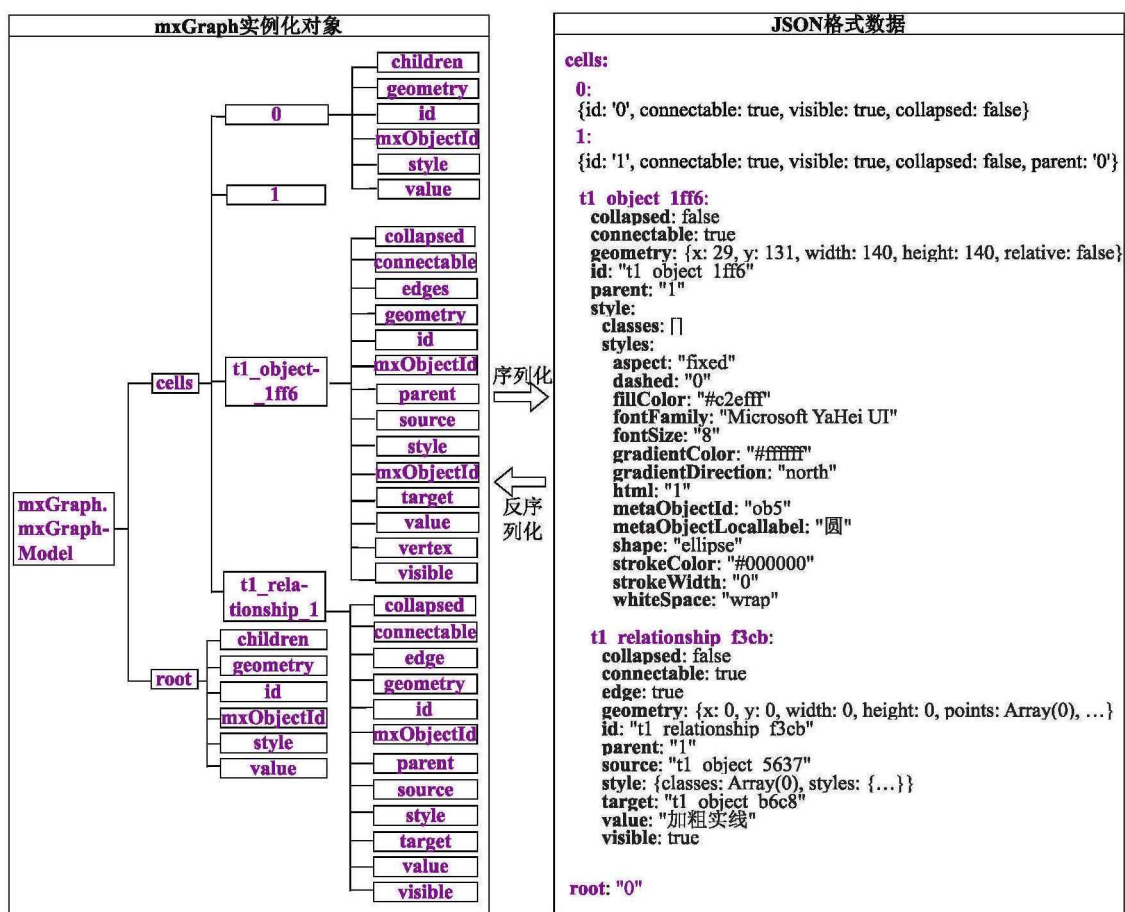


图 4-5 mxGraph 类实例化对象与 JSON 格式转换

及目标节点的值放入对象中。判断当前对象是否是最后一个对象，如果不是，则遍历模型中下一个基于 `mxCell` 类的实例化对象，如果是最后一个对象，则返回对象 JSON 格式数据。

将基于 `mxGraph` 类实例化对象进行序列化后，上传至协同中心进行共享，共享后将共享数据通过反序列化将 JSON 格式数据转化为基于 `mxGraph` 类的实例化对象，在前端进行图形化展示，反序列化流程如下：反序列化是将协同域中的 JSON 格式数据转换为 `mxGraph` 可解析的模型数据。首先，取 `cells` 数组中的 `root` 进行删除，将 JSON 数据中的 `root` 进行反序列化，创建一个基于 `mxGraphModel` 类实例化模型对象，并且将上述 `root` 节点设置为模型对象的根节点，遍历 `cells` 数组中所有元素，将每个元素反序列化为基于 `mxCell` 类的实例化对象，并且加入到模型对象中。对数组中的每个元素判断是否为最后一个元素，若不是，则继续遍历，若是，则再次遍历 `cells` 中的每一个元素，进行关系的反序列化，判断是否为最后一个关系，如果不是则继续遍历，若是就返回最终模型对象，结束模型的反序列化流程。JSON 对象中针对对象进行反序列化流程如下所示：首先，将 JSON 数

据中每个元素的 *Id* 和值的 *cellData* 数据作为参数, 先将 *value* 进行反序列化, 返回 *value* 的字符串。其次将 *style* 进行反序列化, 返回具备样式关键信息的字符串, 将 *geometry* 进行反序列化, 解析 *points*、*sourcePoint*、*targetPoint*、*relative* 等关键信息, 创建并且返回基于 *mxGeometry* 类的实例化对象。创建 *mxCell* 类的实例化对象, 设置其实例化对象的 *collapsed*、*connectable*、*visible*、*vertex*、*edge*、*style* 等关键信息, 最后返回 *mxCell* 类的实例化对象。由于关系两端需要连接对象, 因此在进行元素反序列化过程中先进行对象的反序列化, 再进行关系的反序列化。将每个 *mxCell* 对象、JSON 格式数据的关键信息及模型对象作为参数输入, 解析关键信息, 进行父节点的插入, 再设置关系源节点和目标节点。

完成画布初始化后, 需要进行画布适配器的匹配, 画布适配器用于监听当前模型的更改, 并且将更改信息发布至协同中心, 协同中心按更改信息进行修改, 同时触发登录当前模型的用户前端画布的远程触发的监听器, 对应监听器被触发后进行画布相应的修改, 从而实现实时协同。

首先, 将前端基于 *mxGraph* 类实例化的画布、协同中心数据作为输入并且与图适配器中构造函数变量绑定, 创建存放监听器的数组及创建存放 *mxCell* 类实例化对象适配器的字典。在前端画布中添加监听事件及监听事件对应的响应事件。监听事件包括监听前端页面元素添加、元素移除及元素改变。插入元素的监听及响应事件具体流程如下, 首先通过触发事件获取相关 *cell* 数组, 遍历数组所有元素, 将数组中元素通过反序列化转化为 JSON 格式数据, 将前端的元素与协同中心对应数据进行绑定, 并且刷新前端模型页面。改变元素的监听触发事件流程如下, 首先获取对应元素的变化, 判断该变化是否属于根节点变化, 如果属于根节点变化, 则进行后续处理。如果不属于根节点变化, 判断是否为子节点变化, 如果是子节点变化, 则先对前端模型进行相关处理, 再与协同中心数据绑定, 最后刷新前端页面。如果不是子节点的变化, 则判断变更中放入对象及对象的 *Id* 是否不为 *null*, 如果是则获取并且处理变更对象的适配器, 并且刷新前端模型页面。处理完本地画布的监听事件及触发响应事件后, 在协同中心添加监听事件及对应的响应事件。协同中心的监听事件主要包括元素变更事件及元素添加事件, 首先检测到协同中心元素改变, 将协同中心对应元素数据进行反序列化, 将反序列化结果添加至前端画布模型, 并且将反序列化前的元素数据和反序列化后的对象进行绑定, 将绑定结果放入 *mxCell* 适配器数组中, 并且刷新前端页面。最后, 遍历前端画布中所有对象, 将前端画布对象与 JSON 格式数据进行绑定, 完成图适配器的构建流程。

首先接收前端 *mxCell* 实例化对象、协同中心 JSON 格式数据、图、事件触发器等, 将上述参数分别赋值给构造函数, 并且初始化 *mxCell*、*mxGeometry* 和 *style*,

其中针对 `mxCell` 的初始化包括以下流程，首先针对协同中心的数据添加监听函数，并且在其回调函数中分别针对前端的 `parent`、`target`、`source`、`style`、`value`、`visible`、`collapsed` 等变量进行设置，对由于前端操作导致上述变量发生改变时触发的回调函数进行设置。其中针对对象几何变量的初始化主要包括给对应协同中心数据进行协同，并且实时刷新。针对对象样式变量的初始化主要包括添加设置、移除和插入的监听函数及回调函数。完成监听函数的添加后，若前端数据发生变化，通过触发事件获取模型更改信息，通过判断更改信息类型，设置协同中心对应数据，并且触发相应的回调函数，刷新前端页面。

4.1.3 多域建模技术方案实现

多域建模技术主要包括多域管理模块和实时协同模块。其中多域管理模块分为客户端多域管理及协同中心多域管理，其中客户端多域管理主要基于 `React` 框架构建前端页面，并且基于 `MySQL` 实现数据库管理，客户端多域管理包括工作组管理和项目管理两部分，工作组管理主要如图4-6所示。



图 4-6 工作组管理界面

上图展示了工作组列表，工作组列表展示当前用户允许查看的工作组、工作组相关描述、工作组成员及工作组项目等，一个用户可以参与多个工作组工作，一个工作组中可以包含一个或多个项目，每个工作组可以支持不同的用户进行查看。工作组管理界面支持新建工作组、修改工作组、删除工作组、项目管理及成员管理。支持通过工作组管理界面进入项目管理界面。项目管理主要如图4-7所示。

以上是客户端多域管理内容，数据协同是基于协同中心的协同域实现的，协同中心域提供 `Sessions` 和 `Services`，其中 `Sessions` 可以通过 `domain.session()` 进行查看，表明用户连接到协同中心的进程。`Services` 包括身份服务、在线查看服务、模型服务、活动服务及对话服务等。身份服务是通过 `domain.identity()`



图 4-7 项目管理界面

获取，提供身份验证等服务。通过 *domain.presence()* 提供在线用户查看等服务。通过 *domain.models()* 提供存储、查看、编辑共享数据模型等服务。通过 *domain.activities()* 提供支持查看各用户操作的服务。通过 *domain.chat()* 支持用户通过发送信息进行相关对话等服务，如图4-8所示。

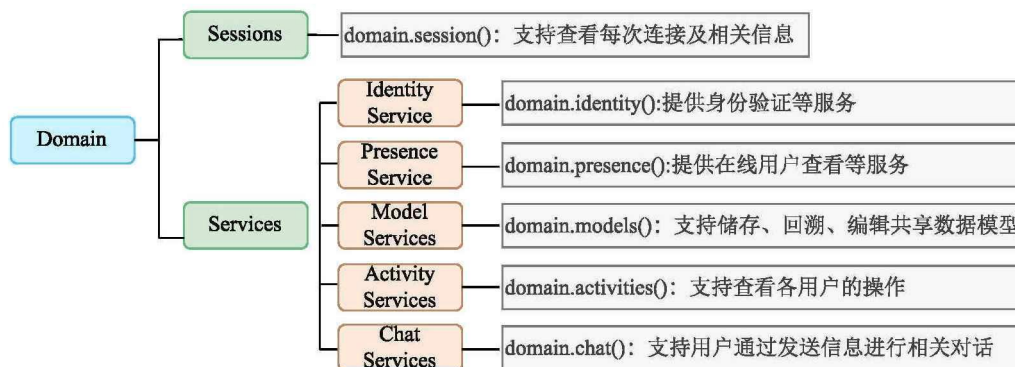


图 4-8 协同引擎服务

一个工作组可以包括多个项目，每个项目的权限可以选择针对当前项目所在工作组中的成员进行开放，项目管理包括创建项目、删除项目及成员管理，其中创建项目可以通过新建项目或上传已有项目进行创建，以上是本地端多域管理实现。基于协同中心的 Collections、Models 及 Activities 服务可实现多人协同，一个 Collection 中具备多个模型，同一个 Collection 中的模型 Id 需要保持唯一，不同 Collection 中 Model 的 Id 可以重复。而多域建模中，同一项目同一建模语言下创建的模型 Id 不能重复。因此，本文将模型在协同中心注册时，首先在协同中心创建一个 Collection，该 Collection 的 Id 由项目 Id 和建模语言名称组合而成，协同中心 Model 的 Id 则与客户端模型 Id 相同。具体创建流程如下，首先通过 *Convergence.connectAnonymously()* 创建或连接到对应协同中心域，并且将协同

中心域 *domain* 作为返回值返回, 通过 *domain.models()* 获取域中模型的相关服务, 通过 *domain.models().openAutoCreate()* 的方式将数据上传至协同中心。协同中心的域支持协同数据的储存及处理, 并且提供进程操作、活动操作等相关服务。

通过上述操作, 以匿名且自动创建模型的方式在协同中心创建 Collection 及 Model, 并且以 JSON 格式储存模型数据。支持不同用户或相同用户不同设备从客户端登录同一组织同一项目同一建模语言同一模型后, 能够在协同中心协同域中连接到同一个 Model, 进行模型协同构建。

4.2 协同冲突问题处理

4.2.1 基于分布式 OT 技术的协同冲突处理

OT 算法通过针对操作进行拆分、转换等更改操作来实现协同操作的一致性。OT 算法包括以下关键的四个步骤, 第一步是针对原子操作进行定义, 第二步是实现版本确认的机制, 第三步是通过操作转换实现协同操作的一致性, 最后一步是实现客户端状态的变更。首先 OT 算法中包括保持操作、插入操作及删除操作三种针对内容的基本操作, 并且保证每次处理都可以基于上述三种操作进行。其次 OT 算法的版本确定主要用于在新客户端建立版本且客户端进行操作更改时, 需要向服务器发送一个同步操作, 该同步操作内容包括此次操作进行更改的具体内容及本次操作的更改是基于第几版本进行的更新。并且根据服务器的版本判断是否需要进行操作转换。第三步中的操作转换是整个 OT 算法的关键步骤, 该步骤的关键公式如下所示。

$$[A', B'] = transform(A, B) \quad (4-1)$$

$$apply(apply(S, A), B') = apply(apply(S, B), A') \quad (4-2)$$

上述公式表述如下, 两个用户分别针对同一版本模型进行操作, 分别对模型进行 A 操作和 B 操作, 由于两个操作导致结果不同, 因此无法实现协同的一致性表达。因此, 通过操作转换, 将操作 A 和操作 B 经过转化为成为操作 A' 和操作 B', 具体在协同过程中实现如下, 通过用户 A 的操作 A 及操作转换后的操作 B', 与通过用户 B 的操作 B 及操作转换后的操作 A', 最终得到相同的模型, 如图4-9所示。

图4-9中每一个点可以理解为一个状态, 每一条边可以理解为针对边所在起点状态的操作, 因此针对于用户 A 和用户 B 的协同操作的一致性处理, 可以理解为用户 A 的操作包含操作 A 和操作 B', 服务器真实的操作是操作 B 和操作 A', 两

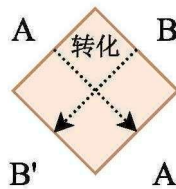
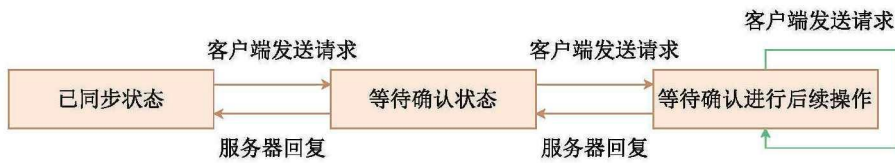


图 4-9 OT 算法

类操作最终实现相同效果，以此实现协同的一致性。客户端的状态转移主要包括已经同步状态、等待服务器端确定及等待服务器确认并有后续发送操作三种状态，客户端状态转移主要操作如图4-10所示。



本文的柔性建模技术采用 mxGraph 进行渲染，mxGraph 支持原子操作，多域建模技术也以原子操作为基础，减少多域建模技术的冲突。

基于以上冲突解决算法，最终冲突操作效果如图4-11所示。

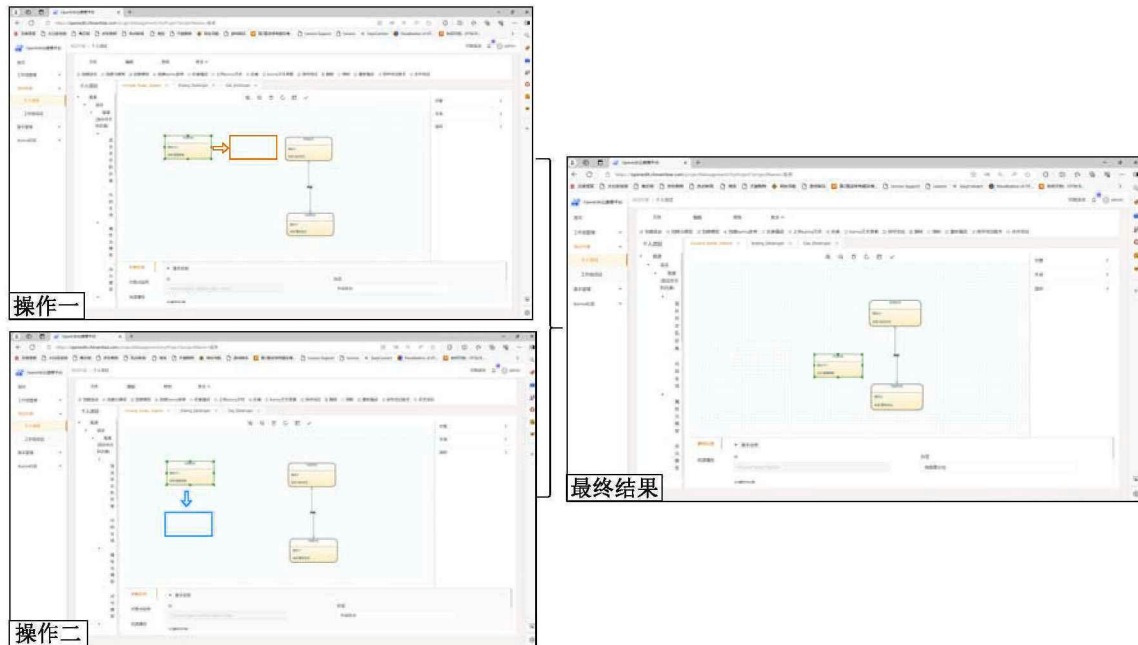


图 4-11 冲突操作解决效果

图4-11中展示了针对同一模型中同一对象，不同用户分别同时进行操作一及操作二的协同效果。用户一与用户二同时选中对象，然后进行不同方向的拖拽，最

终同时释放对象。根据本文协同操作算法,呈现最终操作效果如图4-11中最终结果所示。最终结果是由操作一及操作二基于向量叠加规则通过操作转换所得,最终实现协同的一致性。

4.2.2 基于标记共享的协同冲突处理

多人协同实时建模时,除了采用协同冲突算法进行协同冲突避免外,还可以通过共享用户光标位置及用户当前操作对象,用于展示登录当前模型的用户的操作,从而减少短时间内多个用户针对同一模型进行操作,减少冲突。通过活动颜色管理、光标管理及选择管理来实现上述操作,具体实现操作是将每个用户登录一个模型看作一次活动,协同中心为每次活动分配一个特定颜色,该颜色用于用户光标及用户标记显示。通过不同颜色区分当前编辑该模型的不同用户活动。

将用户登录一次模型作为一次活动,通过协同引擎提供的 API 将活动作为参数进行输入,构建 *ColorAssigner* 类,通过 *activity* 的 *events()* 函数判断用户是登录当前会话还是退出当前会话。如果是登录当前会话,则协同中心增加该会话 *Id*,并且通过会话 *Id* 生成用于当前会话标记的颜色。如果退出会话,则释放当前会话颜色。

光标共享用于展示登录同一模型各个用户的光标位置,并且光标颜色由上文中的颜色管理器进行分配,每一次会话分配一个颜色。通过构建一个构造器类来实现光标管理,该类的构造器包括 *mGraph*、*activity*、*colorManager*、*remotePointers*、*root* 及 *active* 变量,包括画布监听函数和活动监听函数,其中 *listenToGraph()* 是针对于画布添加监听函数,包括针对 *root* 添加事件的监听函数和鼠标监听函数,针对 *root* 添加事件监听函数主要包括鼠标移入和鼠标移出事件,其中鼠标移入事件的回调函数包括移出鼠标状态并且 *active* 设置为 *false*,鼠标移出事件回调函数为将 *active* 设置为 *true*。鼠标的监听函数主要包括鼠标按下、鼠标移动及鼠标放开,其中鼠标按下和鼠标放开的回调函数没有后续操作。鼠标移动的回调函数的后续操作如下,首先判断 *active* 是否为 *true*,如果不是,则无后续操作。如果是,则通过触发事件获取鼠标在画布上的横纵坐标,并且获取当前画布的比例变量,通过以上变量准确计算出当前鼠标所在位置横纵坐标,并且在 *activity* 中加入鼠标状态。*listenToActivity()* 函数中主要是针对 *session* 添加事件的监听函数,主要包括添加加入会话、离开会话、设置状态和移除状态四种触发事件及其触发的回调函数。其中加入会话的回调函数是遍历当前每个用户,添加远程光标。离开会话的回调函数是通过会话 *Id* 获取对应的远程光标,并且通过 *sessionId* 删除远程光标。设置状态的触发事件的回调函数是通过触发事件获取会

话 *Id*，并且判断是否为本地的变量和键，通过以上变量判断是否为鼠标事件及本地事件，如果是，则通过会话 *Id* 获取远程光标并且显示，如果不是，则无后续操作。移除状态的触发事件首先是获取会话 *Id*、本地变量和键，判断是否为鼠标事件并且属于本地，如果是则通过 *sessionId* 获取远程光标并且隐藏，如果不是，则无后续操作。

以上是针对光标进行共享的流程，除此，还可通过共享选择框来避免冲突操作，用户选中对象或者关系的时候，对象或关系边框会出现选中框，当前用户前端界面可以看到此时操作当前模型的其他用户选择的对象或关系，不同用户的选择框通过不同颜色区分，并且同一用户的不同会话也通过不同颜色区分。

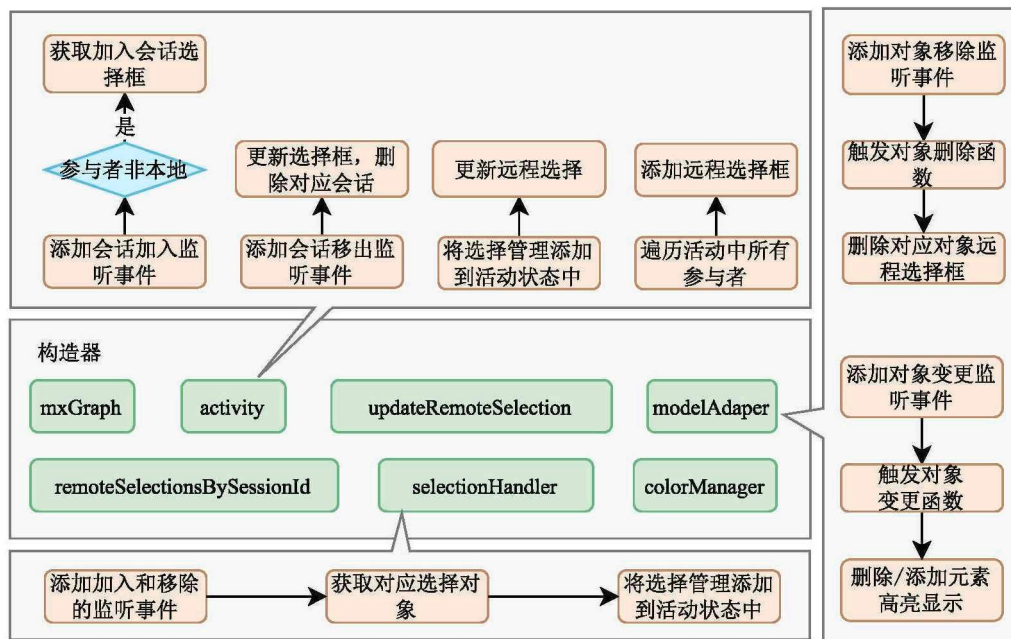


图 4-12 选择框管理构建流程

选择框共享流程通过实例化选择框类完成，该类的构造器需要输入 *mxGraph*、*activity*、*colorManager* 及 *modelAdapter* 参数，并且针对 *activity* 添加会话加入、会话移出及设置状态的监听事件，并且遍历活动中所有的参与用户，通过用户获取每个会话的选择框，通过 *activity* 设置每个登陆界面选择框的状态。将 *mxGraph* 中的选择器作为 *selectionHandler*，基于 *selectionHandler* 设置选择框添加的监听事件及选择框移出的监听事件，并且添加触发事件的回调函数，回调函数包括通过 *mxGraph* 中 *getSelection()* 函数获取画布模型中被选中的元素，通过 *activity* 对每个被选中的对象设置选择框。针对 *modelAdapter* 添加对象移出及对象变更的监听事件，对画布元素进行删除时，触发画布中删除元素的回调函数，并且删除对应对象的远程选择框。画布中被选中元素发生变更时，触发画布中元素被选中时的

回调函数，删除变更前选中对象的选择框高亮显示，针对新选中的对象添加选择框高亮显示。具体标记共享效果如图4-13所示。

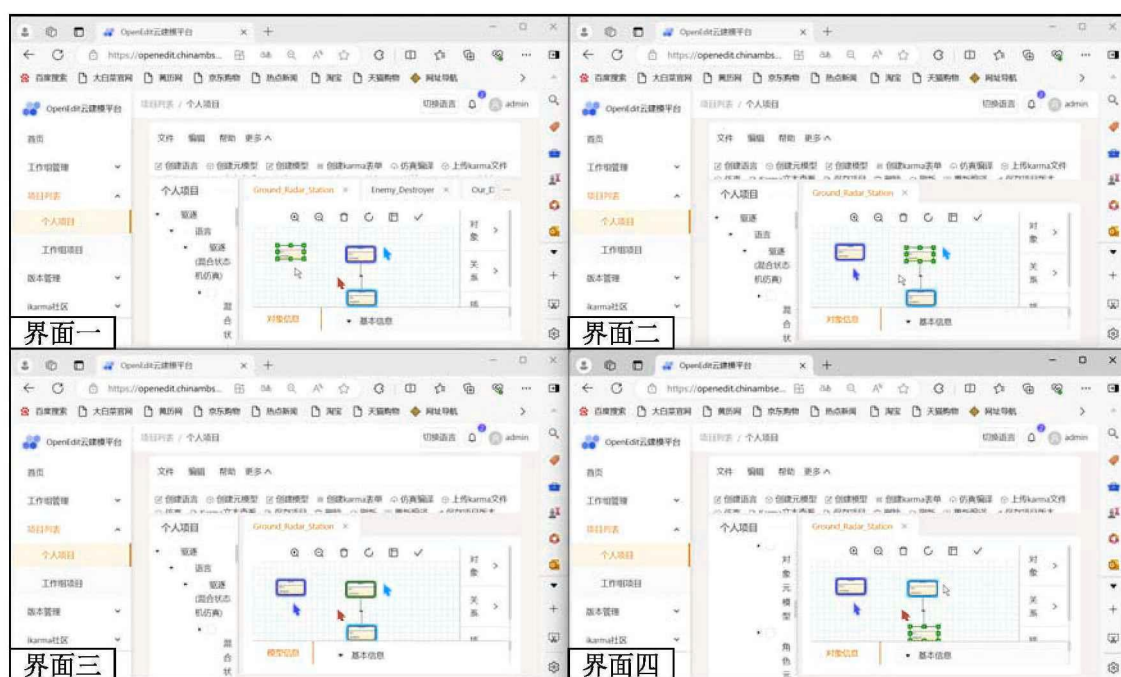


图 4-13 标记共享展示

如图4-13所示，当四个客户端登录协同中心的同一项目同一模型。鼠标在界面一进行移动时，其余界面会实时同步并且高亮展示界面一中鼠标状态。并且每个界面鼠标在其余界面显示时高亮颜色相同。其余界面鼠标共享状态同理。在界面一中选中元素，其余界面中对应元素会进行高亮显示，并且几个界面的高亮显示颜色相同。其余界面选中元素时同理。

通过上述标记共享可以提醒登录该模型的各个用户的状态及操作意图，从而提醒用户避免冲突。

4.3 基于云架构的多域柔性建模实现

多域柔性建模平台部署在云平台具备灵活、可伸缩、高可用、低成本、安全、管理方便等优点。本节重点介绍将柔性建模平台移植到云平台，主要包括两阶段，阶段一是将多域柔性建模平台进行镜像化处理，阶段二是将镜像部署在云服务器上。本节分别针对这两阶段进行介绍。

4.3.1 建模平台 Docker 镜像化处理

目前常用镜像化工具包括 Docker、Podman、Kubernetes 及 LXC 等，其中 Docker 作为主流的镜像化工具，具备轻量级、高度可移植、快速部署、灵活、资源隔离及

生态系统丰富等优点。轻量级主要体现在 Docker 容器与传统虚拟化技术相比，通过支持共享相同的操作系统内核从而更轻量 and 高效。Docker 通过使用 Linux 内核支持容器与容器之间相互独立，实现资源隔离，通过避免资源冲突提高其安全性。并且 Docker 具备丰富的系统和社区支持，具备丰富的服务和第三方工具，利于开发人员进行平台快速搭建及部署。

镜像采用 bootfs 作为最底层，rootfs 作为其上层的分层构建机制，在 Docker 镜像中，bootfs 的上层为基础镜像，基础镜像上层为父镜像，最上层是容器。在上述各层中，除了最上层为可读写层，其余层都是只读层，如下图4-14所示。

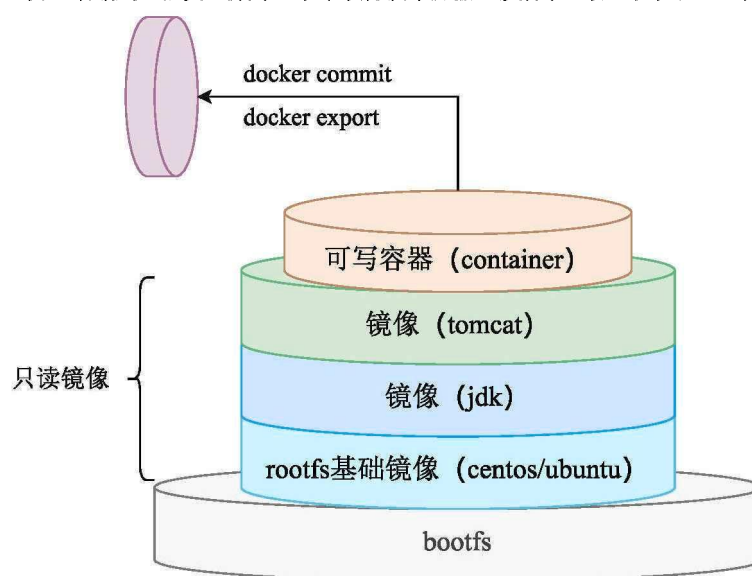


图 4-14 镜像架构

Bootfs 是内核并且支持作为内核引导器引导加载内核，rootfs 是只读层，由应用镜像和多个提供基础操作环境的基础镜像叠加构成。在 rootfs 上层会添加一个可读可写层作为运行的容器。

在容器启动时，Docker 守护进程 Docker daemon 首先尝试基于本地服务器获取镜像，若本地服务器不存在所需镜像，从 Docker Registry 中对所需镜像进行下载，同时保存在本地。

本文采用通过 Dockerfile 制作镜像的方法进行镜像制作。通过定制每一层需要进行添加的文件和配置信息进行镜像的定制，把针对每一层的安装、构建、修改及操作命令写入 Dockerfile 脚本，Dockerfile 中包含多条命令，每条指令可以与 Linux 中的命令相对应，支持 Docker 程序通过 Dockerfile 中的指令自动生成镜像。Dockerfile 文件中包含用户或维护者信息、基础镜像的信息、容器启动指令信息和镜像操作指令信息。柔性建模平台的 Docker 镜像化基于 nginx 基础镜像进行构建，nginx 镜像用于负责前端的服务、后端服务的转发和均衡负载等。将柔性建模平台

分为柔性建模前端、柔性建模后端、多域数据库及多域控制四个模块。基于 nginx 基础镜像将多域柔性建模平台进行镜像化处理。并且通过 Docker Compose 支持多个 Docker 容器的定义、配置和应用，支持统一管理多容器的应用部署和运维，并且保持应用平台的开发、测试和生产环境的一致性。多域柔性建模平台的 Docker 容器集群通过 docker-compose.yaml 文件进行快速编排，docker-compose.yaml 文件主要如图4-15所示。

version	3
services	mariadb: image: docker-registry.chinambse.com/zkhoneycomb/mariadb:10.4.30 container_name: openedit-mariadb environment: MYSQL_ROOT_PASSWORD: 123456 MYSQL_DATABASE: convergence volumes: - "/data/openedit/mysql/data:/var/lib/mysql" ports: - "53306:3306" restart: always networks: - mynet1
	openedit-graph: image: openedit-graph:latest container_name: openedit-frontend ports: - "58020:80" restart: always depends_on: - openedit-server networks: - mynet1
	openedit-server: image: docker-registry.chinambse.com/zkhoneycomb/openedit-server:latest container_name: openedit-backend volumes: - "/data/openedit/karmaProject:/data/karmaProject" ports: - "58030:8030" restart: always depends_on: - mariadb networks: - mynet1
	convergence-console: image: docker-registry.chinambse.com/zkhoneycomb/convergence-omnibus:latest container_name: convergence-console user: root ports: - "8000:8000" - "8041:8080" environment: HTTP_PORT: 8000 volumes: - "/data/convergence/database:/orientdb/databases" networks: - mynet1
networks	mynet1: driver: bridge

图 4-15 docker-compose 架构

主要包括 version、services 及 networks 设置，其中 services 主要包括 convergence-console、openedit-server、openedit-graph 及 mariadb 四个服务，每个服务中针对其 image、container-name、volumes、ports、restart 及 networks 进行配置。

4.3.2 针对 Docker 镜像进行云服务器部署

云平台部署具备方便管理、成本低、具备自动化和可编程性、规模大和可扩展性高等优点。将平台打包为基于 Docker 的镜像服务使其具备可扩展性、可移植性、轻量级、可管理性等优点。云平台主要分为储存型云平台、计算型云平台及综合云计算平台。其中储存型云平台是以数据储存为主，计算型云平台是以处理数据及计算为主的云平台，综合云计算平台是兼顾数据处理和数据存储的云平台。本研究使用的云平台是综合云计算平台。需要储存用户项目模型信息并且支持多域建模技术中的数据计算。将多域柔性建模平台基于 Docker 进行镜像化处理后，将其镜像部署在云平台的主要流程如下，首先需要申请云服务器，在服务器上安装 Ubuntu20.04 的操作系统，并且安装 Docker 和 Docker-Compose，将镜像文件上传至云平台 Ubuntu 系统平台中，通过 `docker load -i xxx.tar` 导入上传的镜像，在 `docker-compose.yml` 文件所在文件夹中，通过 `docker - compose up` 命令在云服务器启动多域柔性建模平台。

4.4 基于云架构协同的应急救援工程多域建模及分析

4.4.1 多域建模流程

多域柔性建模技术在柔性建模技术基础上支持多领域协同建模，支持组织项目管理，通过实时在线协同建模支持同一模型的跨领域协同构建，本节基于云平台的多域柔性建模技术，参照英国应急救援体系工程针对海上遇险船只的搜索和救援作战进行模型构建。

英国搜救组织目前划分为三个领域两个层级，三个领域分别指陆上搜救、海上和陆上航空器搜救、海上搜救。两个层级包括协调组织机构和英国搜救战略委员会。英国搜救战略委员会主要包括国防部、运输部、苏格兰警察委员会、苏格兰山地救助小组、卫生急救委员会、英国山地救助小组、湿地搜寻和救助协调委员会。英国搜救战略委员会主要负责制作搜救的标准、战略、义务和政策。每年战略委员会的会议召开次数不能低于两次，并且由英国搜救组织确定其具体体系框架、明确搜救行动标准。英国搜救战略委员会下设置的办事机构为英国搜救工作小组，工作小组成员包括英国岩洞救援组织、海事和海岸警卫署、威尔士山地救助小组、内务部、运输部及国防部等。

因此在结合 UPDM 建模规范针对应急救援工程中的海上遇险船只的搜索和救援工程进行模型构建时，需要进行建模组织管理并且支持多领域专家在线实时协同建模。本案例是基于 UPDM 规范进行模型构建，因此在基于云平台进行多域柔性建模时，根据 UPDM 视点进行组织的划分，工作组划分完毕后，下面从全景视

点、能力视点、作战视点、服务视点、系统视点五个视角基于多域柔性建模技术进行模型构建。首先是体系一组全景视点的构建，全景视点与各视点关系如图4-16所示。

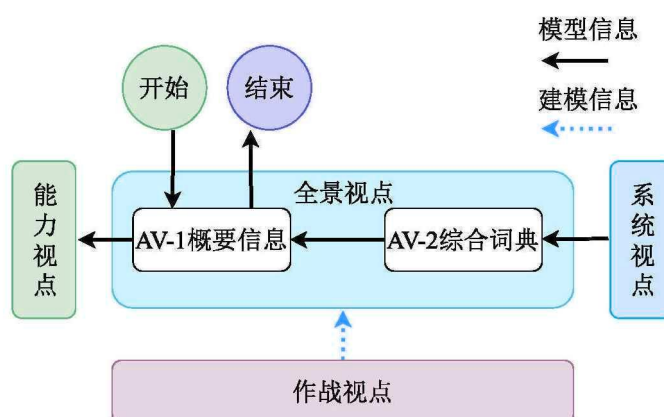


图 4-16 全景视点模型

如图所示，全景视点的构建需要来自系统视点的模型信息，并且全景视点为能力视点提供模型信息。作战视点能为全景视点提供建模信息。因此在全景视点模型进行构建时，需要系统视点、能力视点及作战视点领域相关专业人员协同建模，其中 AV-1 概要信息模型如图4-17所示。

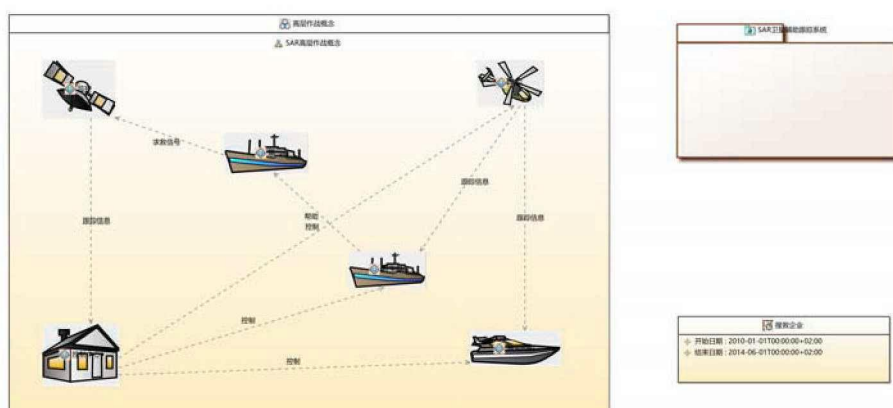


图 4-17 AV-1 概要模型

如图4-17所示，基于 AV-1 概要信息，通过针对高层作战概念对象进行分解，分解出了 OV-1 模型用于进一步表达高层作战概念，并且提供了搜索和救援行动的开始结束日期，以及 SAR 卫星辅助跟踪系统的相关信息。

能力视点概述如图4-18所示，能力视点描述在整个搜救过程中提供相关能力的信息，建构能力视点需要联合全景视点、作战视点、项目视点和服务视点提供模型信息的交互，通过联合建模最终增强体系的价值。

根据能力视点模型概述中的模型信息流向及建模流程顺序进行能力视点的协

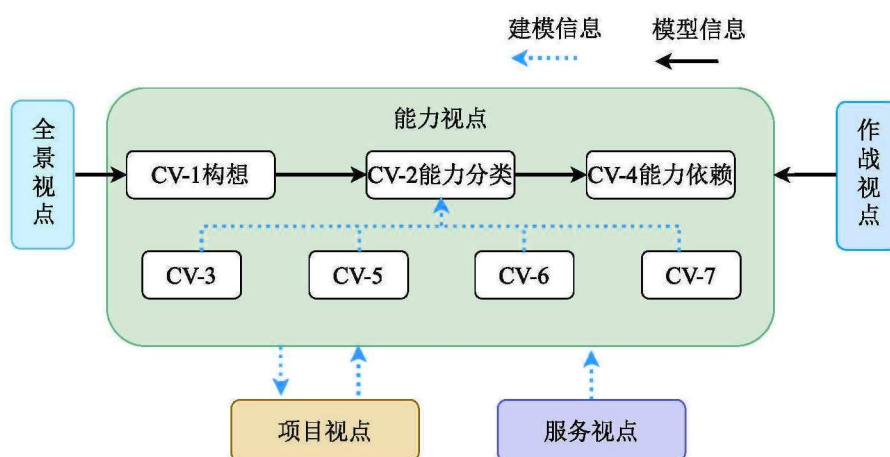


图 4-18 能力视点模型

同构建。模型如图4-19所示。

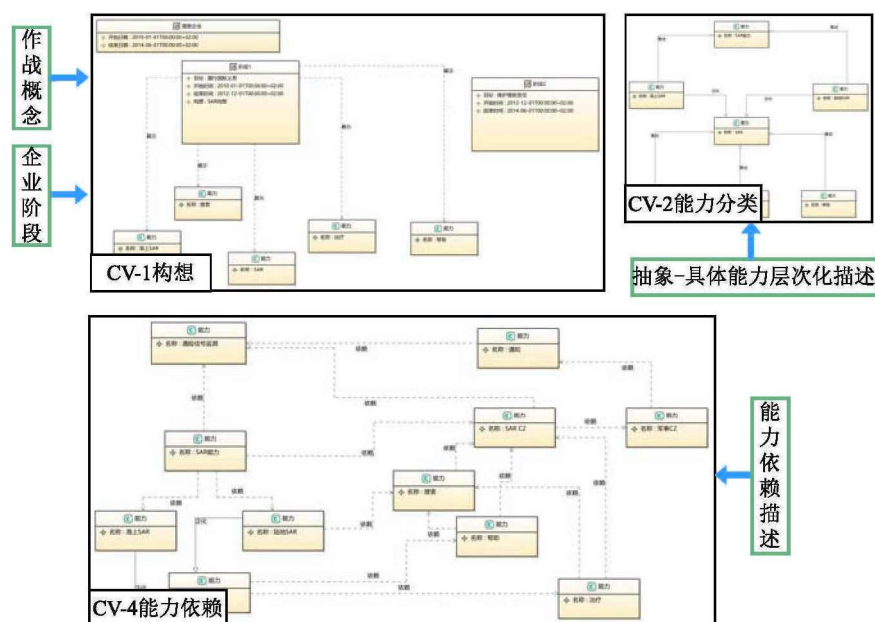


图 4-19 能力视点模型实例化

作战视点模型主要包括作战相关的要素展示、作战资源流交换及针对作战涉及的动作及任务进行明确和指导。作战模型需要高层次模型的相关信息为指导，来解决作战活动过程中物资需求及约束。如图 4-20所示。

OV-1 针对作战体系中内部系统与外部系统间的相互作用及影响进行描述，在当前案例中，OV-1 高层作战概念描绘了海上遇险船只发出呼救信号，监控单元捕捉呼救信号，监控单元发出监控信号，指挥与控制中心接收信号并且负责指挥与协调。OV-5b 负责描述活动之间的各类资源交换，OV-2 主要针对物资、人员、及信息等资源流的模型。本案例的作战视点关键模型如图4-21所示。

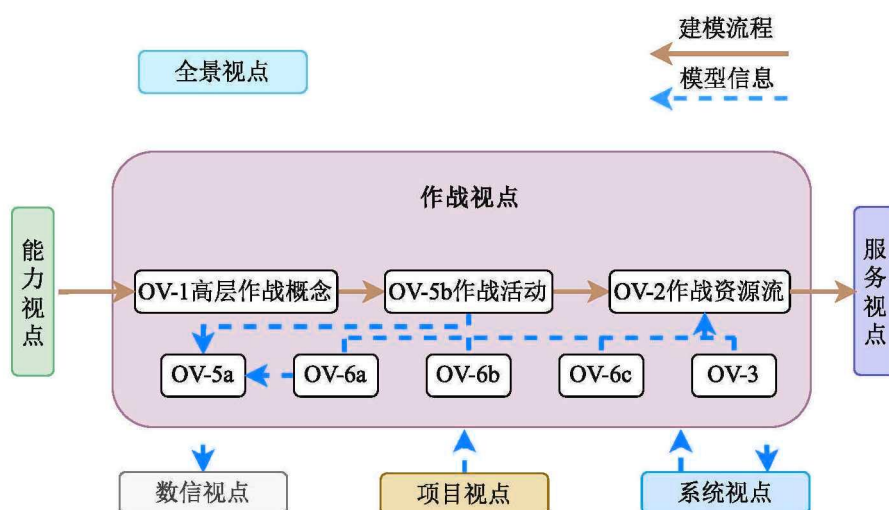


图 4-20 作战视点模型

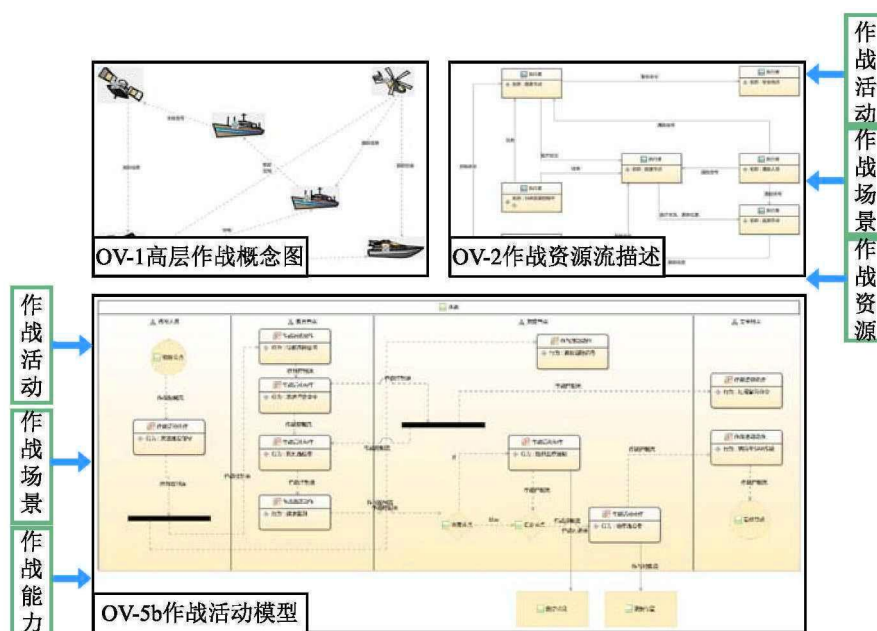


图 4-21 作战视点模型实例化

服务视点用于针对作战活动及业务活动提供服务，为系统工程减少风险和成本、提高可靠度。图4-22是服务视点概述。

如图4-23所示，服务视点主要模型为 SvcV-1 服务环境描述模型及 SvcV-2 服务资源流描述模型。图4-23中模型需要系统视点领域专家及能力视点领域专家提供相关信息协同完成，分别实现作战逻辑架构及作战服务的描述。

系统视点支持针对系统与系统的关系、业务功能及作战功能进行描述，支持将能力需求和系统资源与作战进行关联，因此建模流程需要结合服务视点和全景

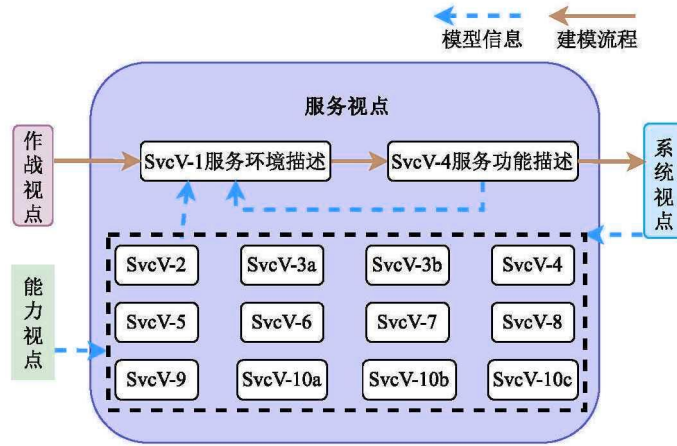


图 4-22 服务视点模型

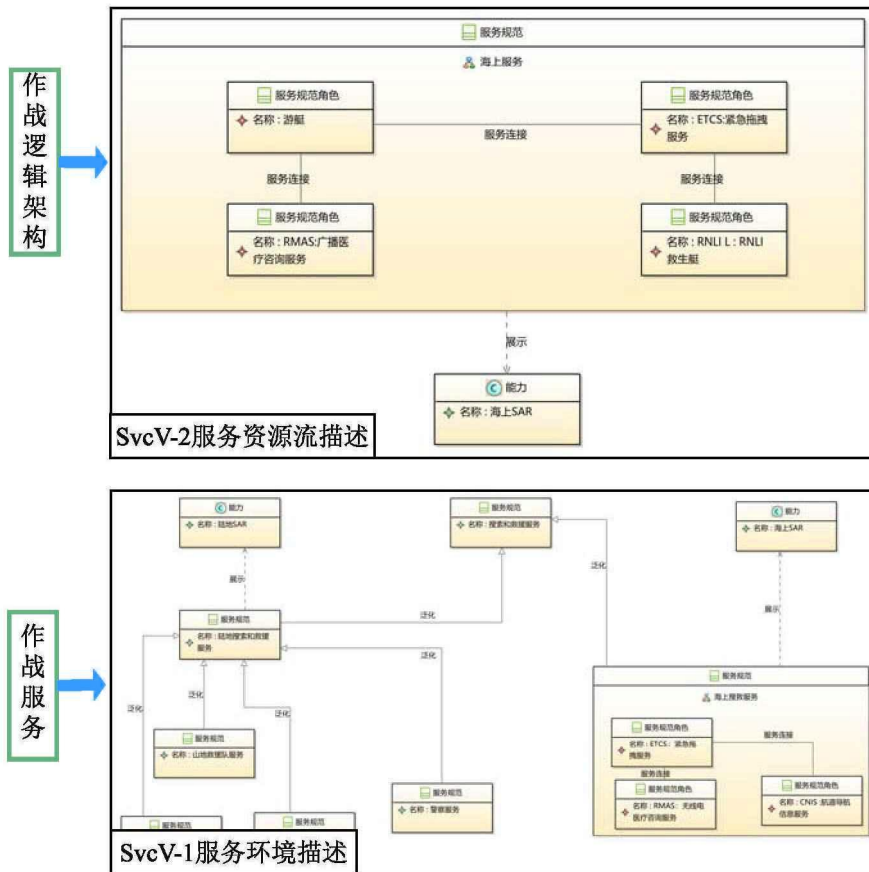


图 4-23 服务视点模型实例化

视点，并且支持项目视点、数信视点、作战视点及标准视点的模型信息交互，因此需要上述视点涉及的领域专业人员协同构建。系统视点概要如图4-24所示。

系统接口描述包括作战资源及作战逻辑架构，系统资源描述包括作战场景及作战系统，系统功能描述包括系统资源，如图4-25所示。

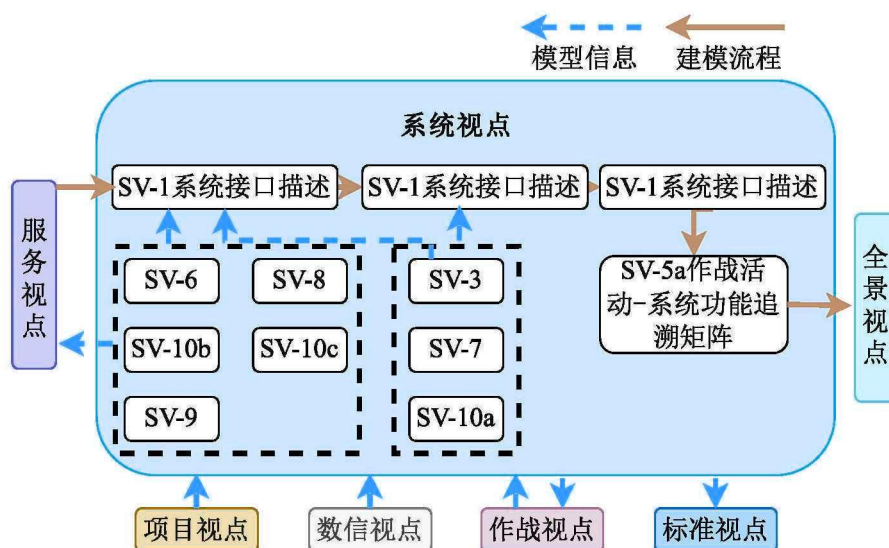


图 4-24 系统视点模型

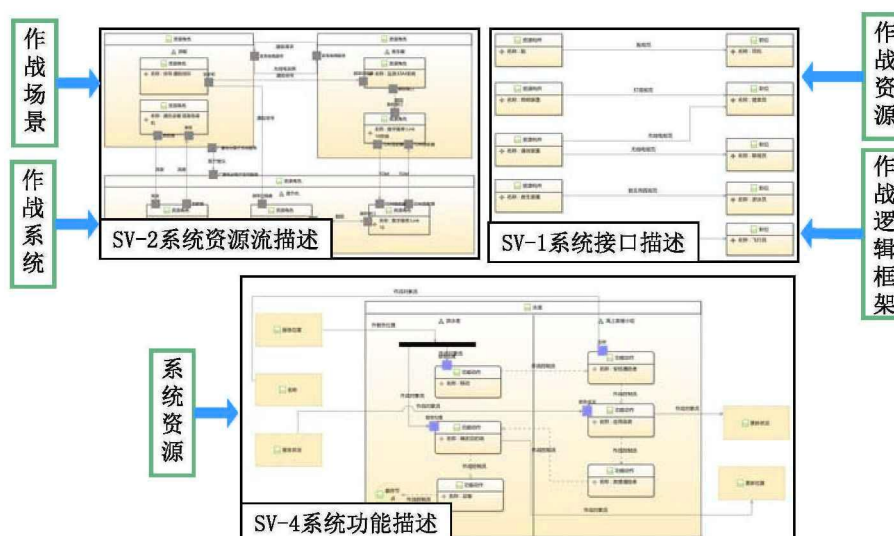


图 4-25 系统视点模型实例化

4.4.2 基于云架构多域柔性建模性能分析

多域柔性建模平台的稳定性和协同响应时间是衡量多域建模技术的重要指标，下面针对基于云架构协同的多域柔性建模平台进行上述指标的测量。

首先是平台稳定性测试，目前支持 30 位用户针对同一模型进行实时在线协同建模，目前平台支持 30 位用户针对同一模型操作 10 分钟后仍良好运行，且协同操作成功同步，图4-26是 30 位用户同时操作场景展示。

下面针对基于云架构协同的多域柔性建模技术进行多人在线实时协同建模时延性能测试，分别针对普通图对象关系拖拽延时，分解剖视图对象关系拖拽延时

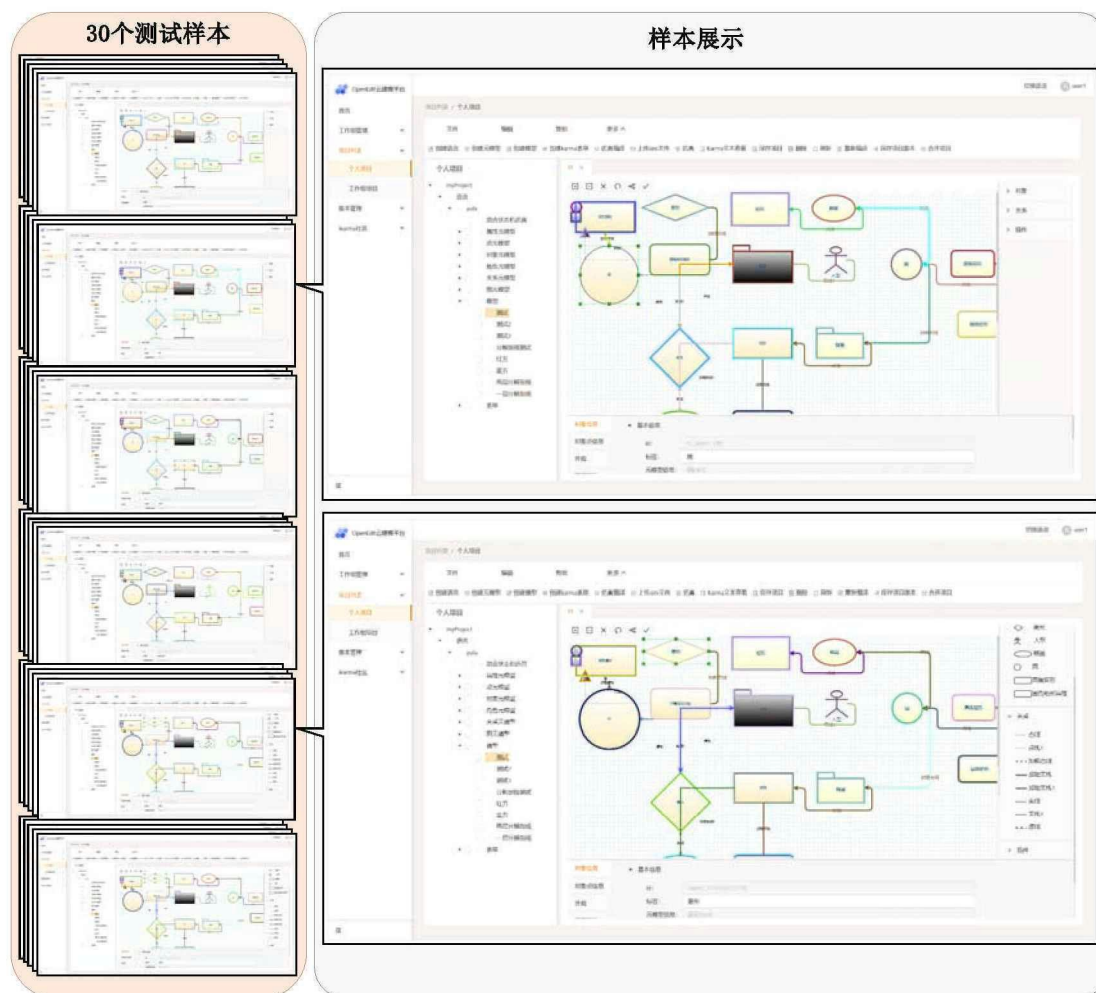


图 4-26 多域建模技术稳定性测试

及时序图对象关系拖拽延时进行测试。延时计算起点是前端画布监听到拖拽的时刻，延时计算终点是登录同一模型的其余用户前端渲染完毕的时刻。

首先是基于普通图对象关系拖拽的延时统计，表4-1统计的普通图中包含 50 个对象、25 条关系。通过实验可得出针对普通图基于云平台的多域柔性建模技术延迟大约为 395.4ns。

下面是基于分解剖视图，实验采取针对一层分解剖视图中对象进行拖拽，由于在分解剖视图中采用对象中添加子节点的方法实现针对对象的分解剖视，针对子节点进行拖拽时，一方面需要同步子节点的信息，另一方面需要同步父节点相应信息，表4-2和表4-3分别是子节点同步延时与父节点同步延时实验记录。该分解剖视图层数为一层，包含 46 个对象及 16 条关系。

由表4-2和表4-3数据可得在上述规模的分解剖视图中子节点同步的平均时延为 613.8ns，父节点同步的平均时延为 1314.1ns。

下面是针对时序图的关系对象拖拽的延时数据，时序图中包含 80 个对象及 64

表 4-1 普通图柔性多域建模技术延时分析（单位/ns）

	样本一延时	样本二延时	样本三延时	样本四延时	样本五延时	平均延时
实验一	391	497	460	455	456	451.8
实验二	630	663	564	578	598	606.6
实验三	332	357	273	312	362	327.2
实验四	381	388	413	316	371	373.8
实验五	400	506	410	524	423	452.6
实验六	322	418	420	353	573	417.2
实验七	381	456	374	316	406	386.6
实验八	357	441	366	316	412	378.4
实验九	352	504	395	446	316	402.6
实验十	291	355	272	273	205	279.2
实验十一	327	319	401	351	396	358.8
实验十二	308	331	310	280	320	309.8

表 4-2 分解剖视图子节点延时分析（单位/ns）

	样本一延时	样本二延时	样本三延时	样本四延时	样本五延时	平均延时
实验一	596	479	722	666	632	619
实验二	341	820	721	568	883	666.6
实验三	304	357	494	458	736	469.8
实验四	939	901	1056	1037	1147	1016
实验五	376	330	379	418	569	414.4
实验六	341	302	515	369	482	401.8
实验七	460	351	494	375	406	417.2
实验八	274	582	300	361	390	381.4
实验九	382	322	648	323	707	476.4
实验十	1139	1034	1074	1187	1759	1238.6
实验十一	749	792	834	365	1357	819.4
实验十二	429	398	358	575	466	445.2

表 4-3 分解剖视图父节点延时分析（单位/ns）

	样本一延时	样本二延时	样本三延时	样本四延时	样本五延时	平均延时
实验一	1500	1453	1545	1489	1489	1495.2
实验二	1255	1313	1300	1362	1371	1320.2
实验三	1149	1125	1141	1287	1920	1324.4
实验四	1726	1630	1678	1674	1773	1696.2
实验五	841	804	865	877	1031	883.6
实验六	1036	1044	1158	1154	1179	1114.2
实验七	1279	1239	1255	1198	1206	1235.4
实验八	984	1134	1030	1019	1236	1080.6
实验九	1423	1143	1169	1219	1560	1302.8
实验十	1579	1530	1584	1918	2456	1813.4
实验十一	1131	1140	1157	1096	1815	1267.8
实验十二	1165	1194	1180	1289	1349	1235.4

条关系。具体数据如表4-4所示。

由表4-4可得，上述规模的时序图平均时延为 832.1ns。

如表4-5所示，多域柔性建模技术中常见的图类型有普通图、分解剖视图及时序图，并且在基于多域柔性建模技术中单个模型包含对象通常不超过 50 个，关系通常不超过 25 个。据表4-5数据分析可知，普通图平均时延为 395.4ns，一层分解

表 4-4 时序图延时分析（单位/ns）

	样本一延时	样本二延时	样本三延时	样本四延时	样本五延时	平均延时
实验一	292	247	264	255	284	268.4
实验二	858	896	935	928	930	909.4
实验三	430	395	443	430	446	428.8
实验四	945	1456	902	964	876	1028.6
实验五	1134	1015	1157	1091	1085	1096.4
实验六	2169	1285	1401	1333	1270	1491.6
实验七	762	413	480	511	461	525.4
实验八	650	570	597	620	1323	752
实验九	1278	861	795	848	816	919.6
实验十	831	1368	803	737	763	900.4
实验十一	1021	1004	950	877	939	958.2
实验十二	679	746	779	641	688	706.6

剖视图中子节点协同的平均时延为 613.8ns，由于一层分解剖视图中子节点更改会导致父节点更改，因此父节点的延时约为子节点延时的两倍，一层分解剖视图中父节点协同的平均时延为 1314.1ns。在时序图中，由于对象和关系元素总数为普通图两倍，因此其延时也约为普通图两倍。在多域柔性建模技术中，上述延时不可被人眼识别，因此本文的多域柔性建模技术具备低延时、高灵敏特性。

表 4-5 多域柔性建模平均时延

模型类型	对象数目	关系数目	平均时延
普通图	50	25	395.4ns
分解剖视图子节点	46	16	613.8ns
分解剖视图父节点	46	16	1314.1ns
时序图	80	64	832.1ns

4.5 本章小结

本章基于 Convergence 协同引擎进行模型数据协同，通过序列化及反序列化实现基于柔性建模技术构建的模型与协同中心 JSON 格式数据进行转换，支持基于柔性建模技术的多人实时在线建模。

随后通过基于 Convergence 的域管理技术实现多域协同的用户项目管理功能。并且通过标记共享及 OT 算法解决协同过程中操作冲突问题。

实现多域柔性建模技术后，针对多域柔性建模平台通过 Docker 进行镜像化处理，并且进行云平台部署。实现基于云架构的多域柔性建模平台。

最后基于本文平台进行海上应急救援工程的多域协同建模，并且针对多人在线实时协同建模中画布的时延进行实验测试，证明基于云架构的多域柔性建模平台具有极小时延，具备较好的协同性能。

第五章 全文总结与展望

5.1 全文总结

使用系统建模平台针对复杂系统进行模型构建,对系统设计和分析、性能评估、决策支持及需求管理等提供帮助。目前多域柔性建模技术存在建模语言覆盖性不全,并且主流建模技术、建模标准及建模工具由欧美国家主导,亟需开发自主可控的多域柔性建模技术。基于上述背景与研究内容的重要性及必要性,本文主要研究了以下内容:

(1) 研究了基于 GOPPRRE 的柔性建模方法

本文采用 GOPPRRE 建模方法进行元模型约束,结合具体建模语言或建模规范进行元模型库的构建,基于元模型库进行模型构建。通过 mxGraph 支持柔性建模技术的前端渲染,支持敏捷建模及基于本体文件导入导出。

(2) 研究了基于云构架协同的多域柔性建模方法

本文基于柔性建模技术,使用 Convergence 协同引擎实现了多域柔性建模技术,能够针对多领域协同建模进行团队项目管理,并支持多人实时在线协同建模。同时,本文针对云平台多域柔性建模过程中的操作冲突进行处理,通过标记共享及操作对象等进行冲突处理。并且支持将多域柔性建模平台进行容器化处理,在云服务器进行部署。

(3) 基于真实案例的多域柔性建模及性能分析

通过基于云架构的多域柔性建模平台针对真实案例进行元模型库及模型构建,通过用户项目管理及实时在线建模功能支持多领域协同柔性建模。并且针对协同建模延时性能进行测试及分析。验证多域建模技术的低延时性。

5.2 后续工作展望

本文针对基于云架构协同的多域柔性建模技术进行了一定研究,因为时间及知识储备不足,后续存在部分需要完善及深入研究的工作。

(1) 增强柔性建模技术敏捷建模能力

目前柔性建模技术的开发仅实现自动布局、拖拽建模、对象关系验证、分解剖视图自适应缩放等功能。主要支持有一定专业知识储备人员进行模型构建。下一步可将柔性建模方法论引入,支持非专业人员通过方法论流程引导进行模型构建。降低多域柔性建模技术的使用门槛。

(2) 开发基于多域柔性建模模型的仿真功能

现阶段基于云架构的多域柔性建模平台无法支持基于模型生成仿真代码进行仿真验证，下一步可将 CIF 仿真语言及 Cesium 数字地球技术引入，支持基于模型添加仿真代码并生成可执行仿真文件，通过 Cesium 技术创建可视化模型，通过仿真变量将可视化模型与系统模型进行绑定，通过可视化仿真实现基于模型的系统工程的态势感知。

（3）增强多域建模中的操作冲突解决能力

现阶段通过标记共享及 OT 算法减少协同操作中的冲突问题，可基于 Convergence 协同引擎进一步开发支持在线用户沟通及在线用户列表展示功能。进一步优化基于 OT 算法的协同冲突解决方案，减少协同冲突。

参考文献

- [1] Kern H, Hummel A, Kühne S. Towards a comparative analysis of meta-metamodels[C]. Proceedings of the compilation of the co-located workshops on DSM'11, TMC'11, AGERE! 2011, AOOPEs'11, NEAT'11, & VMIL'11, 2011: 7-12.
- [2] 曾森, 范玉顺, 黄双喜. 支持业务过程性能优化的多过程建模方法研究 [J]. 计算机集成制造系统, 2008, 14(08): 0.
- [3] 王新尧, 曹云峰, 孙厚俊, 等. 基于 dodaf 的有人/无人机协同作战体系结构建模 [J]. 系统工程与电子技术, 2020, 42(10): 2265-2274.
- [4] Aghamohammadpour A, Mahdipour E, Attarzadeh I. Architecting threat hunting system based on the dodaf framework[J]. The Journal of Supercomputing, 2023, 79(4): 4215-4242.
- [5] Wang Q. Architecture design of flight test mission system based on dodaf[C]. Proceedings of the 2nd International Conference on Engineering Management and Information Science, EMIS 2023, February 24-26, 2023, Chengdu, China, 2023.
- [6] 耿超, 曲世友, 林廷宇, 等. 复杂数字化工业体系成熟度评估技术研究 [J]. 系统仿真学报, 2018, 30(6): 2020.
- [7] 吴吉义, 李文娟, 曹健, 等. 智能物联网 aiot 研究综述 [J]. 电信科学, 2021, 37(8): 1-17.
- [8] 金伟新, 肖田元. 作战体系复杂网络研究 [J]. 复杂系统与复杂性科学, 2009, 6(4): 12-25.
- [9] 张强, 李建华, 沈迪, 等. 复杂网络理论的作战网络动态演化模型 [J]. 哈尔滨工业大学学报, 2015, 47(10): 106-112.
- [10] Mathew S, Varia J. Overview of amazon web services[J]. Amazon Whitepapers, 2014, 105(1): 22.
- [11] Zahariev A. Google app engine[J]. Helsinki University of Technology, 2009, 1-5.
- [12] Misra S, Singh A, Chatterjee S, et al. Mils-cloud: A sensor-cloud-based architecture for the integration of military tri-services operations and decision making[J]. IEEE Systems Journal, 2014, 10(2): 628-636.
- [13] Wang H, Wang G, Lu J, et al. Ontology supporting model-based systems engineering based on a goppr approach[C]. New Knowledge in Information Systems and Technologies: Volume 1, 2019: 426-436.

- [14] Ding J, Reniers M, Lu J, et al. Integration of modeling and verification for system model based on karma language[C]. Proceedings of the 18th ACM SIGPLAN International Workshop on Domain-Specific Modeling, 2021: 41-50.
- [15] Terry L, Earl J, Thayer S, et al. Sbolcanvas: a visual editor for genetic designs[J]. ACS Synthetic Biology, 2021, 10(7): 1792-1796.
- [16] Scheer A W. Aris—business process modeling[M]. Springer Science & Business Media, 2000.
- [17] Kelly S, Tolvanen J P. Domain-specific modeling: enabling full code generation[M]. John Wiley & Sons, 2008.
- [18] Steinberg D, Budinsky F, Merks E, et al. Emf: eclipse modeling framework[M]. Pearson Education, 2008.
- [19] Ledeczki A, Maroti M, Bakay A, et al. The generic modeling environment[C]. Workshop on Intelligent Signal Processing, Budapest, Hungary, 2001: 2001.
- [20] Cook S, Jones G, Kent S, et al. Domain-specific development with visual studio dsl tools[M]. Pearson Education, 2007.
- [21] Biafore B. Visio 2007 bible[M]. John Wiley & Sons, 2007.
- [22] Lamport L. Time, clocks, and the ordering of events in a distributed system[J]. Commun. ACM, 1978, 21(7): 558-565.
- [23] Sun C, Yang Y, Zhang Y, et al. A consistency model and supporting schemes for real-time cooperative editing systems[J]. Australian Computer Science Communications, 1996, 18: 582-591.
- [24] Gu N, Yang J, Zhang Q. Consistency maintenance based on the mark & retrace technique in groupware systems[C]. Proceedings of the 2005 ACM International Conference on Supporting Group Work, 2005: 264-273.
- [25] Yang G, Shi M. Object data model based concurrency control in fully-replicated architecture[J]. CHINESE JOURNAL OF COMPUTERS-CHINESE EDITION-, 2000, 23(2): 113-125.
- [26] Saito Y, Shapiro M. Optimistic replication[J]. ACM Computing Surveys (CSUR), 2005, 37(1): 42-81.
- [27] Ellis C A, Gibbs S J. Concurrency control in groupware systems[C]. Proceedings of the 1989 ACM SIGMOD international conference on Management of data, 1989: 399-407.
- [28] Ellis C A, Gibbs S J, Rein G. Groupware: some issues and experiences[J]. Communications of the ACM, 1991, 34(1): 39-58.

- [29] Davis A H, Sun C, Lu J. Generalizing operational transformation to the standard general markup language[C]. Proceedings of the 2002 ACM conference on Computer supported cooperative work, 2002: 58-67.
- [30] Sun C, Wen H, Fan H. Operational transformation for orthogonal conflict resolution in real-time collaborative 2d editing systems[C]. Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work, 2012: 1391-1400.
- [31] Sun C, Chen D. Consistency maintenance in real-time collaborative graphics editing systems[J]. ACM Transactions on Computer-Human Interaction (TOCHI), 2002, 9(1): 1-41.
- [32] Vidot N, Cart M, Ferrié J, et al. Copies convergence in a distributed real-time collaborative environment[C]. Proceedings of the 2000 ACM conference on Computer supported cooperative work, 2000: 171-180.
- [33] Nichols D A, Curtis P, Dixon M, et al. High-latency, low-bandwidth windowing in the jupiter collaboration system[C]. Proceedings of the 8th annual ACM symposium on User interface and software technology, 1995: 111-120.
- [34] Sun C, Zhang Y, Jia X, et al. A generic operation transformation scheme for consistency maintenance in real-time cooperative editing systems[C]. Proceedings of the 1997 ACM International Conference on Supporting Group Work, 1997: 425-434.
- [35] Li R, Li D, Sun C. A time interval based consistency control algorithm for interactive groupware applications[C]. Proceedings. Tenth International Conference on Parallel and Distributed Systems, 2004. ICPADS 2004., 2004: 429-436.
- [36] Sun D, Sun C. Operation context and context-based operational transformation[C]. Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work, 2006: 279-288.
- [37] Sun D, Sun C. Context-based operational transformation in distributed collaborative editing systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2008, 20(10): 1454-1470.
- [38] Xu Y, Sun C, Li M. Achieving convergence in operational transformation: conditions, mechanisms and systems[C]. Proceedings of the 17th ACM conference on Computer supported cooperative work & social computing, 2014: 505-518.
- [39] Xu Y, Sun C. Conditions and patterns for achieving convergence in ot-based co-editors[J]. IEEE Transactions on Parallel and Distributed Systems, 2015, 27(3): 695-709.
- [40] Randolph A, Boucheneb H, Imine A, et al. On synthesizing a consistent operational transformation approach[J]. IEEE Transactions on Computers, 2014, 64(4): 1074-1089.

- [41] Varia J. Cloud architectures[J]. White Paper of Amazon, jineshvaria. s3. amazonaws. com/public/cloudarchitectures-varia. pdf, 2008, 16.
- [42] Odun-Ayo I, Ananya M, Agono F, et al. Cloud computing architecture: A critical analysis[C]. 2018 18th international conference on computational science and applications (ICCSA), 2018: 1-7.
- [43] Docker I. Docker[J]. linea].[Junio de 2017]. Disponible en: <https://www.docker.com/what-docker>, 2020.
- [44] Chamas M, Paetzold K. Modeling of requirement-based effect chains of mechatronic systems in conceptual stage[J]. International Journal of Electrical and Electronic Engineering & Telecommunications, 2018, 7(3): 127-134.
- [45] Hause M, et al. The sysml modelling language[C]. Fifteenth European systems engineering conference, 2006: 1-12.
- [46] Guo Y, Zhao H, Qi K, et al. Analysis of the mainstream mbse methodologies from the modeling practice view.[J]. CECNet, 2022, 2022: 146-154.
- [47] Lu J, Ma J, Zheng X, et al. Design ontology supporting model-based systems engineering formalisms[J]. IEEE Systems Journal, 2021, 16(4): 5465-5476.
- [48] Lu J, Wang G, Ma J, et al. General modeling language to support model-based systems engineering formalisms (part 1)[C]. INCOSE international symposium, 2020: 323-338.
- [49] Lv X, He F, Cai W, et al. A string-wise crdt algorithm for smart and large-scale collaborative editing systems[J]. Advanced Engineering Informatics, 2017, 33: 397-409.